A discrete portfolio learner: what it is, how it compares with gradient-trained networks, and what it learns

Claude (Anthropic)*
prompted by Kaarel Hänni

28 October 2026

Abstract

This paper describes v3 (`hyb_portfolio3`), the best learner so far of a study that compares discrete learners with gradient-descent-trained networks at matched FLOPs, and tests it on data fixed before it was designed. v3 is a stacked portfolio whose members are chosen by input type. On Boolean inputs the main member is DFG, which enumerates Fourier characters and indicator features of signed sums of inputs, admits each one by a union-bound test against the current residual, and reads the admitted features out linearly. On images the members that carry most stacking weight at $n \geq 10^4$ are gradient-boosted trees over fixed subset sums of pixels, blobs and short "bars" whose pixel sets are chosen from pixel correlations, so the pixel layout is never read. No member learns features by gradient descent; the continuous fits are convex linear readouts (including a logistic regression on the inputs), tree leaf values and stacking weights. On a fresh 12-target Boolean suite ($d = 30$), drawn from a family written in Part 9 by an agent blind to the Part 9 design (v3's toy component was developed on two other draws of the same family), v3 needs clearly less data than the per-target best of three pre-registered network variants on 10 targets and more on none; its predecessor v2 does the same (v3/v2 mean $\log_2$ ratio -0.004). The mean $\log_2$ ratio of the training-set sizes at 10% test error is -1.50, about $2.8\times$ less data. On five of the ten, default gradient boosting or logistic regression also beats the networks, and one target, a residue class mod 3 of a 14-input count, is learned by no learner. On two fresh 10-class QuickDraw sets with permuted pixels, v3 has 12.8% and 12.6% test error at $n = 60000$. It is clearly better than an ensemble of three MLPs at all six cells ($n = 1000$, 10^4, 6×10^4; errors 17–23% lower) and than an MLP given v2's pooled features at both cells where that ran. But it is only 8.6–9.6% below a CPU-matched ensemble of wider MLPs, a tie by the pre-registered rule. Against v2 it ties all six cells, although its error is lower at each. Descriptive interpretability analyses (seed 0, independently verified) show what the fits contain. DFG's model comes to hold the target's exact signed block on 49 of 54 sum-structured targets. On the 25 targets that are functions of one signed sum, its test would admit the block at $n = 128$ every time, and the data needed beyond that grows as the target's share of variance at Fourier degrees 1–2 shrinks (Spearman -0.86 over those 25). On the three mod-3 targets the search never proposes the block, consistent with their near-zero weight at those degrees ($W_1 + W_2 \leq 0.0002$; zero for two of them), although at $n = 16384$ the true block would pass the test at 134–155 times the threshold. On QuickDraw and MNIST the pooling sets are local neighbourhoods of the true pixel grid, and the bar member that v3 adds has the lowest out-of-fold error of all members in every analysed fit. The limitations are as large as the results. The held-out sets are small: 12 targets from one

*Written by an AI system in a series of sessions prompted and steered by Kaarel Hänni. Not reviewed by a human. The interpretability analyses were checked by independent AI verifiers, and the paper's claims by independent AI checkers against the result files and records, as described in Appendix A; the comparisons were not re-audited for leakage, compute or reference strength. Code, result files, the pre-registration and the underlying records are on the `claude/nifty-mccarthy-j0lw2a` branch of https://github.com/kaarelh/AI-works, folder `gd-circuit-growth/`.

family, and two image sets from one source with one seed at $n = 60000$. v3 is a portfolio, not one mechanism. Its image members recover the pixel locality that the protocol withholds, and a small CNN that sees the layout (one seed) has lower error at all three cells where both have run: clearly at `qd_a`, $n = 10^4$ (12.00% against 14.22%), and within the tie band at `qd_c`, $n = 10^3$ and 10^4 (24.36% against 26.58%, 14.84% against 16.17%). A fit takes 23–24 CPU minutes at $n = 60000$. On tabular data v3 is a classical stack whose margin over the hindsight-best MLP is not significant (lower mean error on 8 of 10 fresh datasets, sign test $p = 0.11$).

Contents

1 Introduction **4**

2 What kind of learner it is **6**
2.1 In one paragraph . . . 6
2.2 The portfolio and its budget . . . 7
2.3 DFG v1.3 on Boolean inputs . . . 8
2.4 The image members . . . 9
2.5 What is discrete, and what is not . . . 10
2.6 How v3 came about . . . 11

3 How it is evaluated **11**
3.1 Tasks . . . 11
3.2 References . . . 12
3.3 Metrics, verdicts and budgets . . . 13
3.4 How v3 was built and what is held out for it . . . 13

4 How well it does: toy targets and tabular data **15**
4.1 The held-out suite `dlheld7` . . . 15
4.2 Every toy suite . . . 17
4.3 What changed from v2 to v3 . . . 19
4.4 Targets nobody learns, and where v3 is not better . . . 21
4.5 Tabular data . . . 22

5 How well it does: images **24**
5.1 The held-out sets `qd_c` and `qd_d` . . . 25
5.2 The other image sets: development data for v3 . . . 28
5.3 Where the gain over v2 comes from . . . 30
5.4 Compute . . . 31

6 What it learns **33**
6.1 Toy targets: what DFG holds, and when . . . 34
6.2 Images: what the boosters split on . . . 43

7 Discussion **52**
7.1 What kind of learner this turned out to be . . . 52
7.2 Why it works where it works . . . 52
7.3 Limits . . . 53
7.4 What would test it further . . . 55

A How Part 10 was run **57**
A.1 Protocol and builders 58
A.2 The integration and best-learner decisions 59
A.3 Disclosures and notes 59
A.4 The interpretability workflow 60
A.5 This paper and reproducibility 62

1 Introduction

The question. Can a learner without continuous feature parameters do at least as well as gradient-descent-trained networks, with the same training data and no more FLOPs? This paper reports the latest learner of a ten-part study, whose Parts 6–10 have asked this question, and what that learner does.

The study so far. Each part answered a question posed by the user who prompted the work. From Part 6 on, each part froze its learners before running them on the held-out data that it reports as its main evidence.

- *Parts 1–5* [6] trained rich-regime networks on Boolean targets and described what gradient descent does there. It grows features greedily. A feature's cost is set by how hard its correlation with the current residual is to detect against the sampling noise of the competing directions, and features on inputs already in use are cheaper. The price is paid in samples. For a k-bit parity on d inputs, the networks' sample size at 10% test error, n^*, grows like $d^{1.0}$, $d^{1.9}$ and $d^{2.4}$ for $k = 2, 3, 4$. That is close to the leap and correlational-SQ rate d^{k-1} [1, 2] for $k \leq 3$, somewhat below it for $k = 4$, and far from the $\approx k \log d$ of an Occam learner.
- *Part 6* built DFG (discrete feature growth) from that picture [5]. DFG enumerates Fourier characters and admits each by a union-bound test against the residual. On a blind 7-target suite it needed less data than the networks on every target (median factor 3.1), with at most 0.26 of one network run's FLOPs per fit.
- *Part 7* ran three adaptations of DFG on standard tasks. They were about level with a plain MLP on tabular data, behind a regularized one, and clearly behind on images at large n.
- *Part 8* stacked DFG with LightGBM, ExtraTrees, nearest-neighbour votes and logistic regression (v1). v1 needed clearly less data than the best network variant on 9 of 10 fresh toy targets. Its mean error was lower than the hindsight-best MLP's on 8 of 10 fresh tabular datasets (sign test $p = 0.11$). On images it was behind a regularized MLP ensemble at $n \geq 30000$.
- *Part 9* added sum blocks to DFG and permutation-invariant image members to the stack (v2). On a 12-target suite written blind to the sum-feature design, v2 needed clearly less data than the best pre-registered network variant on 10 targets and more on none. On two fresh QuickDraw sets it beat a strong MLP ensemble at $n \geq 10000$. But a CPU-matched ensemble of wider MLPs and an MLP given v2's own pooled features tied it at $n = 60000$, and a small CNN that sees the pixel layout was clearly better at $n = 10000$.

Part 10, and what this paper adds. The user asked for "a couple more hours" on a better discrete learner, then for interpretability of the best learner so far, and then for a paper about it that explains what kind of learner it is, how well it does against the baselines, and what the interpretability shows. Part 10 pre-registered the whole sequence (Section 3.4). Three independent builders worked on development data only. A rule fixed in advance chose two of their components, a new sum-feature search for DFG and a second image booster on "bar" features. The assembled learner v3 was frozen before any of its held-out results were read, and a second rule fixed in advance made v3, not v2, the subject of this paper. The paper contributes four things:

1. an explanation of what v3 is and what in it is discrete (Section 2);
2. a held-out test of v3 on a fresh toy suite and two fresh image sets, against the pre-registered references (Sections 4.1 and 5.1);

3. a comparison of v3 with every reference that was run, on every toy suite and every real dataset of the study except MNIST (where v3 enters only the interpretability analysis), with the status of each set for v3 stated (Sections 4 and 5);

4. a descriptive analysis of what v3's fitted members contain, checked by independent verifiers (Section 6).

What was found.

- **Held-out toy suite** [Numerical] (Section 4.1). The suite is `dlheld7`, 12 targets drawn from the blind family of Part 9. The family was blind to Part 9's design, not to v3's: the Part 10 toy builder developed on two other draws of it, `dlheld6` and `dlfamdev`. v3's mean $\log_2 n^*$ ratio against the per-target best of three pre-registered network variants is -1.50 over the 11 targets both learn, about 2.8× less data. v3 is clearly better on 10 targets and clearly worse on none. One target is a tie (a halfspace), and one is learned by no learner (a residue class mod 3 of a 14-input count). On five of the ten wins, default gradient boosting or logistic regression already beats the networks, and there v3 leads them by small factors only. The Part 10 toy component changes nothing measurable on this suite: against v2 the mean is -0.004.

- **Every toy suite** [Numerical] (Section 4.2). On the 10 suites with network results (all but `dlfamdev`), v3's suite means against the per-suite best network range from -1.50 to -2.44, and a network is clearly better on one target only (h6_00, a residue class mod 3 that only the most regularized network learns, at $n^* = 12304$). Against the one network variant run on every suite, the margin is smallest on the two draws of the blind family, though by differences within noise. These suites are development data for v3, except `dlheld7`.

- **Held-out image sets** [Numerical] (Section 5.1). On `qd_c` and `qd_d`, v3's test error at $n = 60000$ is 12.76% and 12.64%. It is clearly better than the three-network MLP ensemble `mlp_strong` at all six cells (errors 17–23% lower), than gradient boosting and v1 at all six, and than `mlp_pool`, an MLP given v2's pooled features, at both cells where that ran. It is clearly better than the 512-wide ensemble at five cells and ties it at the sixth. It ties the CPU-matched 1024-wide ensemble at both of its cells (8.6% and 9.6% lower error). It ties v2 at all six cells, with lower error at each (2.2–9.6%).

- **Tabular data** [Numerical] (Section 4.5). On 24 of the 26 datasets v3 runs v1's code, unchanged since Part 8. On the 10 datasets fixed in Part 8 its mean $\log_2$ error ratio against the hindsight-best MLP is -0.168, lower on 8 of 10 datasets ($p = 0.11$).

- **What DFG learns** [Numerical] (seed 0; Section 6.1). On 49 of 54 sum-structured targets, DFG's model comes to hold the target's exact signed blocks, at a median n of 256. On the 25 targets that are functions of one signed sum, DFG's test would admit the true block at $n = 128$ every time. The extra data needed to find it grows as the target's Fourier weight at degrees 1–2 shrinks (Spearman $\rho = -0.86$). The three residue classes mod 3 have almost no weight there ($W_1 + W_2 \leq 0.0002$; exactly zero for h6_00 and f7_05), and the search never proposes their block, although at $n = 16384$ it would pass the test at 134–155 times the threshold.

- **What the image members encode** [Numerical] (Section 6.2). The pooling sets are local neighbourhoods of the true pixel grid on QuickDraw and MNIST (98.3–99.9% of the other members of a 4-pixel set lie within one step on QuickDraw), and mirror-image pairs at some pixels of Fashion-MNIST. The bar booster that v3 adds has the lowest out-of-fold error of all members in every analysed fit, and in the analysed QuickDraw and MNIST fits it holds most

of the boosters' stacking weight (shares 0.32–0.56 against `ggbt`'s 0–0.09; v2's `ggbt` had 0.41 at `qd_dev`, $n = 10^4$).

- **What kind of learner it is** [Heuristic] (Section 7). v3 is three learners under one stacker, chosen by input type. On Boolean targets it is enumerate-and-test, at about an Occam sample cost wherever its search can propose the right feature. On images it is a fixed hierarchy of data-derived subset sums read out by gradient-boosted trees. On other tabular data it is a classical stack.

Scope.

- *What is held out.* For v3 as a whole only three sets are held out: `dlheld7`, `qd_c` and `qd_d` (Table 3). They were fixed in a commit before any Part 10 design work, and no result of a Part 10 learner on them was read before v3 was frozen. The earlier held-out suites and image sets were development data in Part 10 or shaped earlier versions, so they measure v3 but do not test it. On tabular data the cleaner claim is about v1's code path, which v3 runs there unchanged.
- *The references.* They are concrete networks with standard recipes. On the toy suites they are μP two-hidden-layer MLPs of width 128 trained by Adam, in up to five loss and weight-decay settings. On images they are two-hidden-layer MLPs of width 256–1024, up to ensembles of three with weight decay and dropout. There is no augmentation and no architecture search. The per-target and per-cell minima `nets_best` and `mlp_best` are hindsight choices that favour the references. A small CNN on the unpermuted pixels is reported as context, not as a reference.
- *What "discrete" means.* No member fits a continuous feature direction by gradient descent. Features are enumerated, built from data statistics such as correlation ranks, and admitted by tests or by greedy split search. The continuous fits are linear readouts (including a logistic regression on the inputs), tree leaf values and stacking weights. Gradient boosting is functional gradient descent over trees, and two components are borderline (Section 2.5). The fair wording is "compared with gradient-descent-trained networks", not "compared with gradient descent".
- *Power.* The held-out sets are 12 toy targets from one family and two image sets from one source, with one seed at $n = 60000$.

Claims are labelled [Numerical] (measured here, with the source file named) or [Heuristic] (a reading or a back-of-envelope argument, not tested here). There are no theorems in this paper.

Organisation. Section 2 explains the learner and Section 3 the tasks, references, metrics and the pre-registration. Section 4 gives the toy and tabular results and Section 5 the image results. Section 6 reports the interpretability analyses, and Section 7 discusses what the results mean and where they stop. Appendix A records how Part 10 was run.

2 What kind of learner it is

2.1 In one paragraph

v3 (`hyb_portfolio3`) is a *stacked portfolio*. Each member produces class probabilities. A convex stacker, fitted on the members' out-of-fold log-probabilities, combines them. Which members run is decided by the input type, n and the budget.

member	what it is	when it runs
`dfg`, `dfg_forced`	DFG v1.3 (Section 2.3); `dfg_forced` is its small-n variant, which forces the best candidate when nothing passes	2 classes, every column binary or categorical, ≤ 256 bits; `dfg_forced` at $n \leq 512$
`ggbt`	LightGBM on [x, pooled subset sums $p_i^{(k)}(x) = \sum_{j \in S_i(k)} x_j$ for $k = 4, 9, 16, 300$ prototype-pair projections] (Section 2.4)	images (every column continuous, $d \geq 256$)
`cgbt`	LightGBM on [x, the same pooled sums, bar sums $\sum_{j \in \text{bar}} x_j$ for bar half-lengths 2 and 4]	images
`gknn_k`	k-NN votes ($k \in \{1, 3, 9, 27\}$) on the pooled sums at $k = 9$	images
`gbt`, `et`	`gbt`: LightGBM (100 rounds) on the raw columns (categoricals as categories; rows 0.5, columns 0.25 per tree when $nd > 10^7$); `et`: ExtraTrees (100 trees) on the standardized encoding	always
`knn_<metric>_k`	k-NN votes on standardized columns (and on a common scale when column scales differ)	always
`lin_C`	multinomial L2 logistic regression on the standardized encoding, $C \in \{0.05, 1\}$ ($C = 1$ only for $n > 10^4$)	always

Table 1: The members of v3. On toy inputs the image members do not run. On images, DFG does not run. Any member is skipped when the budget guard (Section 2.2) says so.

- *On Boolean inputs* (the toy suites, and binary tabular data with two classes), the member that matters is *DFG*: discrete feature growth. DFG enumerates a structured dictionary of discrete features (Fourier characters $\chi_S(x) = \prod_{i \in S} x_i$ and indicator functions of integer signed sums $\sum_{i \in S} s_i x_i$). It admits a feature only when that feature's correlation with the current residual passes a statistical test, with a union bound over the whole class the feature was drawn from. It then reads the admitted features out by least squares, refined by logistic regression.
- *On images*, the members that carry most stacking weight at $n \geq 10^4$ are *gradient-boosted trees over a fixed bank of subset sums of pixels* (at $n = 1000$, k-NN votes on the same sums carry more). The subsets are built from the training data alone: blobs of the pixels most correlated with each pixel, and short "bars" along lines recovered from the correlations. The pixel layout is never read.
- *Everywhere*, LightGBM, ExtraTrees, k-nearest-neighbour votes and multinomial logistic regression run as well.

No member fits a continuous feature direction by gradient descent. The fitted continuous quantities are readout weights (including those of a logistic regression on the inputs), tree leaf values and stacking weights. Two components are borderline, and they are discussed in Section 2.5.

2.2 The portfolio and its budget

Stacking [27]. Let $L_m(x) \in \mathbb{R}^C$ be member m's out-of-fold log-probabilities. The score is

$$s_c(x) = b_c + \sum_m w_m L_m(x)_c, \qquad w_m \geq 0,$$

fitted by L2-penalized multinomial log-loss, which is convex. Out-of-fold predictions come from 10-fold cross-validation for $n \leq 500$, 5-fold for $n \leq 3000$, and one stratified holdout of $\min(n/5, 6000)$ points above that. Leave-one-out votes serve for k-NN and out-of-bag votes for ExtraTrees. Members

with positive weight are refitted on all n, except the boosters on inputs with $nd > 10^7$, where the holdout-trained booster is kept.

Budget. The study compares learners at matched compute.

- *Toy fits:* one training run of the study's networks, 1.6×10^{11} FLOPs at $d = 30$.
- *Real-data fits:* three training runs of the reference MLP, 1.2–6.2×10^{12} FLOPs on the image sets, depending on n.

Every member's FLOPs are counted: the image boosters by the compute audit's per-tree formula, and Part 8's `gbt` member by its original formula, which Part 9's audit found low by 1.14–1.46× at $n = 3000$. A member is skipped when its projected cost would take the total above 90% of the budget (95% for logistic regression). DFG's out-of-fold fits share 35% of the budget and its final fits at most 50%. Prediction is not charged. CPU time per fit is reported alongside.

2.3 DFG v1.3 on Boolean inputs

DFG works with the residual $r = y - \hat{y}$ of a least-squares fit of the ± 1 labels on the current features, and with $s = \mathrm{rms}(r)$. It runs in rounds. Each round searches several candidate classes and admits everything that passes its test. The fit is then redone, and the next round starts from the new residual. Training stops when a round admits nothing, when every training sign is right, after 8 rounds, when the feature count reaches $n/2$, or once 75% of DFG's budget share is spent.

Characters. The candidates are every χ_S with $|S| \le 4$ over all d inputs (degree 3 after the first round; degrees 5–6 on a budget-limited subsample), extensions $S \cup \{j\}$ and $S \cup \{j, l\}$ of admitted characters, and every subset of the at most 12 inputs already in use (a Walsh transform). A character from a class of size N is admitted when

$$\left| \tfrac{1}{n} \textstyle\sum_t r_t \chi_S(x_t) \right| \;>\; s\sqrt{2\ln(4KN/\delta)/n}, \qquad K = 4,\ \delta = 0.1. \tag{1}$$

The small classes (extensions and local subsets) have low thresholds. Features on inputs already in use are therefore cheap: the discrete counterpart of the composition effect seen in the study's networks.

Sum blocks. A block is a subset S with signs $s_i \in \{\pm 1\}$ and the integer statistic $t(x) = \sum_{i \in S} s_i x_i$, or $|t(x)|$ when the block is *folded*. The training values of t are cut into contiguous groups of at least $\max(3, 0.005n)$ points. The block's features are the group indicators $[t(x) \in \text{group } v]$. Every function of t, so every symmetric function of the signed inputs in S, is a linear combination of them, up to merged rare values. Four variants extend this:

- *modulated* blocks multiply the indicators by an admitted character χ_T, with $T \cap S = \emptyset$;
- *product* blocks take the joint cells of two blocks on disjoint supports;
- *split* blocks replace a mixed-sign candidate by the product block of its positive-sign part and a block searched on its negative-sign part (the joint cells of t_A and t_B);
- v1.3's *weighted* blocks use $u = t + a\,\tau(x)$ with $a \in \{\pm 2, \pm 3\}$ and τ a literal, a parity of two inputs or an AND of two literals.

A block is admitted when the residual's projection on its indicators passes a χ^2 tail bound [17] with a union bound over its whole class:

$$T = \sum_v n_v \bar{r}_v^2/\hat{\sigma}^2 \;>\; m + 2\sqrt{m\,\tau_k} + 2\tau_k, \qquad \tau_k = \ln\Big(4K\tbinom{d}{k}2^{k-1}\cdot 2\,(1+\#\text{modulators})\,k_{\max}/\delta\Big), \quad (2)$$

where m is the number of occupied groups minus one and $k_{\max} = 24$. Weighted blocks pay a further $\ln(2(4d + 20\binom{d}{2})) \approx 9.8$ nats for their inner term. The bound covers every (S, s) of every size, both fold variants and every modulator, so the adaptive search does not invalidate it. It is not free: at $d = 30$ no 7-input block can pass below $n \approx 67$, and below $n \approx 75$ with three or more value groups.

The block search uses only the training residual. Candidates come from five sources:

- prefixes of the inputs ordered by the top eigenvectors of the residual pair matrix $\overline{r\,xx^\top}$ (an even symmetric function of t makes its restriction to S rank one with sign pattern s);
- prefixes ordered by the degree-1 coefficients $\overline{r\,x_i}$;
- greedy growth from the 3 strongest pairs and the 2 strongest single inputs;
- in the additive search, v1.3's weighted candidates: weighted eigenvector prefixes, an inner term on the strongest pair, and extensions of the two best unweighted candidates;
- at $n \le 128$, in the first additive search of a round (v1.3), a width-32 beam over (S, s).

The best candidates are refined by single add, remove and sign-flip moves, then tested on all n. A block must also pass the test on the residual after the round's admitted characters are projected out.

Readout and small n. DFG predicts the sign of the least-squares fit on its features, refined by 15 Newton steps of L2-regularized logistic regression when $n \ge 8p$. It adds one Hebbian step on the final residual over all d inputs, a "kernel background". With few points the union bound passes nothing. The forced variant then takes the best character, or (v1.3, $n \ge 48$) the best block, anyway. The stacker decides by cross-validation whether to trust it.

Why this is sample-efficient where it works [Heuristic]. With ρ the correlation of a feature with the residual in units of s, the test (1) admits it and no null competitor once $n \gtrsim 2\ln(4KN/\delta)/\rho^2$. For a k-sparse character $N \le d^k$, so the sample cost is about $2k\ln d/\rho^2$: the Occam rate. Enumeration costs about nN FLOPs, which at $d = 30$ fits inside one network run up to degree 4. DFG is in effect a greedy, budgeted, test-gated version of the classical low-degree algorithm [20], with a second dictionary for symmetric functions of signed sums. Gradient-trained networks pay the other way round, close to d^{k-1} samples for a k-parity with $k \le 3$ (somewhat fewer at $k = 4$) [2, 6].

Where it cannot work. The test bounds detection, but the search must also *propose* the right candidate. Block candidates are seeded by degree-1 and degree-2 statistics of the residual. A target whose block has almost no Fourier weight at degrees 1 and 2 is invisible to the seeds at every n of the study's grid (≤ 16384); for h6_00 and f7_05 the seeds' statistics are zero in expectation. A residue class mod 3 of a 10–14-input count is such a target (Section 6.1).

2.4 The image members

The comparator on images is an MLP, which is invariant to a fixed permutation of its input pixels. So the image members may not use the pixel layout, and the QuickDraw sets are stored with their columns permuted. The members recover what they need of the layout from the training data.

object	how it is chosen	continuous parameters?
characters χ_S, blocks $(S, s, \text{fold}, a, \tau,$ modulator)	enumeration, greedy or beam search, statistical tests	no
group boundaries of a block, histogram bins	training quantiles / counts	no
pooling sets, bars	correlation ranking of pixels on training rows	no
DFG readout weights	least squares, then logistic Newton steps	yes (linear readout)
logistic regression `lin`: weights on the standardized inputs	convex log-loss (L-BFGS)	yes (linear readout on the inputs)
tree splits; leaf values	greedy split search on histograms; Newton steps (boosting)	leaf values
stacking weights	convex log-loss	yes (≤ 30 weights)
DFG's spectral features $(u \cdot x)^2$	top eigenvectors of the residual pair matrix	borderline (directions u)
`ggbt`'s prototype pairs $x_a - x_b$	two random training points of different classes	borderline (directions from data)

Table 2: What v3 chooses discretely and what it fits continuously.

Pooling sets. For every non-constant column i, $S_i(k)$ is the set of the k columns with the largest correlation with column i on the training rows (i itself first). Ties are broken by a column fingerprint, never by index, so permuting the columns permutes the sets exactly. The pooled feature is the subset sum $p_i^{(k)}(x) = \sum_{j \in S_i(k)} x_j$. On images the most correlated pixels are the nearest ones, so $S_i(k)$ is in effect a small blob around pixel i (Section 6.2).

Bars (v3). The 8 most correlated pixels of i are paired greedily into 4 axes, least correlated pair first. On a grid, opposite neighbours are the least correlated pair. A bar of half-length L is i, the two ends of an axis, and $L - 1$ further steps in each direction. Each step goes "straight on": from j, reached from i', it moves to the partner of i' in j's own pairing. A bar is a subset sum along a recovered line through i, of up to 5 ($L = 2$) or 9 ($L = 4$) pixels, with 4 orientations per pixel. The orientation labels are canonical per pixel but not consistent across pixels. The bars are therefore per-position features: nothing is shared across positions, as a convolution would share it.

The boosters. `ggbt` and `cgbt` are LightGBM [15] with 31 leaves, learning rate 0.1, 63 histogram bins, and per-tree column fractions 0.12 and 0.05. Below 5000 training points they run 100 rounds. Above that they stop early on the error of an inner validation split (patience 50, 150–400 rounds). Every split is a gate $[f(x) > \theta]$, with f a pixel, a blob sum, a bar sum or (in `ggbt`) a prototype-pair projection $(x_a - x_b) \cdot x$ for two random training points of different classes, and θ a histogram-bin boundary. A tree is a conjunction of such gates along each path. The ensemble is a sum of trees fitted by functional gradient descent (boosting).

2.5 What is discrete, and what is not

Table 2 lists what v3 fits.

- *No feature is learned by gradient descent.* Features are enumerated or built from data statistics, and admitted by tests or by greedy split search. The continuous fits are linear readouts and leaf values. The one readout on the inputs themselves, `lin`, is a convex logistic regression fitted by a quasi-Newton method (L-BFGS).
- *Boosting is functional gradient descent.* So the image members are best described as "a fixed, data-derived feature hierarchy read out by gradient-boosted trees".
- *The comparison is with gradient-descent-trained networks, not with gradient descent.* On images, the pooling sets and bars recover the pixel grid from correlations (Section 6.2). The members use locality in spirit, though not in the letter. A comparator that is told the layout, such as a small CNN, is reported as context only.

2.6 How v3 came about

v3 is the last of a sequence, each step pre-registered and frozen before its held-out evaluation:

- DFG v1–v1.1 (Boolean targets only);
- the Part 8 portfolio v1 (DFG plus the classical members);
- v2 (Part 9), which added sum blocks to DFG (v1.2) and the `gknn`/`ggbt` image members;
- v3 (Part 10), which added v1.3's weighted blocks, forced blocks and beam, and the `cgbt` member.

In each Part 10 step, independent builders worked on development data only. Each delivered a frozen candidate and its fixed development evaluation. A rule written before any design work then chose the components (Section 3).

3 How it is evaluated

3.1 Tasks

Toy Boolean targets. Inputs are uniform on $\{\pm 1\}^d$ with $d = 30$, and labels are ± 1 values of a target f. There are 11 suites, 138 targets in all, summarized in Table 3 and defined in `study/boolfuncs.py`.

- `designed` (21) and `random` (20). Parities, majorities, halfspaces, staircases, compositions, a multiplexer, a DNF, a decision list and random small formulas.
- `dlheld`, `dlheld2`, `dlheld3` (12, 7, 10). Dense halfspaces and majorities, χ_5, multiplexers, symmetric functions such as $[|x_1 + \cdots + x_9| \leq 1]$, mixtures of parities with gates, and random formulas.
- `dlsymdev` (8). Symmetric and signed-sum targets used for development.
- `dlheld4`, `dlheld5`, `dlheld6` (12 each). Signed sums with threshold, band, interval, residue-class and arbitrary accept sets, some mixed with parities and gates, plus 2–3 non-symmetric controls per suite.
- `dlfamdev` and `dlheld7` (12 each). Two fresh draws (seeds 61616 and 70707) from `dlheld6`'s family. That family was written by an agent blind to the sum-feature design `study/part9/heldout6/FAMILY.md`. It has 9 structured slots and 3 controls: sums, sums times a parity, sums with a gate, and compositions of sums. Each block's accept set is drawn uniformly from

five types, one of them a residue class mod 3 or 4, except in the inner-term form $u = s + a\tau$, which uses a threshold or a band.

Every suite puts the relevant inputs first. The discrete learners therefore see the columns in a fresh random order in every fit, so that tie-breaking by column index cannot help them.

Tabular data. There are 26 PMLB datasets [23]: 5 for development, 11 held out in Part 7 (including mushroom), and 10 fresh in Part 8 (HELDOUT2). They have continuous, categorical, binary and mixed features and 2–26 classes.

Images. MNIST [19], Fashion-MNIST [28] and five 10-class sets of QuickDraw bitmaps [13]. The QuickDraw sets use 50 categories drawn at random from the 345 (seed 9191) and split into five disjoint sets of ten:

- `qd_dev`, development;
- `qd_a` and `qd_b`, held out in Part 9;
- `qd_c` and `qd_d`, held out in Part 10.[1]

Each set has 6000 training and 1000 test drawings per class, so a 60000-point training pool and a 10000-point test set. The 784 pixel columns are scaled to $[0, 1]$ and permuted by one fixed permutation (seed 9192). Every reference MLP is invariant to that permutation.

3.2 References

Networks on the toy targets. These are the companion paper's μP two-hidden-layer ReLU MLPs [6, 29]: width 128, Adam at learning rate 10^{-3}, batch 64, 20000 steps on a fixed training set, read at the final step. There are five variants:

- `nets`: square loss;
- `nets_wd`: decoupled weight decay 0.1;
- `nets_cewd`, `nets_cewd0.3`, `nets_cewd1.0`: logistic loss with weight decay 0.1, 0.3 and 1.0.

Not every variant was run on every suite (Table 3). `nets_best` is, per target, the variant with the smallest n^* among those run: a hindsight choice that favours the networks. `lin` (cross-validated L2 logistic regression on the raw inputs) and `hgb` (scikit-learn's histogram gradient boosting, default settings [24]) are calibration references.

MLPs on standard tasks. All are ReLU MLPs with softmax cross-entropy:

- `mlp`: 256-256, Adam, learning rate and stopping point chosen on validation;
- `mlp_reg`: the same with an L2 penalty chosen on validation;
- `mlp_strong`: quantile-transformed inputs, AdamW with cosine decay, weight decay in $\{0, 0.1, 1\}$ chosen on validation, dropout 0.1, an ensemble of 3;
- `mlp_wide`: the same at width 512 with weight decay in $\{0.1, 1, 3\}$;
- `mlp_xl`: width 1024 at $n = 60000$, at the weight decay `mlp_wide` chose there. Its CPU time per fit is close to the discrete learners'.

[1] `qd_c`: hospital, submarine, telephone, bathtub, hockey stick, key, snake, bowtie, hat, binoculars. `qd_d`: basketball, purse, train, pencil, remote control, camel, onion, harp, squiggle, beard.

`mlp_best` is the per-cell hindsight minimum over the Part 7–8 variants, and `mlp_best+` adds the wider ones. Two further learners are *not* references:

- `mlp_pool`, `mlp_strong` given v2's correlation-pooled sums as extra inputs, is a control for how much of the image result is representation;
- `cnn`, a small convolutional network on the *unpermuted* pixels, is context: it uses the layout that every other learner here is denied.

Earlier versions. v1 (Part 8) and v2 (Part 9) are run as references. v2 is run on every toy suite and at every image cell of v3. v1 is run at every image cell of v3 and on every toy suite except `dlfamdev` and `dlheld7`. On tabular data without DFG, v1's, v2's and v3's code paths are the same.

3.3 Metrics, verdicts and budgets

Toy targets. n^* is the training-set size at which the seed-mean test error reaches 0.1. It is interpolated in $\log n$ on the grid $n = 16, 32, \ldots, 16384$, with 3 seeds (2 on `random`) and one 4096-point test set per target. A comparison is the $\log_2$ ratio $\log_2(n^*_{\text{learner}}/n^*_{\text{ref}})$. A target is *clearly better* or *clearly worse* when the ratio is below -0.25 or above $+0.25$, or when only one of the two reaches 10% error. Suite means are taken over targets both learn.

Standard tasks. Test error is measured on a fixed test set at $n \in \{100, 300, 1000, 3000, 10000, 30000, 60000\}$ (up to the pool size). There are 3 seeds, except on images at $n \geq 30000$, where every learner runs seed 0 only, except the baseline `mlp`, which has 3 seeds at every n. The control `mlp_pool` and the context `cnn` ran seed 0 only at every n, and `mlp_xl` at $n = 60000$ only. A cell is *better* or *worse* when the seed-mean errors differ by more than 10% relative and by more than 2 standard errors. The s.e. comes from the seed spread of both learners and is at least the test set's binomial noise; otherwise the cell is a *tie*. Errors are floored at 0.002 in $\log_2$ ratios.

Budgets. Compute is counted in FLOPs, everything a learner does in a fit included, prediction excluded.

- *Toy fit:* at most one training run of the toy networks, 1.62×10^{11} FLOPs at $d = 30$.
- *Real-data fit:* at most three training runs of the baseline MLP, the baseline's own cost including its learning-rate search.

CPU time per fit is reported separately, because equal FLOPs are far from equal CPU time.

3.4 How v3 was built and what is held out for it

Pre-registration. The Part 10 protocol `study/part10/PROTOCOL.md` was committed before any design work (`dc70266c`). It fixed six things:

- the fresh held-out sets `dlheld7`, `qd_c` and `qd_d`, and a development draw `dlfamdev`;
- the budgets;
- three builder tasks on development data only: a toy component (DFG's sum-feature search) and two image components from different angles;
- an *integration rule*. The toy component is adopted if its mean $\log_2 n^*$ ratio against v2 on `designed`, `dlsymdev` and `dlfamdev` is below -0.05, with no target worse by more than 0.5.

set	size	status for v3	network references
toy `designed`, `dlsymdev`, `dlfamdev`	41	development; in-sample for the toy component	variants run there (none on `dlfamdev`)
toy `random`	20	development check for DFG in Part 6, evaluated with Part 8's held-out suites; development data in Part 10 (not used by the toy builder)	`nets`, `nets_wd`
toy `dlheld`–`dlheld5`	53	held out for earlier versions; development data in Part 10 (not used by the toy builder)	`nets`, `nets_wd`; `nets_cewd` from `dlheld3` on
toy `dlheld6`	12	held out in Part 9; used by the Part 10 toy builder	all five variants
toy `dlheld7`	12	**held out**	`nets_wd`, `nets_cewd0.3`, `nets_cewd1.0`
tabular (26 sets)	26	v3 runs v1's code there, except on kr_vs_kp and mushroom, where DFG v1.3 replaces v1's DFG v1.1 (kr_vs_kp: the toy builder's development check); HELDOUT2 is held out for that code (Part 8)	MLP variants
MNIST, Fashion-MNIST, `qd_dev`	3	development	MLP variants
`qd_a`, `qd_b`	2	held out in Part 9; development data in Part 10	`mlp_strong`, `mlp_wide`, `mlp_xl`
`qd_c`, `qd_d`	2	**held out**	`mlp_strong`, `mlp_wide`, `mlp_xl`

Table 3: Every set and its status for v3. "Held out" sets were fixed in a commit before any Part 10 design work, and no result of a Part 10 learner on them was read before v3 was frozen.

The image component with the lower mean $\log_2$ error ratio against v2 over 8 development cells is adopted if that mean is below -0.05 and, at `qd_a` with $n = 60000$, its error is within 5% of v2's, its CPU time is at most $1.5\times$ v2's, and it stays within the FLOP budget;

- a *best-learner rule*. v3 replaces v2 as the learner of this paper if its mean $\log_2$ error ratio against v2 on `qd_c` and `qd_d` at $n \geq 10000$ is below 0, and its mean $\log_2 n^*$ ratio against v2 on `dlheld7` is at most $+0.1$;
- the held-out reference runs.

The builders froze their files, recorded md5 sums and ran the fixed development evaluation. The rules were then applied mechanically (Appendix A.2). v3 was assembled and checked identical to its sources on every input type, then frozen (md5 `b045fafe`). Only then were held-out results of any Part 10 learner read.

What is in-sample. v3 is the last of several pre-registered steps, and every step used the earlier held-out sets as development data. For v3 as a whole, only `dlheld7`, `qd_c` and `qd_d` are untouched. The other suites still measure the learner, but they are not held out for its design: the toy builder developed on `dlheld6`, and the image builders on validation rows of `qd_dev`, `qd_a`, `qd_b` and Fashion-MNIST (and MNIST for topology diagnostics). On tabular data the claim is narrower and cleaner: v3's code path there is v1's. HELDOUT2 was fixed before any Part 8 design work (commit `04a215b2`), and v1 was frozen before it was run there, so HELDOUT2 is held out for that code.

4 How well it does: toy targets and tabular data

This section compares v3 with every reference on the Boolean toy suites (Sections 4.1 to 4.4) and on the 26 tabular datasets (Section 4.5). The image sets are in Section 5.

Sources. `study/part10/LEARNER_RESULTS.json` holds every number below for which no other file is named. The script `study/part10/compare_learner.py` computes it from the result files. The tables are generated from it by `make_tables.py`, and Figure 1 and the derived toy summaries by `make_figures.py` (written to `figures/fig_toy_nstar.json`).

- *Toy.* On toy inputs v3 runs exactly the code of the Part 10 toy candidate `p10_sum`; its predictions were checked identical at assembly `study/part10/hyb_portfolio3/ASSEMBLY.md`. v3's toy results are therefore the `p10_sum-perm` result files in `study/hybrid/results/`. v2's are the `hyb_portfolio2-perm` files, and on `designed` and `dlsymdev` the `p9_sum-perm` files (the same code on toy inputs).
- *Metrics* (Section 3.3). n^* is the training-set size at which the seed-mean test error reaches 10%. There are 3 seeds per n (2 on `random`) and one 4096-point test set per target. The discrete learners see the inputs permuted afresh in every fit. "Clearly better" means a $\log_2 n^*$ ratio below -0.25, or that only v3 reaches 10% error by $n = 16384$.

4.1 The held-out suite `dlheld7`

`dlheld7` is the only toy suite held out for v3. It has 12 targets, a fresh draw (seed 70707) from the blind family of `dlheld6`, and it was fixed in a commit before any Part 10 design work. No result of a Part 10 learner on it was read before v3 was frozen (Section 3.4). Table 4 gives every target's definition and n^*.

Against the networks [Numerical].

- *Against `nets_best`:* mean $\log_2 n^*$ ratio -1.50 over the 11 targets that both learn, about 2.8× less data. v3 is clearly better on 10 of 12 targets and clearly worse on none.
- *The two that are not wins.* f7_01, a threshold of a 13-input count, is a tie: 106 against 119. f7_05, a residue class mod 3 of a 14-input count, is learned by no learner (Section 4.4).
- *Against each variant alone:* -1.83 against `nets_wd` (11 of 12 targets clearly better), -1.59 against `nets_cewd0.3` (10 of 12) and -1.58 against `nets_cewd1.0` (10 of 12). No variant is clearly better than v3 on any target.
- *Largest margins.* f7_00, an OR of conditions on two signed sums: 203 against 1545 ($\log_2$ ratio -2.93). f7_08, an arbitrary accept set of an 11-input count: 634 against 2932. f7_03, a multiplexer between a residue class mod 4 of a 6-input count and a condition on $|s|$ for a 4-input count s: 352 against 1599. f7_09, a threshold of a signed 10-input sum times the parity $x_{11}x_{12}$: 215 against 883.
- *Smallest margins.* The two halfspaces: f7_01 (106 against 119) and f7_10, $[x_1+\cdots+x_5+2x_6 \leq -1]$ (82 against 119).

Which wins need no feature search [Numerical]. `lin` reaches 10% error on only 4 of the 12 targets, and `hgb` on 9. v3's mean $\log_2$ ratio is -0.70 against `lin` (over those 4) and -1.64 against

target	definition	v3	v2	nets_wd	nets_ cewd0.3	nets_ cewd1.0	nets_best	lin	hgb
f7_00	OR([|s1| ≤ 0], [s2 == -1]), s1 = x1+x2-x3-x4, s2 = -x5-x6-x7+x8+x9+x10+x11	203	207	2418	1545	1565	1545	never	6208
f7_01	[s ≤ 1], s = x1+...+x13	106	107	205	121	119	119	111	684
f7_02	OR(AND(x2,-x6,-x8), AND(x1,-x2,-x4,-x5), AND(x3,-x5,-x7))	106	106	337	341	348	337	never	120
f7_03	-x11 ? [((s1+6)/2) mod 4 != 2] : [|s2| > 2], s1 = x1+...+x6, s2 = x7+...+x10	352	342	1710	1665	1599	1599	never	2199
f7_04	if AND(-x6,-x9) → +1; elif -x8 → -1; elif -x4 → +1; elif -x2 → -1; elif x7 → -1; elif x5 → -1; elif AND(x3,x1) → -1; else +1	70	74	164	175	181	164	never	84
f7_05	[((s+14)/2) mod 3 != 0], s = x1+...+x14	never	never	never	never	never	never	never	never
f7_06	OR(AND(AND(-x1,x2),AND(AND(-x3,x4),-x5)),AND(OR(-x6,x7),x8))	44	43	108	124	124	108	127	59
f7_07	[-2 ≤ s ≤ 8] * x13, s = -x1+x2-x3+x4+x5+x6+x7+x8-x9+x10-x11+x12	174	174	445	443	415	415	never	1373
f7_08	[s in -11,-9,-3,1,9,11], s = x1+...+x11	634	634	5843	3239	2932	2932	never	never
f7_09	[s ≤ 0] * x11 * x12, s = x1-x2+x3-x4+x5-x6+x7+x8+x9+x10	215	215	1000	883	928	883	never	never
f7_10	[u ≤ -1], u = s + 2*(x6), s = x1+...+x5	82	82	210	120	119	119	87	114
f7_11	-x11 ? [s1 in -4,4] : [|s2| ≤ 4], s1 = x1+x2-x3+x4, s2 = -x5-x6+x7-x8-x9+x10	55	55	114	144	141	114	119	63

Table 4: The held-out toy suite `dlheld7`: n^*, the training-set size at 10% test error (mean of 3 seeds, interpolated in $\log n$), for v3, v2, the three pre-registered network variants, their per-target minimum `nets_best`, and the calibration references `lin` and `hgb` [Numerical]. v3 and v2 see the inputs permuted per fit. "never": 10% error is not reached by $n = 16384$. In the definitions, $[\cdot]$ is $+1$ when the condition holds and -1 otherwise; `*` multiplies ± 1 values; `c ? a : b` is a multiplexer (a when $c = +1$); a literal `x5` or `-x5` inside AND/OR, a decision-list rule or a selector stands for $x_5 = +1$ or $x_5 = -1$. f7_02, f7_04 and f7_06 are the family's non-symmetric controls. Source: `study/part10/LEARNER_RESULTS.json`.

`hgb` (over those 9); it is clearly better than `hgb` on 9 targets and than `lin` on 9, and clearly worse than neither on any. The wins over the networks fall into two groups.

- *Five wins need no feature search.* A learner that searches no features is already clearly better than `nets_best` on five targets: `hgb` on the three controls (f7_02, a DNF; f7_04, a decision list; f7_06, a read-once formula) and on the multiplexer f7_11, and `lin` on the halfspace f7_10. On these five, v3 is ahead of the better calibration reference by factors of 1.06–1.35 only ($\log_2$ ratios -0.09 to -0.43).
- *Five wins need the feature search.* On the other seven targets neither `lin` nor `hgb` beats the networks. `hgb` needs 684–6208 samples or never reaches 10%, and `lin` learns only f7_01. There v3's mean against `nets_best` is -1.80: clearly better on 5 (f7_00, f7_03, f7_07, f7_08, f7_09, all sum-structured), a tie on f7_01, and f7_05 learned by none.

Against v2 [Numerical]. The mean $\log_2 n^*$ ratio is -0.004. No target is clearly better or worse, and the largest per-target change is 0.09 in $\log_2$ (f7_04: 70 against 74). So the Part 10 toy component changes nothing measurable on the held-out suite. The best-learner rule only asked that v3 not be worse than v2 there (mean $\leq +0.1$). v3 became the learner of this paper through its image side

suite	status	targets	v3 vs `nets_best`	v3 vs v2	v3 vs `lin`	v3 vs `hgb`
`designed`	dev	21	−2.42 (21/0)	−0.03 (1/0)	−0.85 (19/0)	–
`random`	dev check (P6)[c]	20	−2.26 (20/0)	−0.02 (1/0)	−1.29 (20/0)	−1.78 (20/0)
`dlheld`	held out (P6)	12	−2.03 (12/0)	+0.02 (0/1)	−0.62 (10/0)	−2.13 (10/0)
`dlheld2`	held out (P6)	7	−1.99 (7/0)	−0.14 (1/0)	−0.58 (5/0)	−2.79 (6/0)
`dlheld3`	held out (P8)	10	−1.80 (10/0)	−0.04 (0/0)	−0.99 (9/0)	−1.82 (9/0)
`dlsymdev`	dev (P9)	8	−2.22 (8/0)	−0.18 (2/0)	n/a (8/0)	–
`dlheld4`	held out (P9)[a]	12	−1.87 (12/0)	−0.19 (3/0)	−1.54 (12/0)	−2.85 (11/0)
`dlheld5`	held out (P9)	12	−2.44 (10/0)	−0.04 (1/0)	n/a (11/0)	−4.11 (11/0)
`dlheld6`	held out (P9), dev (P10)[b]	12	−1.50 (11/1)	−0.25 (2/0)	−1.13 (11/0)	−2.21 (10/0)
`dlfamdev`	dev (P10)	12	–	−0.05 (1/1)	–	–
`dlheld7`	**held out (P10)**	12	−1.50 (10/0)	−0.00 (0/0)	−0.70 (9/0)	−1.64 (9/0)

Table 5: v3 on every toy suite [Numerical]. Each entry is the mean $\log_2 n^*$ ratio of v3 to the reference over the targets both learn, with the numbers of targets on which v3 is clearly better / clearly worse in brackets. "n/a": the reference learns none of the targets, so no mean exists; "–": the reference was not run on the suite. `nets_best` is the per-target minimum over the network variants run on the suite, which differ between suites (Table 3). Status: the Part (P6–P10) in which the suite was held out or used for development. Only `dlheld7` is held out for v3; for v3 every other suite is development data. [a]The Part 9 sum-feature builder saw the source of `dlheld4`. [b]The Part 10 toy builder developed on `dlheld6`. [c]`random` was a development check for DFG in Part 6 (run and read with intermediate versions) and was evaluated with the held-out suites in Part 8. Source: `study/part10/LEARNER_RESULTS.json`.

(Section 5).

Compute [Numerical]. Every v3 fit on `dlheld7` used at most 0.53 of one network training run's FLOPs (f7_04). Over all 11 suites (every v3 toy fit) the maximum is 0.83 (h5s7 on `dlheld5`). CPU time per fit is at most 46 s on the shared 4-core machine (`designed`, thr10). Source: the `flops` and `cpu_seconds` fields of the v3 result files.

4.2 Every toy suite

Table 5 summarizes all 11 suites, and Figure 1 shows every target with a network reference.

The status of each suite (Table 3) decides how much a row says about v3.

- *Development for v3:* `designed`, `dlsymdev` and `dlfamdev`. The integration rule was computed on them.
- *Used by the Part 10 toy builder:* `dlheld6`, held out in Part 9. The builder designed weighted blocks after v2's failure on its target h6_10.
- *Earlier evaluation suites:* `dlheld`–`dlheld5`, held out for v1 or v2, and `random`, a development check for DFG in Part 6 that Part 8 evaluated with its held-out suites. They are development data in Part 10, but the toy builder did not use them `study/part10/p10_sum/DESIGN.md`. Some of them shaped earlier versions: v1's failure on sym9 (`dlheld3`) motivated the sum blocks of v2, and the Part 9 builder saw the source of `dlheld4`.
- *Held out for v3:* `dlheld7` only.

Results [Numerical]. On the 126 targets of Figure 1, v3 is clearly better than `nets_best` on 121 and needs at least 4× less data on 54 (`figures/fig_toy_nstar.json`). Five targets are not wins:

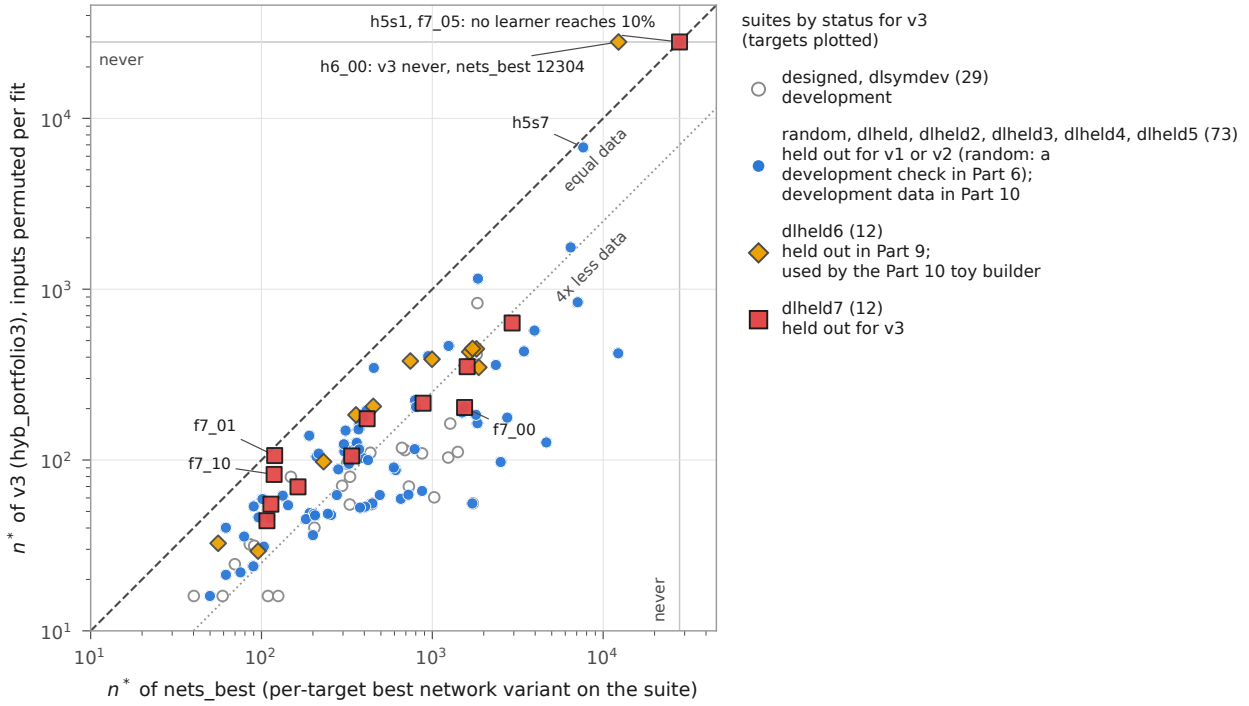


Figure 1: n^* of v3 (inputs permuted per fit) against `nets_best` for every target of every toy suite with network references, one point per target [Numerical]. Colour and shape give the status of the suite for v3 (Table 3); the held-out suite `dlheld7` is drawn as large outlined squares. "never": 10% error not reached by $n = 16384$, drawn at the edge. Points below the dashed line need less data than the best network variant, and points below the dotted line at least 4× less. `nets_best` is a minimum over 2 variants on `designed`, `random`, `dlheld` and `dlheld2`, 3 on `dlsymdev` and `dlheld3`–`dlheld5`, 5 on `dlheld6` and the 3 pre-registered ones on `dlheld7`. `dlfamdev` has no network runs and is not shown. Generated by `make_figures.py` (`fig_toy_nstar`) from `study/part10/LEARNER_RESULTS.json`; the plotted values are in `figures/fig_toy_nstar.json`.

- h6_00 (`dlheld6`) is clearly worse: only the most regularized network learns it, at 12304;
- f7_01 (`dlheld7`) and h5s7 (`dlheld5`) are ties;
- f7_05 (`dlheld7`) and h5s1 (`dlheld5`) are learned by no learner.

Section 4.4 discusses them. The suite means against `nets_best` range from −1.50 to −2.44 (Table 5).

Against the one variant run on every suite [Numerical]. The suite means against `nets_best` are not comparable across suites, because `nets_best` minimizes over different sets of variants. `nets_wd` is the one variant run on every suite with network results. Against it, v3's mean $\log_2 n^*$ ratio is:

- development suites: −2.42 on `designed` and −2.53 on `dlsymdev`; the Part 6 development check `random`: −2.26;
- earlier held-out suites: −2.03 on `dlheld`, −1.99 on `dlheld2`, −2.07 on `dlheld3`, −2.25 on `dlheld4` and −2.66 on `dlheld5`;
- the two draws of the blind family: −1.94 on `dlheld6` and −1.83 on `dlheld7`.

The two draws of the blind family give the two smallest margins, but `dlheld2` is within 0.05 of `dlheld6`, and differences of this size on suites of 7 to 12 targets are within noise. If the gap is real,

two explanations fit it, and the data do not separate them [Heuristic]. Several of the other suites informed the design of v3 or of an earlier version, so some selection in v3's favour is expected there. And the suites differ in kind: `dlheld`–`dlheld5` were written within the study, mostly from families close to its development targets, while the blind family was written by an agent that had not seen the sum-feature design.

Pooled over the held-out suites [Numerical]. Pooled over the seven held-out suites (`dlheld`, `dlheld2`, `dlheld3`–`dlheld7`; 77 targets), the mean $\log_2 n^*$ ratio against `nets_best` is -1.87 over the 74 targets both learn. v3 is clearly better on 72 targets and clearly worse on 1 (h6_00). This matches `study/part10/LEARNER_RESULTS.md`; the development check `random` is not included. The pooled number is a summary of the record, not a held-out estimate: 65 of the 77 targets were development data for v3, and only the 12 of `dlheld7` are held out. The held-out estimate is the `dlheld7` row alone.

`lin` and `hgb` [Numerical]. Some of the wins over the networks need no feature search, and they are of three kinds.

- *Halfspaces.* Logistic regression alone is clearly better than `nets_best` on halfspace-like targets: dict, maj3, maj9, rand2 and thr10 on `designed`, maj15 and thr30 on `dlheld`, maj21 and thr20 on `dlheld2`, thrint on `dlheld3` and f7_10 on `dlheld7`. On these, v3 is within the tie band of `lin` (thr10 121 against 113, thr20 124 against 117, thr30 149 against 148, maj9 80 against 80, maj15 105 against 110, maj21 109 against 116, thrint 139 against 137, f7_10 82 against 87) or clearly below it (maj3 31 against 56, dict 16 against 27, rand2 16 against 50).
- *Formulas, decision lists and multiplexers.* `hgb` is clearly better than `nets_best` on six targets of `dlheld` (the random formulas hrf0, hrf1 and hrf2, the majority of majorities majmaj5, the multiplexer mux6 and the staircase stair6), on three of `dlheld2` (dnf4x2, h2rf0, h2rf1), on the five formula and multiplexer targets of `dlheld3` (cnf3, dlist8, h3rf0, h3rf1, mux11), on 8 of the 11 non-symmetric controls of `dlheld4`–`dlheld7`, on the multiplexer f7_11, and narrowly on h5s4, a signed-sum condition OR a literal (1465 against 1801).
- *Random formulas.* On the development check `random`, `hgb` is clearly better than `nets_best` on 9 of the 20 formulas and `lin` on 2 (rf06, rf10). v3 is clearly better than both on all 20 (Table 5).

Across the seven held-out suites, one of the two calibration references is clearly better than `nets_best` on 30 of the 77 targets. On the other 47, v3's mean against `nets_best` is -2.11 (over the 44 that both learn), with 42 targets clearly better and 1 clearly worse (`figures/fig_toy_nstar.json`). So the margins do not come from targets that are easy for trees or linear models. They come from the targets with parity or signed-sum structure, where both `lin` and `hgb` fail: for example, `hgb` never reaches 10% error on 5 of the 12 targets of `dlheld4` and on 6 of `dlheld5`, and `lin` on 11 of `dlheld4` and all 12 of `dlheld5`. Part 8's ablations located the toy wins in DFG [5].

4.3 What changed from v2 to v3

The only difference between v2 and v3 on toy inputs is DFG: v1.3 replaces v1.2 (Section 2.3). v1.3 adds three things to the sum-block search, all of which act together:

- weighted blocks $u = t + a\tau$;
- forced blocks in the small-n variant at $n \geq 48$;

suite	target	v2	v3	$\log_2$(v3/v2)	nets_best
designed	parmaj	142	110	−0.36	434
random	rf03	78	62	−0.32	276
dlheld	hrf0	46	62	+0.43	133
dlheld2	sym8	100	53	−0.92	377
dlsymdev	sym7	110	55	−1.00	329
	far9	87	71	−0.30	296
dlheld4	majsym	189	90	−1.06	595
	tail13	176	100	−0.81	420
	sym11	203	160	−0.34	374
dlheld5	h5rf0	268	203	−0.40	1547
dlheld6	h6_10	702	206	−1.77	451
	h6_07	678	447	−0.60	1812
dlfamdev	fd_09	171	101	−0.76	–
	fd_06	266	318	+0.26	–

Table 6: Every toy target on which v3's n^* differs clearly from v2's ($|\log_2 \text{ratio}| > 0.25$), on all suites with results for both [Numerical]. There are 14 such targets among the 138. None is on dlheld3 or on the held-out dlheld7. Source: study/part10/LEARNER_RESULTS.json.

- a beam search at $n \leq 128$.

On the development suites the integration rule measured a mean $\log_2 n^*$ ratio of −0.066 against v2 over 40 targets, with the largest increase +0.26 on fd_06 study/part10/RESULTS.md. That is a small gain. Table 6 shows where it sits over all suites: 12 targets clearly better and 2 clearly worse, out of 138.

Where n^* moved, and what the models show. The descriptions below come from the interpretability analysis (seed 0, Section 6.1). They report what the fitted DFG model contains; no ablation separates the three changes, so they are observations, not attributions [Heuristic].

- *h6_10* $= [|x_1 + \cdots + x_{12} - 3x_{13}| > 1]$ (dlheld6): 702 to 206. v3's model holds the weighted block $|\pm(x_1 + \cdots + x_{12}) + 3x_{13}|$ from $n = 256$, with the right relative sign from $n = 512$. v2 could represent the target as well: from $n = 512$ its model holds two folded 13-input blocks, $|s + x_{13}|$ and $|s - x_{13}|$, which together determine the target (DFG error 0.000 at $n = 1024$). So the change is in how early the structure is found, not in what can be represented. This is the target the weighted blocks were designed for, on a suite the builder used, so it is not evidence for v3. It turns one of dlheld6's two Part 9 losses to the networks (h6_10: v2 702 against nets_best 451) into a win (206); the other, h6_00, remains.
- *h6_07* (dlheld6), a multiplexer between conditions on two signed sums: 678 to 447. The selector x_{11} enters the model as the heavy literal of weighted blocks: from $n = 512$ on the 4-input branch, $(x_7 + x_8 + x_9 - x_{10} - 3x_{11})$, and from $n = 1024$ on the 6-input branch, $(x_1 - x_2 - \cdots - x_6 + 3x_{11})$.
- *sym7* (dlsymdev, 110 to 55) and *majsym* (dlheld4, 189 to 90). At the first n at which v3's model holds the exact block (64 and 128, one doubling earlier than v2's), the block is in the forced variant, below the test's threshold (0.89 of it for sym7; 0.36–0.39 for majsym's plain block). On sym7 at $n = 64$ the stacker puts all its DFG weight on that forced member.

- *fd_09, tail13, sym11*: the exact block appears one doubling earlier, at 128 instead of 256. The true block already passes DFG's test at 128 (the unweighted test is the same in both versions), so the difference is in the search.
- *parmaj, far9, h5rf0, sym8 and rf03*: not diagnosed. h5rf0 is a random-formula control (268 to 203), sym8 is on `dlheld2` (100 to 53) and rf03 is a random formula of `random` (78 to 62). On development data the toy builder traced parmaj to the beam at $n = 128$ and far9 to forced blocks at $n = 64$ `study/part10/p10_sum/DESIGN.md`.
- *The two losses*, not diagnosed: hrf0 (`dlheld`, a random formula; 46 to 62, +0.43, the larger of the two), on a suite the toy builder did not use; and fd_06 (+0.26), an XOR of conditions on two signed sums.

The weighted class is used far more widely than for h6_10. A weighted block with at least 20% of the largest term's share appears at some n in the models of 26 targets of the six sum-structured suites: 16 sum-structured targets and 10 of the 14 controls. In the controls its inner term is an AND of two literals in 9 cases and a literal in 1 (Section 6.1). On most of these 26 targets n^* does not move clearly (Table 6).

Why nothing moved on `dlheld7` [Heuristic]. The gains of v1.3 are concentrated on a few targets: 12 of 138, 11 of which v3 now learns at $n \leq 256$. A 12-target draw can easily contain none of them. The three draws of the blind family agree with this: v3's mean against v2 is -0.05 on `dlfamdev` (development), -0.25 on `dlheld6` (used by the builder) and -0.004 on `dlheld7`. `dlheld7` has one weighted-sum target, f7_10. It is a halfspace that v2 already learns at the same n^* (82), although at $n = 256$ only v3's model holds the exact weighted block.

4.4 Targets nobody learns, and where v3 is not better

Residue classes mod 3 [Numerical]. Three targets are residue classes mod 3 of a signed count of 10–14 inputs:

- fd_10 on 10 inputs (`dlfamdev`, development);
- h6_00 on 13 (`dlheld6`);
- f7_05 $= [\frac{1}{2}(s + 14) \bmod 3 \neq 0]$, $s = x_1 + \cdots + x_{14}$, on 14 (`dlheld7`).

v3 and v2 learn none of them by $n = 16384$. On f7_05, neither does any network variant, `lin` or `hgb`. On h6_00, the only learner that does is the most regularized network (`nets_cewd1.0`, $n^* = 12304$). fd_10 has no network runs. The interpretability analysis (seed 0, Section 6.1) locates the failure in DFG's search, not in its test:

- *The test would accept the block.* The true block passes DFG's test from $n = 128$, and at $n = 16384$ it is 134–155 times its threshold.
- *The search does not propose it.* The targets have almost no Fourier weight at degrees 1 and 2: about 2×10^{-4} for fd_10, and zero to five decimals for h6_00 and f7_05. The best pair of inputs inside the block scores 0.12–0.25 of its threshold, and the search's input orders reach the block no better than chance. (Reading [Heuristic]: every seed of the block search reads the block's inputs off degrees 1 and 2, so it has nothing to start from.)
- *The model stays empty.* At every $n \geq 1024$, DFG's model holds only the constant and the kernel background, and its test error is the minority rate (0.32–0.34). (At several of these n,

including $n = 16384$ for fd_10 and h6_00, the summarised model is an out-of-fold DFG fit on 13107 rows with zero stacking weight.)

Section 2.3 explains why a block with no low-degree foothold is invisible to the seeds.

h5s1 (`dlheld5`) is a signed 9-input count with accept set $\{-9, -5, -1, 3, 9\}$, multiplied by the parity $x_{10}x_{11}$. No learner reaches 10% error by $n = 16384$ (3 seeds). 92% of its variance lies on the single degree-11 character $x_1 \cdots x_{11}$. The modulated block passes DFG's test from $n = 128$, but DFG takes modulators only from the characters already in its model, and the coefficient of $x_{10}x_{11}$ is only -0.031. On seed 0, v3 finds the structure only at $n = 16384$.

Ties [Numerical].

- *f7_01* $= [x_1 + \cdots + x_{13} \leq 1]$ is a halfspace. v3 (106), `lin` (111) and the logistic-loss networks (119–121) are all within the tie band. Its 13-input block is in v3's model, but so are degree-1 characters, and a linear readout of the inputs alone already does about as well.
- *h5s7* is an arbitrary accept set of a signed 13-input count, with n^* 6760 against 7641 (`nets_cewd`). Its weight at degrees 1–2 is only 0.015. The true block passes the test from $n = 128$, but the search first finds it at $n = 8192$ (seed 0): the same mechanism as the mod-3 counts, in weaker form.

The next-smallest margin is inter2 (`dlheld`, an AND of two halfspaces on 10 inputs: 346 against 454, -0.39). Then come the halfspace-like thrint (-0.46) and f7_10 (-0.53), where `lin` is already as good as v3, a random formula of `random`, rf00 (-0.62), and h5s2 (-0.68), an arbitrary accept set whose weight at degrees 1–2 is also small.

The one loss. h6_00 is the only target of any suite on which a network is clearly better than v3. It was v2's loss as well. The Part 10 toy builder measured the search landscape of the analogous fd_10 and concluded that a random-restart search would need far more than 10^5 restarts; fd_10 and h6_00 were left unlearned `study/part10/p10_sum/DESIGN.md`.

4.5 Tabular data

What v3 runs on tabular data. DFG runs only when there are 2 classes and every column is binary or categorical (≤ 256 bits). The image members run only on images. Of the 26 tabular datasets, only two qualify for DFG: kr_vs_kp (development) and mushroom (Part 7 held-out set).

- *The other 24 datasets.* v3 runs v1's code: the stack of LightGBM, ExtraTrees, k-NN and logistic regression. Its predictions were checked identical to v2's on one optdigits fit, and v2 runs v1's code path there. So v3's results on these datasets are v1's Part 8 result files.
- *mushroom.* v3's own runs give the same seed-mean test error as v1 at every n.
- *kr_vs_kp.* v3's own runs differ from v1 at 3 of 4 sizes, but no cell differs clearly (at $n = 1000$, 0.0075 against 0.0100; at the full 2397, 0.0029 against 0.0021).

What is held out. HELDOUT2 (10 datasets) was fixed in a commit before any Part 8 design work (`04a215b2`). v1 was developed on the 5 development datasets only, and its code path has not changed since. HELDOUT2 contains no dataset on which DFG runs, so it is held out for this code path. The Part 7 held-out set is held out in the weaker sense that the Part 8 designers had read its MLP references. The 5 development datasets are in-sample.

split	reference	cells (better / tie / worse)	mean $\log_2$ ratio	datasets with lower mean error (sign test p)
Part 7 held-out set	`mlp`	59 (22/36/1)	−0.331	10 of 11 (0.012)
	`mlp_reg`	59 (16/40/3)	−0.160	9 of 11 (0.065)
	`mlp_strong`	54 (22/30/2)	−0.339	10 of 11 (0.012)
	`mlp_best`	59 (12/44/3)	−0.114	8 of 11 (0.23)
	`hgb`	59 (20/38/1)	−0.259	10 of 11 (0.012)
Part 8 fresh set (HELDOUT2)	`mlp`	53 (15/36/2)	−0.445	9 of 10 (0.021)
	`mlp_reg`	53 (16/35/2)	−0.272	8 of 10 (0.11)
	`mlp_strong`	53 (15/36/2)	−0.284	8 of 10 (0.11)
	`mlp_best`	53 (10/41/2)	−0.168	8 of 10 (0.11)
	`hgb`	53 (12/40/1)	−0.250	9 of 10 (0.021)
development	`mlp`	24 (4/20/0)	−0.379	5 of 5 (0.062)
	`mlp_reg`	24 (3/21/0)	−0.267	5 of 5 (0.062)
	`mlp_best`	24 (1/23/0)	−0.244	5 of 5 (0.062)
	`hgb`	24 (4/20/0)	−0.270	5 of 5 (0.062)

Table 7: v3 on the tabular datasets against the MLP references and `hgb` [Numerical]. "Cells": the (dataset, n) pairs compared, each a mean over 3 seeds, with the verdict rule of Section 3.3. "Mean $\log_2$ ratio": of v3's test error to the reference's, over cells, errors floored at 0.002. The last column counts the datasets whose mean ratio over n is below 0, with the two-sided sign-test p. `mlp_best` is the per-cell best of the Part 7–8 MLP variants (hindsight). `mlp_strong` was run on the Part 7 set only at $n \leq 10000$ (54 of 59 cells) and not on the development sets. On 24 of the 26 datasets v3's code path is v1's, so these are Part 8's numbers, except for v3's own runs on mushroom (identical errors) and kr_vs_kp. Source: `study/part10/LEARNER_RESULTS.json`.

Results (Table 7) [Numerical].

- *HELDOUT2, against `mlp_best`:* mean $\log_2$ error ratio −0.168, a geometric-mean error about 11% lower. 10 of 53 cells are better and 2 worse. The mean error is lower on 8 of 10 datasets, which is not significant (sign test $p = 0.11$).
- *HELDOUT2, against single references:* the margins are larger: −0.445 against the baseline `mlp` (9 of 10 datasets, $p = 0.021$), −0.284 against `mlp_strong` (8 of 10, $p = 0.11$) and −0.250 against `hgb` (9 of 10, $p = 0.021$).
- *The Part 7 set:* −0.114 against `mlp_best` (12 better, 3 worse, 8 of 11 datasets, $p = 0.23$) and −0.259 against `hgb` (10 of 11, $p = 0.012$).
- *Development:* −0.244 against `mlp_best`, lower on 5 of 5 ($p = 0.062$).

The per-dataset picture is Part 8's `study/hybrid/RESULTS.md`.

- *HELDOUT2.* The large margins against `mlp_best` are on ann_thyroid (−0.77), clean2 (−1.12, mostly from $n \leq 300$) and churn (−0.35). Three of the 8 datasets with lower error are within about ±0.05. The losses are texture (+0.61; no cell resolves, since errors are below 1.6% from $n = 1000$) and krkopt (+0.19; clearly worse at $n \geq 10000$).
- *The Part 7 set.* The largest wins are dna (−0.77) and ring (−0.50). The losses are nursery (+0.70), pendigits (+0.19) and connect_4 (+0.03, worse at full n).

Where the tabular result comes from [Heuristic]. The tabular numbers are those of a classical stack, and they say little about discrete feature search. Part 8 found three things [5]:

- *DFG does not carry them.* DFG runs on none of the HELDOUT2 datasets. In Part 7, DFG's

own dictionaries were about as good as a plain MLP on tabular data and worse than a regularized one.

- *No single member family matches the stack.* On Part 8's 48 ablation cells (HELDOUT2 and two image sets, $n \leq 3000$, seeds 0–1), LightGBM alone was +0.08 against the best MLP, where the full stack was −0.19. Different members carry different datasets: trees carry churn, ann_thyroid and clean2, k-NN carries banana, and logistic regression carries texture.
- *Default* `hgb` *alone does not beat the MLPs.* Against `mlp_best` it is +0.08 on HELDOUT2 and +0.14 on the Part 7 set `study/hybrid/RESULTS.md`. The stack's mean $\log_2$ ratio against it is about −0.25 on both (Table 7).

This agrees with the tabular-learning literature. Tree ensembles tend to beat plain MLPs on medium-sized tabular data [12, 25]. Stacking diverse models adds to that [10, 27]. And careful regularization and tuning of the MLP narrow the gap [11, 14]. The MLP references here are standard recipes without tabular-specific architectures, so the margins would likely shrink against a tuned tabular network [Heuristic]. The tabular evidence for v3 is therefore modest: it is not worse than the MLPs on average on fresh tabular data (2 of 10 datasets have higher mean error), it is better on average without statistical significance, and the reason is the classical stack, unchanged since Part 8.

5 How well it does: images

On images v3 runs its image members and the classical ones (Table 1); DFG does not run. So on images v3 is v2 with one more member, `cgbt`, and a stacker refitted over all members (Sections 2.4 and 2.6). This section gives v3's test error on the two QuickDraw sets held out for it (Section 5.1) and on the image sets it was developed on (Section 5.2). It then asks where the gain over v2 comes from (Section 5.3) and what a fit costs (Section 5.4).

Sources and rules. Errors and verdicts come from `study/part10/LEARNER_RESULTS.json`, which `study/part10/compare_learner.py` computes from the result files `<learner>__<set>.json` in `study/standard_tasks/results/`. Stacking weights, FLOPs and CPU seconds come from the same files and their `.info.json` records. The tables are generated by `make_tables.py` and Figure 2 by `make_figures.py`.

- *Which file is v3.* On `qd_c` and `qd_d` it is v3's own runs (`hyb_portfolio3`). On the development sets it is the frozen image candidate `p10_codes`, whose predictions on images are identical to v3's (checked identical on one `qd_dev` fit at assembly, $n = 300$; `study/part10/hyb_portfolio3/ASSEMBLY.md`).
- *Cells and verdicts.* A cell is a (set, n) pair. The error is measured on the set's fixed 10000-point test set. v3 is *better* or *worse* than a reference at a cell when the errors differ by more than 10% of the reference's error *and* by more than 2 standard errors; otherwise the cell is a *tie* (Section 3.3).
- *Noise.* At $n = 60000$ every learner except the Part 7 `mlp` ran seed 0 only. The standard error is then the test set's binomial noise alone, about 0.34 points at an error of 13%, so 2 s.e. of a difference is about 1.0 point. The 10% criterion asks for 1.3 points there, so it is the binding one.

set	n	v3	v2	v1	hgb	mlp_ strong	mlp_ wide	mlp_ xl	mlp_ best+	mlp_ pool	cnn
(a) held out for v3 (Part 10): 3 seeds at $n \leq 10000$, seed 0 at $n = 60000$											
qd_c	1000	**26.58**	27.48°	31.68	37.52	33.01	33.02	–	33.01	–	24.36°
qd_c	10000	**16.17**	17.79°	20.05	22.76	20.94	20.41	–	20.41	19.37	14.84°
qd_c	60000	**12.76**	13.63°	15.77	18.22	16.08	14.12°	14.12°	14.12°	14.48	–
qd_d	1000	**26.12**	26.71°	30.21	35.33	31.62	31.40	–	31.40	–	–
qd_d	10000	**16.38**	17.82°	19.70	21.85	20.30	19.89	–	19.89	–	–
qd_d	60000	**12.64**	13.99°	15.97	17.48	15.34	14.31	13.83°	13.83°	–	–
(b) Part 9 held-out sets, development data for v3: v3 seed 0											
qd_a	3000	**17.48**	18.60°	21.28	24.31	23.02	22.67	–	22.67	–	–
qd_a	10000	**14.22**	15.00°	17.21	19.40	18.44	17.83	–	17.83	16.42	12.00*
qd_a	60000	**10.82**	11.97°	14.21	15.23	13.32	12.85	12.46	12.46	12.25	–
qd_b	3000	**21.76**	22.33°	25.13	27.96	26.41	26.13	–	26.13	–	–
qd_b	10000	**16.78**	17.95°	20.27	22.72	21.17	20.64	–	20.64	19.60	–
(c) development sets: v3 and v2 seed 0											
qd_dev	1000	**21.18**	22.22°	26.14	30.31	27.04	–	–	27.04	–	–
qd_dev	10000	**13.30**	14.63°	16.88	18.94	17.49	–	–	17.49	–	–
Fashion	1000	**17.97**	18.19°	18.29°	19.72°	18.60°	–	–	18.60°	–	–
Fashion	10000	**12.38**	12.59°	13.07°	12.94°	12.64°	–	–	12.64°	–	–

Table 8: Test error (%) on every image cell with a v3 result [Numerical]. Seed mean of 3 at $n \leq 10000$ and seed 0 at $n = 60000$, except where a panel says otherwise. `mlp_xl`, `mlp_pool` and `cnn` are seed 0 everywhere. Bold: the lowest error among v3, v2, v1, `hgb`, `mlp_strong`, `mlp_wide` and `mlp_xl`. A reference's entry carries ° where the cell is a tie with v3 and * where the reference is clearly better than v3. An unmarked entry is clearly worse than v3. "–": not run. `mlp_best+` is the per-cell minimum over the Part 7–9 MLP variants (`mlp_pool` excluded); it needs the Part 7 `mlp`, which has run at every cell here. `mlp_pool` is `mlp_strong` given v2's pooled sums (a control). `cnn` sees the unpermuted pixels (context, not a reference). Panel (b): v3 is seed 0 and the references are seed means. Panel (c): v3 and v2 are seed 0, and `mlp_strong`, `hgb` and v1 are seed means. "Fashion" is Fashion-MNIST. Source: `study/part10/LEARNER_RESULTS.json`; markers by `compare_hybrid.cells`.

5.1 The held-out sets qd_c and qd_d

`qd_c` and `qd_d` are ten fresh QuickDraw classes each (Section 3.1). They were fixed in a commit before any Part 10 design work. No result of a Part 10 learner on them was read before v3 was frozen (Section 3.4).

- *Runs.* v3, v2, v1, `hgb`, `mlp_strong` and `mlp_wide` ran 3 seeds at $n = 1000$ and $n = 10000$ and seed 0 at $n = 60000$. `mlp_xl` ran at $n = 60000$ only. The control `mlp_pool` ran on `qd_c` at $n \in \{10000, 60000\}$, seed 0, and the context `cnn` on `qd_c` at $n \in \{1000, 10000\}$, seed 0. These are the pre-registered cells. The Part 7 `mlp`, which enters `mlp_best`, was also run, with 3 seeds at every n (not pre-registered).
- *Where to look.* Panel (a) of Table 8 gives every held-out cell, panel (a) of Table 9 the verdicts, and Figure 2 the learning curves.

Against the MLP references [Numerical]. At all six held-out cells v3's error is lower than every reference's and the control's (the context `cnn`'s is lower at both `qd_c` cells where it ran).

- `mlp_strong` (ensemble of three 256-256 networks): v3 is clearly better at all 6 cells. Its error is 17–23% lower, and the mean $\log_2$ ratio is -0.31 (-0.32 at $n \geq 10000$).

	all n		$n \geq 10000$	
reference	cells (better/tie/worse)	mean $\log_2$ ratio	cells (better/tie/worse)	mean $\log_2$ ratio
(a) held out for v3 (Part 10): `qd_c`, `qd_d`				
`mlp_strong`	6 (6/0/0)	−0.314	4 (4/0/0)	−0.324
`mlp_wide`	6 (5/1/0)	−0.253	4 (3/1/0)	−0.235
`mlp_xl`	2 (0/2/0)	−0.138	2 (0/2/0)	−0.138
`mlp_best`	6 (6/0/0)	−0.314	4 (4/0/0)	−0.324
`mlp_best+`	6 (4/2/0)	−0.245	4 (2/2/0)	−0.223
`hgb`	6 (6/0/0)	−0.471	4 (4/0/0)	−0.473
v2	6 (0/6/0)	−0.097	4 (0/4/0)	−0.125
v1	6 (6/0/0)	−0.281	4 (4/0/0)	−0.305
`mlp_pool` (control)	2 (2/0/0)	−0.222	2 (2/0/0)	−0.222
`cnn` (context)	2 (0/2/0)	+0.125	1 (0/1/0)	+0.124
(b) Part 9 held-out sets, development data for v3: `qd_a`, `qd_b`				
`mlp_strong`	5 (5/0/0)	−0.337	3 (3/0/0)	−0.337
`mlp_wide`	5 (5/0/0)	−0.302	3 (3/0/0)	−0.291
`mlp_xl`	1 (1/0/0)	−0.204	1 (1/0/0)	−0.204
`mlp_best`	5 (5/0/0)	−0.337	3 (3/0/0)	−0.337
`mlp_best+`	5 (5/0/0)	−0.294	3 (3/0/0)	−0.276
`hgb`	5 (5/0/0)	−0.443	3 (3/0/0)	−0.460
v2	5 (0/5/0)	−0.089	3 (0/3/0)	−0.107
v1	5 (5/0/0)	−0.286	3 (3/0/0)	−0.314
`mlp_pool` (control)	3 (3/0/0)	−0.204	3 (3/0/0)	−0.204
`cnn` (context)	1 (0/0/1)	+0.245	1 (0/0/1)	+0.245
(c) development sets: `qd_dev`, *Fashion*				
`mlp_strong`	4 (2/2/0)	−0.207	2 (1/1/0)	−0.212
`mlp_best`	4 (2/2/0)	−0.207	2 (1/1/0)	−0.212
`mlp_best+`	4 (2/2/0)	−0.207	2 (1/1/0)	−0.212
`hgb`	4 (2/2/0)	−0.306	2 (1/1/0)	−0.287
v2	4 (0/4/0)	−0.062	2 (0/2/0)	−0.081
v1	4 (2/2/0)	−0.188	2 (1/1/0)	−0.211

Table 9: v3 against each image reference [Numerical]. "Cells" counts the (set, n) pairs compared, with v3 better / tied / worse by the cell rule (Section 3.3). "Mean $\log_2$ ratio" is the mean over those cells of $\log_2$(v3's error / the reference's error); negative means v3 is lower. `mlp_best` is the per-cell minimum over the Part 7–8 MLP variants (`mlp`, its regularized versions, `mlp_strong` and its single network `mlp_strong1`). `mlp_best+` adds `mlp_wide`, its single network `mlp_wide1`, and `mlp_xl`. Both need the Part 7 `mlp`, which has run at every cell here. Panels (b) and (c) compare single v3 fits (seed 0) with the references' seed means where 3 seeds exist (v2 in panel (c), `mlp_xl`, `mlp_pool`, `cnn` and all $n = 60000$ cells are seed 0). Source: `study/part10/LEARNER_RESULTS.json`.

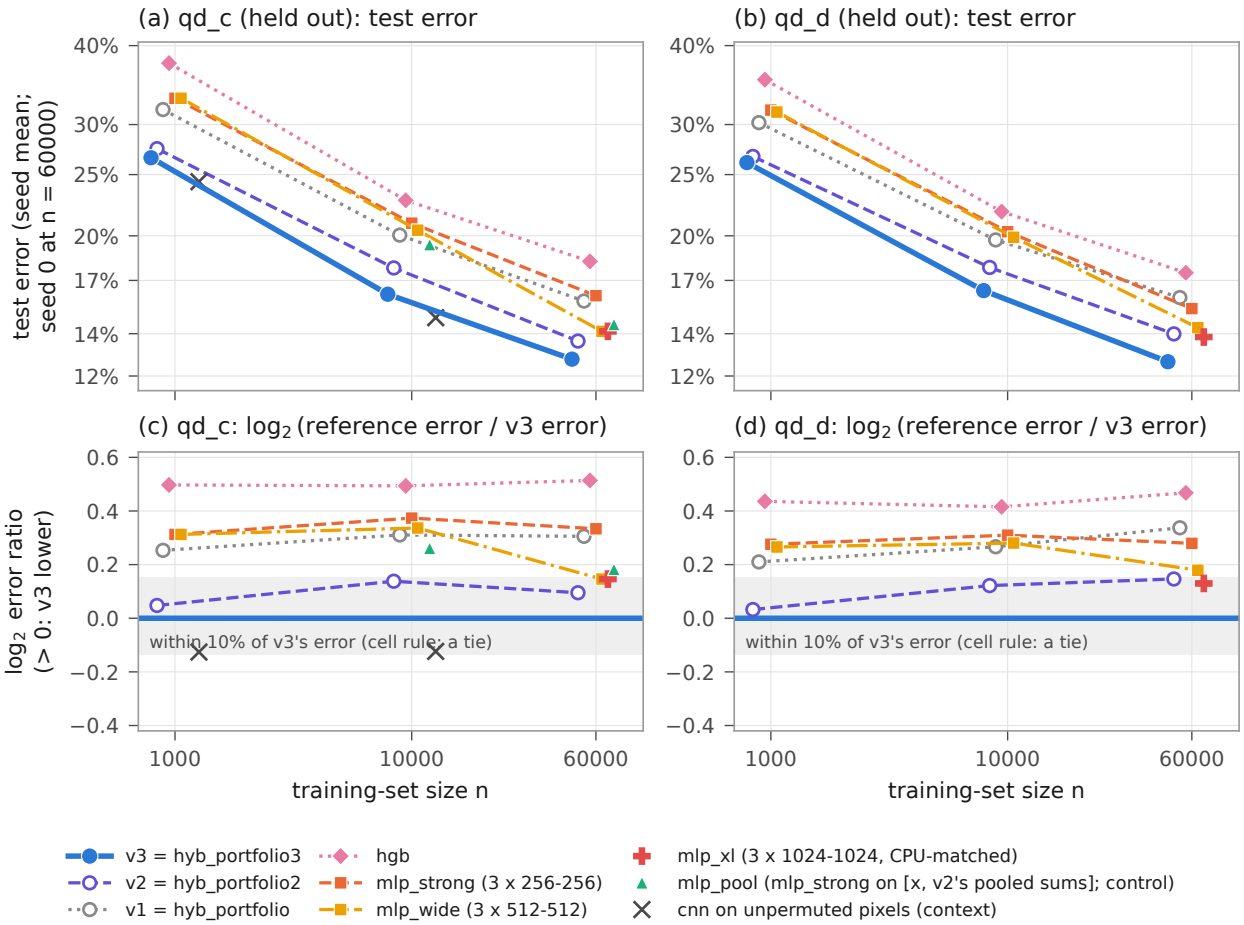


Figure 2: The held-out QuickDraw sets [Numerical]. Top: test error against n (log–log; seed mean of 3 at $n \leq 10000$, seed 0 at $n = 60000$; `mlp_xl`, `mlp_pool` and `cnn` seed 0). Bottom: each reference's error relative to v3's at the same n, as $\log_2$(reference error / v3 error), so points above 0 are cells where v3 is lower. In the grey band the two errors are within 10% of each other, and the cell rule calls the cell a tie whatever the noise. Markers are shifted slightly sideways so that they do not hide each other. Generated by `make_figures.py` (`fig_images`) from the result files; the plotted values are in `figures/fig_images.json`.

- `mlp_wide` (three 512-512 networks): clearly better at 5 cells. The sixth, `qd_c` at $n = 60000$, is a tie: 12.76% against 14.12%, 9.6% lower, just inside the 10% threshold. Mean −0.25.
- `mlp_xl` (three 1024-1024 networks, the CPU-matched reference; $n = 60000$ only): two ties. v3 is 9.6% lower on `qd_c` (12.76% against 14.12%) and 8.6% lower on `qd_d` (12.64% against 13.83%). Mean −0.14.
- `mlp_best` equals `mlp_strong` at all six held-out cells, so v3 is clearly better at all six (mean −0.31). `mlp_best+` adds the wider ensembles: 4 cells better and 2 ties, the two $n = 60000$ cells (mean −0.25). v3 is also clearly better than the Part 7 baseline `mlp` at all six cells (mean −0.53).

So at $n \leq 10000$ v3 is clearly better than every MLP reference. At $n = 60000$ the two widest ensembles come within 10% of it on `qd_c`, and `mlp_xl` also on `qd_d`.

Against the other references [Numerical].

- `hgb`: v3 is clearly better at all 6 cells, with errors 25–30% lower (`qd_c` 29–30%, `qd_d` 25–28%); mean $\log_2$ ratio -0.47.
- v1, the Part 8 portfolio: clearly better at all 6 cells, by 13.5–20.9% (`qd_c` 16.1%, 19.4%, 19.1% and `qd_d` 13.5%, 16.9%, 20.9% at $n = 1000$, 10000 and 60000); mean -0.28.
- v2: all 6 cells are ties. v3's error is lower at every cell, by 2.2–3.3% at $n = 1000$ and by 6.4–9.6% at $n \geq 10000$, but never by more than 10%. The mean $\log_2$ ratio is -0.10, and -0.125 at $n \geq 10000$.

The control and the context [Numerical].

- `mlp_pool` is `mlp_strong` given v2's pooled subset sums as extra inputs (seed 0, `qd_c` only). v3 is clearly better at both cells: 16.17% against 19.37% at $n = 10000$, and 12.76% against 14.48% at $n = 60000$. v2 ties it at both cells (17.79% and 13.63%, which are 8.2% and 5.9% lower), as it did on `qd_a` and `qd_b` in Part 9. This control matches v2's representation, not v3's: no MLP was given the bar sums (Section 5.3).
- `cnn`, a small convolutional network on the unpermuted pixels, is context, not a reference. Its `qd_c` runs (seed 0) give 24.36% at $n = 1000$ and 14.84% at $n = 10000$, 8.4% and 8.2% below v3's errors (26.58% and 16.17%): two ties by the cell rule, but lower at both cells.

The best-learner rule [Numerical]. The pre-registered rule does not use cell verdicts (Section 3.4). It makes v3 the learner of this paper if two conditions hold:

- the mean $\log_2$ error ratio of v3 to v2 over the four held-out image cells with $n \geq 10000$ is below 0. It is -0.125: the cells give -0.138 and -0.095 on `qd_c` ($n = 10000$, 60000) and -0.122 and -0.146 on `qd_d`;
- the mean $\log_2 n^*$ ratio of v3 to v2 on `dlheld7` is at most $+0.1$. It is -0.004 (Section 4.1).

Both hold, so v3 replaced v2. The rule was applied by `study/part10/compare_part10.py` (`study/part10/RESULTS.md`) after every held-out cell of v3 and v2 was in, and the decision is recorded in `study/part10/PROTOCOL.md` (28 October, 04:25 UTC).

How firm the gain over v2 is. By the cell rule it is not established at any single cell. Its direction is consistent: in all 14 seed-paired fits (same training sample, same test set), v3's error is below v2's, by 0.47 to 1.86 points [Numerical]. This is a description made after the results were in, not a pre-registered test. The 14 fits share two test sets, so they are not independent.

5.2 The other image sets: development data for v3

Panels (b) and (c) of Tables 8 and 9 give v3 at the cells of its fixed development evaluation. These sets are not held out for v3, and the cells are in-sample for a choice:

- *Tuning.* `p10_codes`'s builder tuned on validation rows of the training pools of `qd_dev`, `qd_a` and Fashion-MNIST (not `qd_b`; `study/part10/p10_codes/DESIGN.md`); `p10_hier`'s builder also used `qd_b`'s (`study/part10/p10_hier/DESIGN.md`). Neither tuned on a test set.
- *Selection.* After each freeze, the candidate's test error was computed once at 8 cells: `qd_dev` and Fashion-MNIST at $n \in \{1000, 10000\}$, and `qd_a` and `qd_b` at $n \in \{3000, 10000\}$. A pre-registered ninth cell, `qd_a` at $n = 60000$, checked error, budget and CPU time. The

integration rule then used these test errors to qualify the two candidates and choose between them (study/part10/RESULTS.md).

- *The choice.* `p10_codes` had a mean $\log_2$ ratio against v2 of -0.078 over the 8 cells, and at `qd_a`, $n = 60000$, 10.82% error, 1453 CPU s and 0.58 of the FLOP budget. `p10_hier` had -0.071, 11.08%, 924 s and 0.44. The rule took the lower mean.

The v3 numbers here are therefore the errors of the candidate that these very numbers selected. With two candidates this close the selection effect is small, but these cells are not a test of v3. Each v3 entry is one fit (seed 0).

`qd_a` and `qd_b` (held out in Part 9) [Numerical].

- *References.* v3 is clearly better than `mlp_strong`, `mlp_wide`, `hgb` and v1 at all 5 cells. It is also clearly better than `mlp_xl` at `qd_a`, $n = 60000$ (10.82% against 12.46%, 13.2% lower), and than `mlp_pool` at the 3 cells where both were run (at `qd_a`, $n = 60000$: 10.82% against 12.25%).
- *v2.* All 5 cells are ties, with v3 2.6–9.6% lower (mean $\log_2$ ratio -0.089). Here v2's entries are seed means. Against v2's seed-0 fits, as the integration rule compared them, the margins are similar: for example 17.48% against 18.75% at `qd_a`, $n = 3000$.
- *The context.* The `cnn` on unpermuted pixels is the only learner better than v3: 12.00% against 14.22% at `qd_a`, $n = 10000$, an error 15.6% lower. Among v3's cells, only the one at $n = 60000$ (10.82%) is below it.
- *Comparison with Part 9.* At $n = 60000$, v2 tied `mlp_xl` on both sets (11.97% against 12.46% on `qd_a`; 14.88% against 14.79% on `qd_b`). The control `mlp_pool` matched v2 (12.25% and 14.74%). v3's `qd_a` cell (10.82%) is clearly better than both. v3 was not run at `qd_b`, $n = 60000$.

`qd_dev` and Fashion-MNIST (development throughout) [Numerical].

- *`qd_dev`.* v3 is clearly better than `mlp_strong`, `hgb` and v1 at both cells, for example 13.30% against `mlp_strong`'s 17.49% at $n = 10000$. It ties v2 at both cells (21.18% against 22.22%, 13.30% against 14.63%).
- *Fashion-MNIST.* Against `mlp_strong` (= `mlp_best`), v2, v1 and `hgb` every cell is a tie. v3 has 17.97% and 12.38% against `mlp_strong`'s 18.60% and 12.64% (3.4% and 2.0% lower). Only the weaker baseline `mlp` (at $n = 10000$, 14.45%) and `mlp_reg` (20.32% and 14.17%) are clearly worse than v3. This is the set where the bars do not help. On the builder's validation rows, a booster on `cgbt`'s bank was worse there than one on v2's bank (18.30% against 17.74% at $n = 1000$; study/part10/p10_codes/DESIGN.md, §4.2). `cgbt`'s raw stacking weight is 0.05 and 0.07 there (shares of the total weight 0.07 and 0.10), against 0.23–0.50 (shares 0.28–0.68) on the QuickDraw development cells.
- *Larger n.* v3 was not run on Fashion-MNIST beyond $n = 10000$. v2's image path (`p9_gates`, Part 9, seed 0) reached 9.64% at $n = 60000$, against `mlp_strong`'s 9.66%.
- *A threading caveat.* v2's own cells on these two sets were run with multi-threaded BLAS, while every v3 and held-out run is single-threaded. A single-threaded refit of v2 at `qd_dev`, $n = 10000$, gives 14.67% instead of 14.63% (study/part10/PROTOCOL.md, note).

		v3: stacking weight					v2: stacking weight				out-of-fold error (%)			
set	n	cgbt	ggbt	gknn	knn	rest	ggbt	gknn	knn	rest	cgbt	ggbt	v3	v2
qd_c	1000	0.27	0.02	0.38	0.15	0.03	0.24	0.46	0.17	0.04	30.8	32.5	27.6	29.0
qd_c	10000	0.36	0.04	0.15	0.15	0.01	0.35	0.24	0.18	0.03	16.8	18.9	16.2	17.2
qd_c	60000	0.49	0.12	0.11	0.17	0.00	0.50	0.20	0.24	0.00	13.3	14.6	12.8	13.5
qd_d	1000	0.23	0.08	0.30	0.18	0.03	0.27	0.35	0.21	0.04	28.1	29.8	26.3	26.1
qd_d	10000	0.39	0.01	0.13	0.19	0.01	0.29	0.20	0.26	0.10	17.1	19.2	16.2	17.3
qd_d	60000	0.45	0.12	0.04	0.12	0.00	0.50	0.11	0.23	0.00	12.6	14.3	12.5	13.5

Table 10: What carries v3 and v2 on the held-out image sets [Numerical]. Left: stacking weight per member family (seed mean of 3 fits at $n \leq 10000$; one fit at $n = 60000$). `gknn`: k-NN votes on the pooled sums. `knn`: k-NN votes on the pixels. "rest": Part 8's `gbt`, `et` and `lin`; `lin` has weight 0 in every fit here. Right: out-of-fold error on the training rows of the two boosters and of the whole stack (5-fold at $n \leq 3000$; one holdout of 2000 rows at $n = 10000$ and 6000 rows at $n = 60000$). `ggbt`'s out-of-fold error is the same in v3 and v2 in every fit. The weights multiply log-probabilities of different calibration, so they are not shares of the decision. Source: `hyb_portfolio{3,2}__qd_{c,d}.info.json` in `study/standard_tasks/results/`.

5.3 Where the gain over v2 comes from

The same members, plus one [Numerical]. In each of the 14 held-out fits, v3's and v2's records give every shared member the same out-of-fold error, and `ggbt` the same number of boosting rounds. `cgbt` draws from its own random stream, so v3's other members are v2's fits. v3 and v2 therefore differ only by `cgbt` and by the stacking weights. The final refits also differ where a weight moves between zero and a positive value, because a member with weight 0 is not refitted. Table 10 gives the weights and the out-of-fold errors.

- *`cgbt` is the better booster.* Its out-of-fold error is below `ggbt`'s in all 14 fits: by 0.2–4.0 points at $n = 1000$, and by 0.13–0.20 in $\log_2$ at $n \geq 10000$. It is the lowest of all members in 13 of the 14 fits. The exception is `qd_c`, $n = 1000$, seed 1, where k-NN on pooled sums is lower (30.1% against 31.8%).
- *It takes `ggbt`'s place in the stack.* At $n \geq 10000$, `cgbt` has the largest family weight in all 8 fits (0.32–0.49). `ggbt`'s weight falls from 0.25–0.50 in v2 to 0–0.12 in v3. At $n = 1000$, the k-NN votes on pooled sums keep the largest family weight in 5 of 6 fits, in v3 as in v2.
- *The stack improves.* v3's stacked out-of-fold error is below v2's in 12 of the 14 fits. At `qd_d`, $n = 1000$, it is equal in one fit and higher in one (27.2% against 26.3%).

Two cautions about the weights apply (Section 6.2). They multiply log-probabilities of different calibration: boosters stopped early on error are over-confident, while k-NN votes are smooth. And the convex stacker splits weight between two nearly collinear boosters on one holdout. So "`ggbt` gets weight 0" describes one fit, not a property of the learner.

What `cgbt` changes: the bank, not the schedule [Numerical]. `cgbt` has `ggbt`'s tree settings and early-stopping rule. It differs from `ggbt` in three ways: its bank adds bar sums at two lengths, it drops the 300 prototype-pair projections, and its per-tree column fraction is 0.05 instead of 0.12 (Section 2.4). The builder's development runs separate these effects (one booster alone, $n = 5000$, validation rows, seed 0; `study/part10/p10_codes/DESIGN.md`, §4.2). On `qd_dev`:

- v2's bank gave 17.66% error, and dropping the pairs gave 17.20%;
- adding the bars at both lengths gave 15.92%, and the lower column fraction then gave 15.82%.

On `qd_a` the full change gave 17.68% → 16.78%. So on QuickDraw most of the gain is the bars. On

Fashion-MNIST the bars did not help. In `cgbt`'s fitted trees the bars, two thirds of its columns, carry 0.72–0.80 of the split gain on QuickDraw and MNIST (development fits; Section 6.2). Split gain describes how the trees split, not what the predictions depend on.

Compared with Part 9 [Numerical]. In Part 9, at $n = 60000$ on `qd_a` and `qd_b`, v2 tied the CPU-matched `mlp_xl`, and the control `mlp_pool` matched v2. Most of v2's large-n advantage over the MLPs was therefore the pooled-sum representation, which an MLP uses as well as the trees do. The Part 10 held-out sets repeat this for v2:

- v2 ties `mlp_xl` at $n = 60000$ on both sets (13.63% against 14.12%; 13.99% against 13.83%);
- v2 ties `mlp_pool` at both `qd_c` cells.

v3 moves past the control: it is clearly better than `mlp_pool` at both cells, by 16.5% and 11.9%. Against `mlp_xl` it is 9.6% and 8.6% lower, just inside the 10% threshold.

Reading [Heuristic]. On images v3's gain over v2 is a better fixed representation for gradient-boosted trees: sums over short runs of pixels along lines recovered from pixel correlations, used at fixed positions. Recovering the image topology from such correlations is an old idea [18], and so is grouping inputs into receptive fields by their correlations [7]. The gain is not evidence that discrete search beats gradient-based learning on images, for three reasons.

- *The control for v3's own representation was not run.* No MLP was given the bar sums as well as the pooled sums. Part 9's control matched v2 once it had v2's features, so part of v3's margin over the MLPs may again be available to a network given the same features.
- *Gradient-trained networks that learn local structure were not run.* For example, an MLP trained to learn sparse local connectivity [22] would be the information-matched comparison.
- *A network told the layout is better.* The small `cnn` (seed 0) has lower error at all three cells where both have run: clearly at `qd_a`, $n = 10000$ (12.00% against 14.22%), and within 10% at both `qd_c` cells (24.36% against 26.58%; 14.84% against 16.17%).

What v3 shows is narrower: a learner that never reads the pixel layout, and whose features are fixed subset sums chosen by rank statistics, reaches 12.6–12.8% error on fresh QuickDraw classes at $n = 60000$. That is 8.6–9.6% below a CPU-matched MLP ensemble, a tie by the cell rule, and clearly below an MLP given v2's representation (`qd_c`).

5.4 Compute

FLOPs [Numerical]. The budget of a real-data fit is three training runs of the baseline MLP: 1.24×10^{12} FLOPs at $n = 1000$, 2.42×10^{12} at $n = 10000$ and 6.21×10^{12} at $n = 60000$ (Table 11).

- *v3 and v2.* By its own count v3 uses 0.30–0.58 of the budget on the held-out cells, the most at $n = 60000$. v2 uses 0.22–0.42. `cgbt` itself is charged 0.14–0.22 of the budget; the net increase over v2 is 0.05–0.22, because `ggbt` is not refitted where its weight is 0.
- *Near the guard at $n = 3000$.* On the development sets, at $n = 3000$ (no held-out cell has this size), v3 used 0.85 of the budget on `qd_a` and on `qd_b`. That is close to the guard at 0.9, which skips members whose projected cost would exceed it (Section 2.2). Part 9's compute audit found that the FLOP charge of Part 8's `gbt` member is too low by a factor of 1.14–1.46 at this size, and it projected v2's $n = 3000$ QuickDraw cells at about 0.72 (0.69–0.81) of the budget, against v2's own count of at most 0.62. v3 keeps that charge, and no compute audit

		FLOPs / budget		CPU s per fit			MLP references: wall s per fit (FLOPs / budget)			
					of which					
set	n	v3	v2	v3	`cgbt`	v2	`mlp_strong`	`mlp_wide`	`mlp_xl`	`mlp_pool`
`qd_c`	1000	0.49–0.51	0.35	229–256	120–132	144–166	384 (2.0)	249 (5.0)	–	–
`qd_c`	10000	0.34–0.48	0.22–0.26	513–785	251–430	284–440	490 (2.0)	396 (5.0)	–	333 (5.9)
`qd_c`	60000	0.57	0.42	1361	668	1296	963 (2.0)	1168 (5.0)	2196 (9.2)	716 (5.9)
`qd_d`	1000	0.50–0.51	0.35	266–291	128–140	162–175	98 (2.0)	248 (5.0)	–	–
`qd_d`	10000	0.30–0.46	0.24–0.29	402–768	233–377	394–602	152 (2.0)	440 (5.0)	–	–
`qd_d`	60000	0.58	0.42	1464	760	1531	422 (2.0)	989 (5.0)	1335 (9.2)	–

Table 11: Compute per fit on the held-out image cells [Numerical]. Budget: three training runs of the baseline MLP (`common.budget_real`), 1.24, 2.42 and 6.21×10^{12} FLOPs at $n = 1000$, 10000 and 60000. FLOPs / budget: each learner's own count, everything in the fit included and prediction excluded. CPU s: process CPU seconds per fit, and the seconds v3 attributes to `cgbt`. A range is over the 3 seeds; $n = 60000$ has one fit. The MLP references record wall-clock seconds per fit on the shared 4-core machine, not process CPU time (the `cpu_seconds` field of their result files), with their FLOPs per fit in units of the budget in brackets. Sources: the result files and `.info.json` records in `study/standard_tasks/results/`.

was made of v3. Applying the audit's factor to `gbt` alone (0.28 of the budget by v3's count) puts these two cells at 0.88–0.98 [Heuristic]. So v3 is within the budget at $n = 3000$ by its own count, but not with a clear margin.

- *The MLP references.* Every MLP reference spends more FLOPs than the budget: `mlp_strong` 2.0 budgets per fit (six training runs), `mlp_wide` 5.0, `mlp_pool` 5.9 and `mlp_xl` 9.2. So by FLOPs v3 is cheaper than every MLP reference at every cell, by a factor of at least 3.4 (`mlp_strong` at `qd_d`, $n = 60000$).

CPU time [Numerical]. Equal FLOPs are far from equal CPU time.

- *v3 and v2.* A v3 fit takes 229–291 CPU s at $n = 1000$, 402–785 s at $n = 10000$, and 1361 and 1464 s at $n = 60000$. v2 takes 144–175, 284–602, and 1296 and 1531 s.
- *Why v3 is not slower at $n = 60000$.* There v3 takes about as long as v2 (1.05× on `qd_c`, 0.96× on `qd_d`), although it fits a second booster. `cgbt` takes 668 and 760 s. But v3's `ggbt` takes 482 and 474 s against v2's 1091 and 1305 s. The reason is that v3 sets LightGBM's column-wise layout, which leaves the predictions unchanged (`study/part10/p10_codes/DESIGN.md`, §1). Part 9 found this fix but did not apply it to v2.
- *A like-for-like estimate.* The comparison with v2 is not like for like. v3's time minus `cgbt`'s, 693 and 704 s, estimates v2 with the same layout. By that estimate the second booster about doubles the CPU time of a fit at $n = 60000$.
- *The MLP references.* At $n = 60000$ they take 963 and 422 s of wall time (`mlp_strong`), 1168 and 989 s (`mlp_wide`) and 2196 and 1335 s (`mlp_xl`); `mlp_pool` takes 716 s. So `mlp_xl` takes 0.9–1.6× v3's time (wall against CPU seconds), which is the sense in which it is CPU-matched. The other ensembles take 0.3–0.9 of v3's time.
- *How rough the MLP times are.* They are wall-clock seconds on the shared machine, and they vary with the load from concurrent jobs. The same `mlp_strong` recipe took 384 s per fit at `qd_c`, $n = 1000$, and 98 s at `qd_d`, $n = 1000$. In Part 9 it took 28 s at `qd_a`.

Prediction is not charged for any learner. A v2 query costs about 2.8×10^8 FLOPs at $n = 60000$, about 500× an MLP's, because the k-NN members scan the training set (Part 9). v3 adds `cgbt`'s

bank projection, about 1.2×10^7 FLOPs per query, and up to about 4000 more tree traversals (study/part10/p10_codes/DESIGN.md, §6).

Summary of the image results.

- *Strong points* [Numerical]. On the two held-out sets v3 is clearly better than `mlp_strong` at every cell, and than every MLP reference at $n \leq 10000$. It is clearly better than the control `mlp_pool` at both cells where that was run. Its error is below every reference's at every held-out cell (the context `cnn`'s is lower at both `qd_c` cells).
- *Weak points* [Numerical]. At $n = 60000$ the CPU-matched `mlp_xl` is within 10% of v3 on both sets (two ties), and `mlp_wide` on `qd_c`. v3's gain over v2 is below the cell rule's threshold at every cell; the best-learner rule, which uses mean $\log_2$ ratios, still prefers v3. On Fashion-MNIST v3 is level with `mlp_strong`. The `cnn` that sees the layout (seed 0) has lower error at all three cells where both have run: clearly at `qd_a`, $n = 10000$ (12.00% against 14.22%), and within 10% at both `qd_c` cells. A fit takes 23–24 CPU minutes at $n = 60000$. The FLOP count reaches 0.85 of the budget at $n = 3000$, and with Part 9's audited correction it would be close to 1.
- *What it means* [Heuristic]. The gain over v2 comes from a richer fixed feature bank for gradient-boosted trees, not from a new kind of learning. Whether an MLP would gain as much from the same bank was not tested.

6 What it learns

This section describes what v3's fitted members contain on the toy targets (Section 6.1) and on images (Section 6.2). The analysis is descriptive. The protocol fixed it as analysis of the chosen learner, to be run after the held-out results were in, and not as evidence that the learner is better than anything study/part10/PROTOCOL.md. In practice it ran in parallel with the held-out evaluation, on v3 before the best-learner rule had chosen it, and touched none of the rule's inputs (Appendix A.4). It says what structure the fits hold and when it appears. It does not show that this structure is why v3 does well: nothing here is an ablation of the learner.

Fits. Every analysed fit is seed 0, and each (target, n) or (set, n) pair is one fit. The fits are instrumented reruns of the evaluation's own fits: the same training rows, column permutation and random streams, with the learner's internals recorded.

- *Toy.* At all 875 cells of v3 (every toy suite) and all 477 cells of v2 (the six sum-structured suites) that have an evaluation result, the reruns' test errors equal the evaluation's seed-0 errors. Equal errors at every cell are strong evidence that the fits are the same, but not a proof. The analysis's own check (`check_identity` in study/part10/interp/toy_interp.py) covered 587 of the v3 cells, because v3's result files on `random` and `dlheld`–`dlheld3` were incomplete when it ran. paper_learner/check_toy_identity.py repeats the same comparison on the final result files (output: `check_toy_identity.json`); it adds the 288 cells of those four suites, all equal.
- *Images.* The four v3 fits that have an evaluation cell (`qd_dev` at $n = 1000$ and 10000, `qd_a` and Fashion-MNIST at $n = 10000$) reproduce its test error, stacking weights, `ggbt`'s number of rounds and every member's out-of-fold error. The fits at `qd_dev`, $n = 3000$, and MNIST, $n = 10000$, have no cell and are unchecked. No result on `qd_c` or `qd_d` was opened.

Sources and checks.

- *Toy analysis:* `study/part10/interp/TOY_INTERP.md`.
- *Image analysis:* `study/part10/interp/IMAGE_INTERP.md`. The scripts, tables and result files of both are in the same folder.
- *The verifications:* `study/part10/interp/VERIFICATION.json`. Two independent verifiers per side: one recomputed every number from the files, the other reviewed the readings against the files and the code. Where a verifier corrected a number or a reading, this section uses the corrected one.
- *This section's figures and tables* are generated from the analysis files by `make_interp.py`, which applies those corrections. The plotted values are in `figures/fig_interp.json`.

Measured quantities are [Numerical]. Descriptions of maps and explanations of mechanisms are [Heuristic].

6.1 Toy targets: what DFG holds, and when

On toy inputs v3 is a stack of DFG v1.3 (a default variant and, at $n \leq 512$, a forced small-n variant) and the classical members (Section 2.2). The analysis looks at DFG's final model in each fit. It covers the 54 sum-structured targets of the six suites built around signed sums (`dlsymdev`, `dlfamdev`, `dlheld4`–`dlheld7`; the 14 non-symmetric controls are excluded) and, more briefly, the 70 targets of the other suites. The grid is $n = 16, 32, \ldots, 16384$, stopped after two consecutive sizes with test error ≤ 0.005.

Definitions.

- *Terms and shares.* The model's terms are its characters, sum blocks (Section 2.3), spectral features and the kernel background. A term's *share* is the standard deviation of its contribution to the model's output over the 4096 test inputs.
- *Exact structure.* The target's signed-symmetry classes are the sets of inputs it treats alike, up to sign. A class of size ≥ 3 is *matched exactly* when a sum block, a factor of a product block or the modulator χ_M of a modulated block has the same input set and the same signs up to a global flip. n_{struct} is the first grid size at which every class of size ≥ 3 is matched in the same model. For a weighted target $u = s + a\tau$ the class is the unit-weight part s; the inner term is checked separately (below).
- *Detection.* n_{detect} is the first grid size at which DFG's own block test (2) would admit the *true* block (the largest class, with its signs) if it were proposed in round 0 on the seed-0 training rows.
- *Low-degree share.* W_k is the target's Fourier weight at degree k, and $(W_1 + W_2)/(1 - W_0)$ is the share of its variance at degrees 1 and 2.

What these measurements can and cannot show.

- The exact-structure criterion checks the input set and the signs, not the learned function of the block or its fold. It still tracks the function closely: at n_{struct} the DFG member's test error is at most 0.03 for every target found. It is also strict: some models fit a target exactly with another representation, and those count as not found (below).

	targets	exact structure	median n_{struct}	range of n_{struct}	detectable at $n = 128$	delay (doublings) < 0 / 0 / 1 / ≥ 2	never found
by mixture							
pure symmetric	25	22	256	64–8192	25	1 / 9 / 3 / 9	fd_10, f7_05, h6_00
with a parity	9	8	256	128–16384	9	0 / 3 / 3 / 2	h6_02
with a gate	5	5	256	128–1024	0	2 / 3 / 0 / 0	–
multiplexer	4	3	512	512–1024	0	0 / 2 / 0 / 1	f7_11
two sums	5	5	256	256–512	0	3 / 2 / 0 / 0	–
weighted sum	6	6	256	128–2048	5	0 / 1 / 3 / 2	–
by accept set (a two-block target counts once under each kind it has)							
threshold	11	11	256	128–2048	7	2 / 3 / 4 / 2	–
band	19	18	192	64–1024	10	4 / 11 / 1 / 2	f7_11
interval	10	9	256	128–1024	6	2 / 5 / 2 / 0	h6_02
arbitrary set	16	15	512	128–16384	13	0 / 3 / 2 / 10	f7_11
residue class	5	2	768	512–1024	3	0 / 1 / 0 / 1	fd_10, f7_05, h6_00
all	54	49	256	64–16384	39	6 / 20 / 9 / 14	fd_10, f7_05, f7_11, h6_00, h6_02

Table 12: Exact structure in v3's DFG model on the 54 sum-structured targets, seed 0 [Numerical]. "Exact structure": targets whose model matches every signed-symmetry class of size ≥ 3 at some analysed n; n_{struct} is the first such n. "Detectable at $n = 128$": the true block passes DFG's test (2) on the seed-0 training rows at $n = 128$. For the five weighted targets other than sumpar the tested block is the weighted one, with v1.3's larger threshold. For the gate and multiplexer mixtures the detection points are the verifier's, recomputed with the residual deflated by the modulator as DFG tests it; the analysis file has 4 more targets at $n = 128$. "Delay": $\log_2(n_{\text{struct}}/n_{\text{detect}})$ over the targets found. In the lower panel a target with two blocks counts once under each accept kind it has. Sources: `study/part10/interp/tables/toy_timing.csv` and `study/part10/interp/VERIFICATION.json`, via `make_interp.py`.

- n_{detect} uses the true block, an oracle. It measures what the test would accept, not what the search would propose. For mixtures it is a rougher proxy, because DFG tests a block in later rounds on a residual from which the gate or parity characters are already fitted out.
- The summarised model depends on the stacker. When both DFG variants are kept, only the forced one is seen; when neither is kept, the summary is the last out-of-fold DFG fit, on fewer rows.

Exact structure is recovered on 49 of 54 targets [Numerical]. Table 12 gives the counts.

- *When.* The median n_{struct} is 256 (range 64–16384). 15 targets have the exact structure by $n = 128$, 30 by 256 and 41 by 512.
- *By suite.* `dlsymdev` 8 of 8, `dlfamdev` 8 of 9, `dlheld7` 7 of 9, `dlheld6` 7 of 9, `dlheld5` 10 of 10, `dlheld4` 9 of 9.
- *Never found* (by the largest analysed n; f7_11's and h6_02's runs stopped at 4096 and 1024 because the portfolio's error was already ≤ 0.005): the three residue classes mod 3 of a 10–14-input count (fd_10, h6_00, f7_05), and two targets that DFG fits by another representation. On f7_11, a multiplexer between a 4-input and a 6-input block, each class is matched at some n (128 and 2048), but never both in one model; at $n = 4096$ the 4-input branch is carried by characters of degree 2–3. On h6_02 $= [1 \leq s \leq 5]\, x_{10}x_{11}$, the model is a modulated block plus characters at $n = 512$ (DFG error 0.002), and at $n = 1024$ the degree-11 character on $x_1, \ldots, x_{11}$ plus degree-3 characters.

- *Accept kinds.* Of the accept kinds learned on most of their targets, arbitrary sets need the most data (median 512; h5s7 8192, h5s1 16384). The only residue-class blocks that are learned are the 6-input multiplexer branches of f7_03 (mod 4, $n = 512$) and h6_08 (mod 3, $n = 1024$).

What the model looks like when it has the structure [Numerical]. The leading terms at each target's largest analysed n take a few recurring forms.

- *Pure symmetric targets.* One block on the class with its signs, folded ($|t|$) for an accept set symmetric under $t \to -t$ and unfolded otherwise. Examples: band11, $|x_1 + \cdots + x_{11}|$ with share 0.97; h6_01, $(-x_1+x_2-x_3+x_4+x_5+x_6-x_7-x_8-x_9)$ with share 0.91. One exception: h5s6, with the asymmetric set $\{-9,-5,-1,1,5\}$, gets a folded block. In 11 of the 22 pure targets found, a spectral feature whose support contains the block's inputs (and 9–17 others) sits next to the block.
- *Thresholds.* Degree-1 characters carry as much as the block. On f7_01 the 13-input block has share 0.26, next to $\chi_{\{4\}}$ and $\chi_{\{1\}}$ at 0.19 each.
- *A parity times a symmetric function.* The parity enters first as a character and then modulates a block: fd_04 is $\chi_{\{7,8,9\}} \cdot |x_1 + \cdots + x_6|$, share 1.06.
- *Gates and multiplexer selectors.* They enter as characters and, in v3, as the heavy inner literal of a weighted block: $(x_1 - x_2 - \cdots - x_6 + 3x_{11})$ on h6_07 (share 0.40, from $n = 1024$; at $n = 512$ the selector is the inner literal of the 4-input branch's block), $(x_1 + \cdots + x_6 + 3x_{11})$ on f7_03 (0.24).
- *Two sums.* A product block over the joint cells, as on twosym ($|x_1 + \cdots + x_5| \times |x_6 + \cdots + x_{10}|$, share 0.72), or a block over the union with one side's signs flipped.
- *A quirk.* In 15 cells (13 targets) the true block appears inside a product with a factor on two irrelevant inputs (shares 0.13–0.40), for example $|x_1 + \cdots + x_7| \times |x_{15} - x_{16}|$ on sym7 at $n = 128$ (share 0.40). The test error there is at or near 0. The product test scores the whole joint partition, not its increment over the stronger factor, so a strong block carries any second factor through the test (confirmed in the code).

Weighted targets: is the inner term right? [Numerical]. Six targets are weighted sums $u = s + a\tau$.

- *h6_10* $= [|s - 3x_{13}| > 1]$, $s = x_1 + \cdots + x_{12}$. At $n = 256$ the model holds $|s + 3x_{13}|$: the weight is right and the relative sign wrong. Its DFG error, 0.017, falls entirely in the four cells $(t, x_{13}) = (\pm 8, \mp 1)$ and $(\pm 10, \mp 1)$, 68 of 4096 test points. From $n = 512$ the block is exact.
- *f7_10* $= [s + 2x_6 \leq -1]$: a block modulated by $\chi_{\{6\}}$ at $n = 128$, the exact $(x_1 + \cdots + x_5 + 2x_6)$ at 256.
- *h5t0*: a product with $|x_8 + x_9|$ at 256, the exact weighted block at 512.
- *thrsum and h5t1* are fitted through a product with a factor on the parity's two inputs. In h5t1 the weight is 3 where the truth has 2.
- *sumpar* has a degree-3 inner term, outside v1.3's class. Characters carry it: DFG error 0 at $n = 256$, 0.013 at 512. Both classes are matched in one model only at 2048.

A product with $|x_j \pm x_l|$ represents every function of $(s, x_j x_l)$, so the product forms are exact representations, though not the weighted ones. The five targets other than sumpar are fitted to DFG error 0 at their largest analysed n.

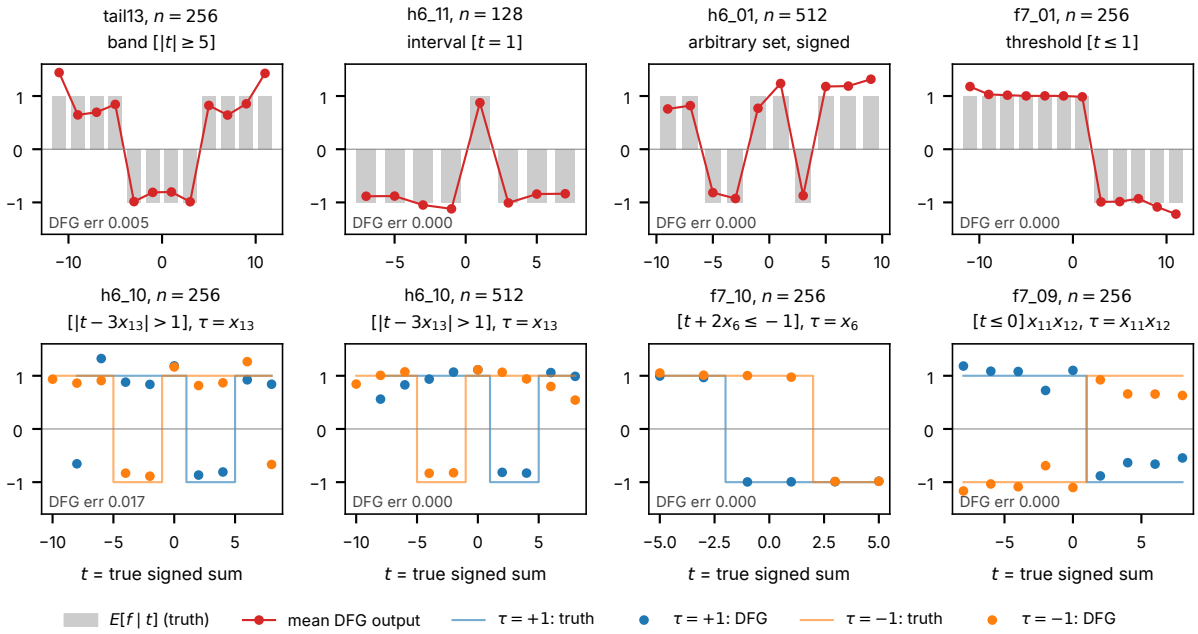


Figure 3: The learned function of the true block statistic t (seed 0) [Numerical]. Top: grey bars are $E[f \mid t]$ over the 4096 test inputs; red points are the mean output of the whole DFG model at each level of t. Bottom: the same split by the second term τ, from refits that reproduce the analysed cells. The plotted value is the conditional mean of the whole model, which includes spectral features, characters and stray product terms, not the block's own coefficients. Levels with fewer than 5 test points are not drawn; for h6_10 at $n = 256$ these include the cells $(t, x_{13}) = (-10, +1)$ and $(10, -1)$, where the model is also wrong. "DFG err": the DFG member's test error. Sources: the structure runs and `toy_g` refits in `study/part10/interp/results/toy_final/`; figure by `make_interp.py`, redrawn from the data of the analysis's `fig_toy_g.pdf`.

The learned function of the block [Numerical] (Figure 3). Where the block is present, the model's mean output follows $E[f \mid t]$ level by level. This includes non-convex accept sets, such as h6_01's $\{-9, -7, -5, -1, 1, 7, 9\}$, and both branches of a weighted or modulated target. The values are not ± 1: the logistic readout rescales them and overshoots at rare extreme levels (tail13 at $|t| = 11$). The panels add shape, not a new test. The share of test points at which the mean output has the sign of $E[f \mid t]$ is about one minus the DFG error by construction (0.983 for h6_10 at $n = 256$, 1.00 for the other seven panels). Below the discovery size the model has no block structure: for h6_10 at $n = 128$ the mean output is positive at every (t, x_{13}) level (0.01–0.90), so its sign agrees with $E[f \mid \cdot]$ only on the majority's 0.70 of the points, and the DFG error (0.39) is worse than a constant predictor's (0.30). The eight panels were chosen by hand; over all 53 exact cells of the 22 pure targets found, the agreement is at least 0.985 (`study/part10/interp/VERIFICATION.json`, toy reading, C8).

Timing: the 25 pure symmetric targets [Numerical]. On these targets the analysis is cleanest: the target is a function of one signed sum, and the round-0 test on the true block is close to the test DFG applies. Figure 4a shows each target.

- *Detection is cheap.* The true block passes DFG's test at $n = 128$ for all 25 targets. The test statistic grows linearly in n: its ratio to the threshold is at most 0.89 at $n = 64$ and 1.07–1.78 at $n = 128$.

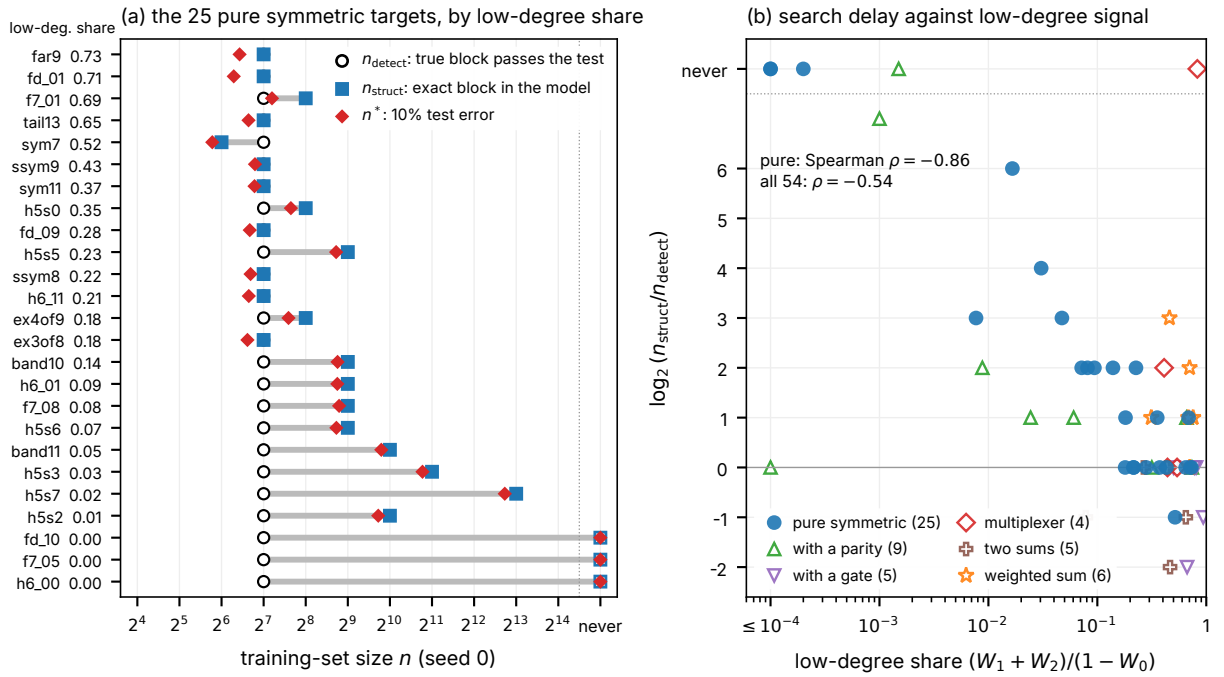


Figure 4: When the structure appears (seed 0) [Numerical]. (a) The 25 pure symmetric targets, sorted by their share of variance at Fourier degrees 1–2 (number after the name). Open circle: n_{detect}, the first grid size at which DFG's test would admit the true block. Square: n_{struct}, the first grid size at which the model holds it exactly (open at "never"). Diamond: n^*, the 10% test-error point of the same seed-0 fits, interpolated in $\log n$. The grey bar spans detection to discovery. (b) The delay $\log_2(n_{\text{struct}}/n_{\text{detect}})$ against the low-degree share for all 54 sum-structured targets; filled: pure symmetric, open: mixtures. Targets never found are drawn in the top row. Shares below 10^{-4} are drawn at 10^{-4}. For the mixtures the detection points are the verifier's corrected ones (Table 12). Spearman ρ counts a target never found as $n_{\text{struct}} = 2^{15}$. No point is jittered. This figure replaces the analysis's `fig_toy_timing.pdf`, which used the uncorrected detection points. Sources: `study/part10/interp/tables/toy_timing.csv` and `study/part10/interp/VERIFICATION.json`; figure by `make_interp.py`.

- *Discovery is later where the low-degree signal is weak.* 22 of the 25 are found. Against n_{detect}, n_{struct} is one doubling earlier for 1 target (sym7, held by the forced variant below the threshold), equal for 9, one doubling later for 3, and two or more later for 9 (band11, f7_08, h6_01, h5s2, h5s3, h5s5, h5s6, h5s7, band10).
- *The delay tracks the low-degree share* (Figure 4b). Spearman's ρ between the delay and $(W_1+W_2)/(1-W_0)$ is -0.86 over the 25 targets (a target never found counted as $n_{\text{struct}} = 2^{15}$), and -0.80 over the 22 found. The targets come from a few designed families, so a p-value that treats them as independent draws would overstate the evidence.
- *The readout is not the bottleneck.* For all 22 targets found, the seed-0 n^* falls in the doubling just below n_{struct}: the test error crosses 10% between the last grid size without the exact block and the first with it. Because n_{struct} is a grid point and n^* is interpolated, a finer statement cannot be made. In particular, the median offset of $\log_2(n^*/n_{\text{struct}})$ (-0.35 over all 49 targets found) reflects the grid, not the readout.

Reading [Heuristic]. For a function of one signed sum, DFG's data cost is set by its search, not by its test and not by its readout. Every seed of the block search reads the block's inputs off degree-1 and

degree-2 statistics of the residual: prefixes of the top eigenvectors of the pair matrix, the degree-1 order and greedy growth from strong pairs (Section 2.3). The weaker those statistics, the more data the search needs before it proposes the block, although the test would accept the block much earlier. No intervention isolates the search (such as seeding it with the true block); acceptance in later rounds and the stacker's choice of variant could also contribute.

Timing: the 29 mixtures, with caveats [Numerical]. For targets with a parity, gate, selector or second block, the round-0 test on the true block is a cruder proxy for what DFG does.

- *A corrected detection test.* The analysis script tested a modulated block on a residual that was neither centred nor fitted on the modulator, and so it counted the modulator character's own signal as block signal. DFG tests modulated blocks on a residual already fitted on χ_M. A verifier recomputed the statistic that way. For pure parity mixtures nothing changes. For seven gate and multiplexer targets the detection point moves later: fd_11 and h5s4 from 128 to 256, f7_03 from 128 to 512, f7_11 from 128 to 1024, h6_07 from 256 to 512, and majsym and fd_08, whose modulated statistic never passes, to the points where their plain block passes (512 and 2048).
- *With the corrected points*, 39 of the 54 targets are detectable at $n = 128$ (43 in the analysis file), and the median delay over the 49 found is 0 doublings (1 in the analysis file). Spearman's ρ between delay and low-degree share is -0.54 over all 54 (-0.44 with the uncorrected points).
- *Where the proxy fails.* fd_08 is found at $n = 1024$, before its plain block passes the round-0 test at 2048: in DFG's own run the gate's characters are fitted first. For the weighted targets detection uses the weighted block but n_{struct} the unit-weight class, so h5t1 counts as two doublings late only because of the definitions.
- *Search orders.* For 8 of the 49 targets found, the search's spectral and degree-1 orders never reach the block on the plain residual: the 7 parity mixtures fd_02, fd_04, symxor, h5s1, h6_06, f7_09, symxpar, and the multiplexer h6_07. On 22 targets the block is found before those orders list its inputs, presumably through greedy growth, local refinement or, at $n \leq 128$, the beam (not measured). The reach measure covers only the eigenvector and degree-1 orders.

The failures: residue classes mod 3 [Numerical]. Let $B = (s + k)/2$ count the k signed inputs of the block that equal $+1$. Then fd_10 $= [B \bmod 3 = 0]$ on $k = 10$ inputs, h6_00 $= [B \bmod 3 \neq 1]$ on 13 and f7_05 $= [B \bmod 3 \neq 0]$ on 14. Neither v3 nor v2 learns them (Section 4.4). Figure 5 shows why.

- *The model.* At every $n \geq 1024$ (fd_10 from 128, f7_05 from 256) the DFG model is the constant plus the kernel background. Its test error at $n = 16384$ (0.32–0.34) is the test set's minority rate. For fd_10 and h6_00 these errors come from out-of-fold DFG fits on 13107 rows, which the stacker gave no weight. At smaller n the forced variant adds 1–2 spurious characters or a spurious partial block (h6_00 at $n = 256$: $|-x_4 + x_{12} - x_{22} - x_{23}|$, 2 of its 4 inputs in S).
- *The test would accept the block.* The true block passes from $n = 128$ unfolded (512 folded). At $n = 16384$ it is 155, 139 and 134 times the threshold.
- *There is no low-degree signal.* For h6_00 and f7_05, $W_1 = W_2 = 0$ to five decimals; for fd_10, $W_1 = 4 \times 10^{-5}$ and $W_2 = 1.7 \times 10^{-4}$. The Fourier weight peaks at degrees 7, 9 and 11.

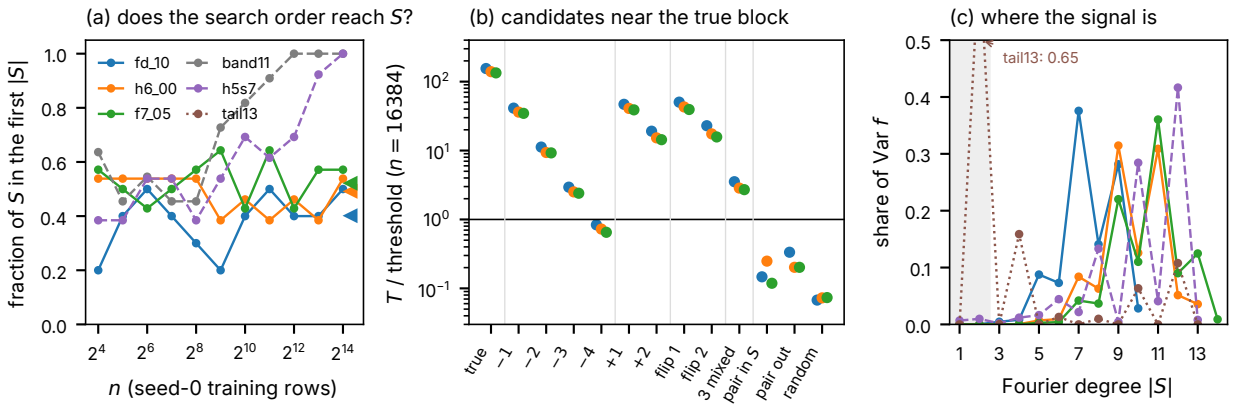


Figure 5: Why the residue classes mod 3 are never found (seed-0 training rows) [Numerical]. (a) The fraction of the true class S among the first $|S|$ inputs of the better of the search's two eigenvector orders. Triangles at the right edge: the value expected from the better of two random orders (0.40, 0.49, 0.52 for $|S| = 10, 13, 14$). band11 and h5s7 (dashed in both panels) are found at the first n at which all of S lies within the order's first $|S| + 5$ inputs (plotted fraction 0.82 and 0.92). The legend of (a) also serves (c). (b) DFG's test statistic over its threshold at $n = 16384$ for the true block and for candidates near it: medians over 200 random candidates with 1–4 inputs dropped, 1–2 added, 1–2 signs flipped, or 3 errors of mixed kinds; the best 2-input block inside S; the best pair among 12 inputs outside S; random blocks of the true size. Above 1 the candidate passes. (c) The share of each target's variance by Fourier degree; degrees 1–2 shaded. tail13, found at $n = 128$, has 0.65 at degree 2 (clipped). Sources: `power/` and `toy_landscape/` in `study/part10/interp/results/toy_final/`; figure by `make_interp.py`, a compact version of the analysis's `fig_toy_failures.pdf` with the chance level of panel (a) corrected for the better of two orders.

- *The search's orders do not reach the block* (Figure 5a). At $n = 16384$ the eigenvector orders put 0.50, 0.54 and 0.57 of S among their first $|S|$ inputs, against 0.40, 0.49 and 0.52 for the better of two random orders, and the fraction does not grow with n.
- *The basin is narrow* (Figure 5b). At $n = 16384$ the statistic falls about 4× per missing input (to 41, 36 and 35 times the threshold with one input dropped; 2.9, 2.5 and 2.4 with three; 0.84, 0.73 and 0.65 with four). The first wrong sign or extra input costs 3.1–3.4×, the second 2.2–2.7×. Every sampled candidate with three errors of mixed kinds still passes. But no piece of the block carries signal: the best pair inside S is at 0.15, 0.25 and 0.12 of the threshold, at the level of the best pair among 12 inputs outside S (0.33, 0.20, 0.20), and random blocks of the true size are at 0.07. The basin estimate rests on medians of sampled candidates; other 4-error mixes were not measured.
- *A needle.* Within three errors of the true block there are $\sum_{j\le 3} 2^j \binom{30}{j} = 34281$ candidates, against $\binom{30}{k}2^{k-1} = 1.5 \times 10^{10}$, 4.9×10^{11} and 1.2×10^{12} blocks of the true size $k = 10, 13, 14$.

Reading [Heuristic]. A residue class mod 3 of a 10–14-input count is detectable once proposed and invisible from every low-order foothold the search has. For h6_00 and f7_05 the single and pair statistics are zero in expectation, so more data cannot help the seeds. For fd_10 they are about 0.002, which the seeds could resolve only at n of order 10^6. Of the networks, only the most regularized variant learns h6_00 ($n^* = 12304$), and none learns f7_05 (Table 4).

Late targets [Numerical]. The pure targets found late ($n_{\text{struct}} \ge 1024$, two or more doublings after detection), and h5s1, are consistent with the same mechanism in weaker form. (h6_08 and sumpar

are also late, for representational reasons; sumpar's inner term is outside v1.3's class.) Each is one seed, so the coincidences below are suggestive, not established.

- *band11* $= [|x_1 + \cdots + x_{11}| \in \{3, 5\}]$ ($W_1 = 0$, $W_2 = 0.047$): detectable at 128, found at 1024, the first n at which the top eigenvector order lists all 11 inputs among its first 16. At $n = 512$ one sign flip drops the statistic from 5.4 to 0.61 of the threshold, and pairs inside S are at noise level (0.27).
- *h5s7*, an arbitrary set of a 13-input signed count ($W_1 + W_2 = 0.015$): detectable at 128, found at 8192, when the eigenvector order covers 0.92 of S. At $n = 2048$ the true block is 18.6 times the threshold, but only 12% of candidates with three mixed errors pass.
- *h5s1*, a signed 9-input count times the parity $x_{10}x_{11}$: 92% of its variance is on the single degree-11 character $x_1 \cdots x_{11}$. The plain block carries no signal. The modulated block passes from $n = 128$, but DFG takes modulators only from characters already in its model, and $\chi_{\{10,11\}}$ has coefficient -0.031. So the bottleneck is detecting the modulator. It is found at $n = 16384$ on seed 0, and the 3-seed n^* is "never".
- *h5s2, h5s3* (arbitrary sets; $W_1 + W_2 = 0.007$ and 0.03): found at 1024 and 2048.

What v1.3 changed [Numerical]. v2's DFG is v1.2; v3's is v1.3, which adds weighted blocks, forced blocks at $n \geq 48$ and a beam at $n \leq 128$ (Section 2.3). Structure runs of both versions on the six sum-structured suites (68 targets with the controls) show where the changes act; Table 6 gives the targets whose n^* moved. Over the 64 targets with a finite 3-seed n^* for both, the mean $\log_2 n^*$ ratio of v3 to v2 is -0.116 (in-sample: every suite except `dlheld7` is development data for v3, and the toy builder developed on `dlheld6`; on the held-out `dlheld7` the mean is -0.004, Section 4.1).

- *Weighted blocks change what is learned* on four targets whose exact structure v2's models never match: h6_10 ($n_{\text{struct}} = 256$ in v3; n^* 702 $\to$ 206), h6_07 (512, with the selector as inner literal; 678 $\to$ 447), h6_08 (1024; 446 $\to$ 449) and sumpar (2048, with $(x_1 + \cdots + x_5 + 2x_6x_7)$; 164 in both). v2 does represent h6_10 exactly from $n = 512$, by two folded 13-input blocks $|s + x_{13}|$ and $|s - x_{13}|$ that the matcher does not count. v3's model is nearly right one doubling earlier ($n = 256$: DFG error 0.017, with the inner sign wrong, against v2's 0.33), and both are exact from 512. On f7_10 both versions have $n_{\text{struct}} = 128$, but at $n = 256$ only v3's model holds the exact $(x_1 + \cdots + x_5 + 2x_6)$.
- *The beam and forced blocks move the first exact block one doubling earlier* on five pure targets (sym7 to 64; fd_09, tail13, sym11 and h6_11 to 128) and on the gate mixture majsym (to 128). On sym7 and majsym the block is in the forced variant, below the threshold (0.89 of it for sym7; 0.36–0.39 for majsym's plain block): forced, not admitted. For fd_09, tail13, sym11 and h6_11 the true block already passes the test at 128 (1.34–1.62 times the threshold), so the earlier find is a difference in the search, not attributable to the beam specifically.
- *Later or lost.* h5t1's n_{struct} moves from 256 to 512, with identical portfolio errors. f7_11 loses the joint structure: v2 holds both classes at 2048 and 4096, v3 never in one model (n^* 55 in both).
- *The same n_{struct} is not the same fit.* 41 targets have the same n_{struct} in both versions. On them the 3-seed n^* still moves, from -0.30 (far9) to $+0.26$ (fd_06) in $\log_2$ (mean -0.005 over 40), and the portfolio's errors differ at some analysed n on 26 of the 41. They are identical at every analysed n on 20 of the 68 targets.

The attribution of each change to weighted blocks or to the small-n search is read off what v3's model holds at its n_{struct}. No ablation separates the three additions, so these labels are [Heuristic].

Where weighted blocks appear [Numerical]. At some n, 26 of the 68 targets have a weighted block with at least 20% of the largest term's share. Its inner term τ is one of five kinds.

- the true weighted term: h6_10, f7_10, h5t0, h5t1;
- a parity partner or one of its literals: symxpar ($+3x_7x_8$), h6_06 (first $+2x_7$ at $n = 512$, then $\pm 3x_7x_8$), f7_07, symxor ($-2x_8$);
- a multiplexer's selector: h6_07, h6_08, f7_03 ($\pm 3x_{11}$);
- the other inputs of a gate: h5s4 ($+3x_6$), symand ($+3\,\text{AND}(-x_8, -x_7)$), majsym ($+2x_6x_7$);
- an input or inputs of a true class: h6_09 ($+3\,\text{AND}(x_9, -x_{10})$) and, at $n = 16384$ only, h5s7 ($+3x_{11}$, a literal of its own class).

The other 10 are controls (all but dnf3x3, f7_02, h4rf0 and h5rf1), mostly decision lists, DNFs and read-once formulas. There the inner term is typically an AND of two literals inside a weighted threshold, as in h6_04's $(x_1 + x_2 - 3\,\text{AND}(x_4, -x_3))$. On 8 of the 26 targets (symxpar, f7_06, f7_10, h6_03, h5s7, h5t0, h5t1, majsym) the weighted block appears at a single analysed n.

Reading [Heuristic]. Any function of a block and one more ± 1 term that the block's partition cannot separate, such as a parity partner, a selector or a gate, can be written wholly or partly as a function of $s + a\tau$. The search finds these because τ is usually a strong single input or pair. The class was built for h6_10 and is used far more widely, mostly without a measurable change in n^*.

Below the discovery size [Numerical]. 216 analysed cells of the 49 found targets lie below n_{struct}.

- *Contents.* The summarised model is the forced variant in 177 cells and the default in 39. The default variant is empty (constant and kernel background) in 26 of its 39 cells; the others hold characters of degree 1–5, true or spurious. The forced variant holds 1–2 forced characters of degree 1–4 (up to 7 from the subsampled high-degree search). They are spurious, outside the relevant inputs, in 152 of the 177 cells, and 87 cells also hold a true character such as a parity partner, a gate literal or a threshold's input. Partial blocks appear in 21 cells, 14 of them one doubling before n_{struct}. In 83 of the 216 cells the summary is an out-of-fold fit, because the stacker gave DFG no weight.
- *Error.* The DFG member's median test error is 0.41 below n_{struct} (the portfolio's 0.37) and 0 from n_{struct} on.
- *Stacking weight.* The two DFG members' median share of the stacking weight is 0.21 below n_{struct} (mean 0.33) and 0.89 from n_{struct} on (mean 0.87). DFG has nonzero weight in 62% of the cells below. By $\log_2(n/n_{\text{struct}})$ the median share is 0.03 at -3, 0.21 at -2, 0.44 at -1, 0.87 at 0 and 0.92 at $+1$. Below n_{struct} the total weight is small (median 0.35, against 1.52 from n_{struct} on; all weights are 0 in 17 cells).

Weight shares multiply log-probabilities and are not contributions to the prediction. What they show is that the stacker's trust in DFG changes abruptly, within one or two doublings of n_{struct}, and that below that point the DFG members get a modest share (median 0.21) of a total weight that is itself small (median 0.35, against 1.52), so their spurious characters have little absolute weight [Heuristic].

The other suites [Numerical]. On the 70 targets of `designed`, `random` and `dlheld`–`dlheld3`, every model reaches test error ≤ 0.005 at its largest analysed n. These runs, too, reproduce the evaluation's seed-0 errors at every cell (above), and their n_{struct} is seed 0. Parities are their own characters, and small juntas their full Fourier expansions. Majorities and symmetric targets get one block (maj9, maj15, maj21 and sym9 at $n_{\text{struct}} = 128$, sym8 at 64). Halfspaces with unequal weights are carried by degree-1 characters (thr10: 209 of them at $n = 4096$). Formulas are carried mostly by characters of degree 1–4: the characters present in the model cover a per-suite median of 0.77–0.88 of the target's non-constant Fourier weight, and at least 90% on 23 of the 67 targets with an exact Fourier table (presence only; coefficients not compared). 25 of the 70 models have a sum block among their three leading terms, 9 of them weighted. Outside the sum-structured suites the model is thus what Part 6's DFG learned, a sparse set of low-degree characters plus the linear block for halfspaces [5], with sum blocks where a target has a symmetric class of five or more inputs [Heuristic].

Caveats.

- Every structure measurement above is seed 0; n^* values marked 3-seed are means over seeds 0–2. n_{struct}, n_{detect} and the model contents can differ on other seeds.
- The symmetry classes are found from 4096 random inputs. Several controls have small classes, which is why controls are excluded from every summary of sum-structured targets.
- The structure runs store only the 10 leading terms per cell, while n_{struct} uses all terms; on 8 v3 cells the matching term is beyond the stored ten.
- The exact-structure match accepts a modulator χ_M, which carries no signs, for a signed class. This affects fd_06's n_{stable}, not its n_{struct}.

6.2 Images: what the boosters split on

On images v3 is v2's members plus `cgbt`, with the stacker refitted over all of them (Sections 2.4 and 5.3). The analysis covers v3 on `qd_dev` at $n = 1000$, 3000 and 10000, on `qd_a`, MNIST and Fashion-MNIST at $n = 10000$, and v2 on `qd_dev` at $n = 10000$. All four sets are development data for v3 (the held-out image sets are `qd_c` and `qd_d`), so the error comparisons below are in-sample; the held-out comparison is in Section 5.3.

- *The v2 fit.* It has test error 0.1467, its evaluation cell 0.1463. The cell was run with multi-threaded BLAS, which can change summation order and so `ggbt`'s early stopping (166 rounds there, 123 here). An earlier analysis fit of v2 reproduces the cell exactly. That BLAS threading is the cause is plausible, not verified (Section 5.2).
- *Which booster is analysed.* The final `ggbt` and `cgbt` of each fit. On `qd_a` and MNIST the stacker gave `ggbt` weight 0, so it was not refitted, and the booster analysed is its out-of-fold fit on 8000 (7998) rows.
- *Geometry.* It is measured on the true 28×28 grid, which the learner never sees; for QuickDraw the columns are mapped back through the stored permutation. Distances are Chebyshev steps, so the 8 neighbours of a pixel are 1 step away.

Pooling sets are local neighbourhoods; on Fashion-MNIST some include the mirror pixel [Numerical]. Table 13 gives the geometry.

		pooling sets: members other than i				Isomap R^2		bars: exact lines	
set	n	$S_i(4)$ ≤ 1 step	$S_i(9)$ ≤ 1 step	$S_i(9)$ ≤ 2 steps	$S_i(9)$ > 2 steps	$k = 4$ union	$k = 9$ mutual	$L = 2$	$L = 4$
`qd_dev`	1000	0.983	0.823	0.985	79	0.84	0.87	0.33	0.16
`qd_dev`	3000	0.991	0.829	0.991	46	0.97	0.89	0.38	0.21
`qd_dev`	10000	0.994	0.834	0.994	31	0.97	0.92	0.42	0.24
`qd_a`	10000	0.999	0.817	0.989	59	0.99	0.83	0.40	0.25
MNIST	10000	0.976	0.787	0.966	186	0.92	0.97	0.20	0.06
Fashion	10000	0.881	0.633	0.840	1006	0.13	0.23	0.11	0.04

Table 13: The pooling sets and bars of v3's image members on the true pixel grid (seed 0) [Numerical]. Pooling sets: the fraction of the $k-1$ members of $S_i(k)$ other than i itself that lie within 1 or 2 Chebyshev steps of i, over all non-constant pixels i; "> 2 steps": the number of such members (of 5408 on QuickDraw at $k = 9$). Isomap R^2: Procrustes R^2 between a 2-D Isomap embedding of the pooling-set graph (an edge from i to each member of $S_i(k)$; "mutual": only edges in both directions) and the true pixel coordinates. Bars: the fraction of bars of half-length L whose pixels lie on one horizontal, vertical or diagonal lattice line. MNIST's $k = 9$ mutual graph covers 674 of its 682 non-constant pixels. Sources: the analysis files in `study/part10/interp/results/image_final/v3/`, via `make_interp.py`.

- *QuickDraw.* 98.3–99.9% of the other members of $S_i(4)$ lie within one step of i, and 98.5–99.4% of those of $S_i(9)$ within two. Only 8 pixels lie within one step, so for $k = 16$ at most $8/15 = 0.53$ of the other members can; 0.49 do. On `qd_dev` the members of $S_i(9)$ farther than two steps fall from 79 to 31 as n grows from 1000 to 10000. The sets overlap the k truly nearest pixels by 0.82, 0.88 and 0.78 for $k = 4$, 9 and 16 at $n = 10000$.
- *MNIST.* The 23 far members of $S_i(4)$ sit at 13 border pixels that are nonzero in at most 0.04% of the 10000 training images. Their correlations rest on a handful of images.
- *Fashion-MNIST.* At 57 of the 784 pixels (7%), $S_i(4)$ contains a far member, and all 65 far members are the centre's mirror pixel: the same row ± 1, with columns summing to 27 ± 2. At $k = 9$, 619 of the 1006 far members are mirror pixels.

Reading [Heuristic]. Garments are nearly left–right symmetric, so a pixel can correlate with its mirror image as strongly as with its neighbours. Sets at those pixels pool both sides of the garment.

The grid is recovered globally where no long edges short-circuit it [Numerical].

- *The $k = 4$ graph.* The 2-D Isomap embedding [26] of the $k = 4$ pooling-set graph matches the true grid with Procrustes R^2 0.97–0.99 on QuickDraw at $n \geq 3000$ (0.84 at $n = 1000$), 0.92 on MNIST and 0.13 on Fashion-MNIST.
- *Variants collapse when a few edges are long.* `qd_a`'s $k = 9$ union graph is about as local as `qd_dev`'s (0.989 and 0.994 of the other members within two steps), but it has 10 edges longer than 5 steps, and its R^2 is 0.24 against 0.86. A verifier removed the edges longer than 5 steps (an oracle ablation that uses the true grid). R^2 then rose to 0.96–0.99 on every QuickDraw and MNIST graph, and to 0.95 on Fashion-MNIST's $k = 9$ union graph.
- *Not always.* MNIST's $k = 4$ graph has 23 edges longer than 5 steps and still scores 0.92.

The learner never builds a global embedding; it uses each set as a local sum. So R^2 summarizes how map-like the recovered neighbourhoods are, not a property the learner uses. Recovering image topology from pixel correlations is an old idea [18].

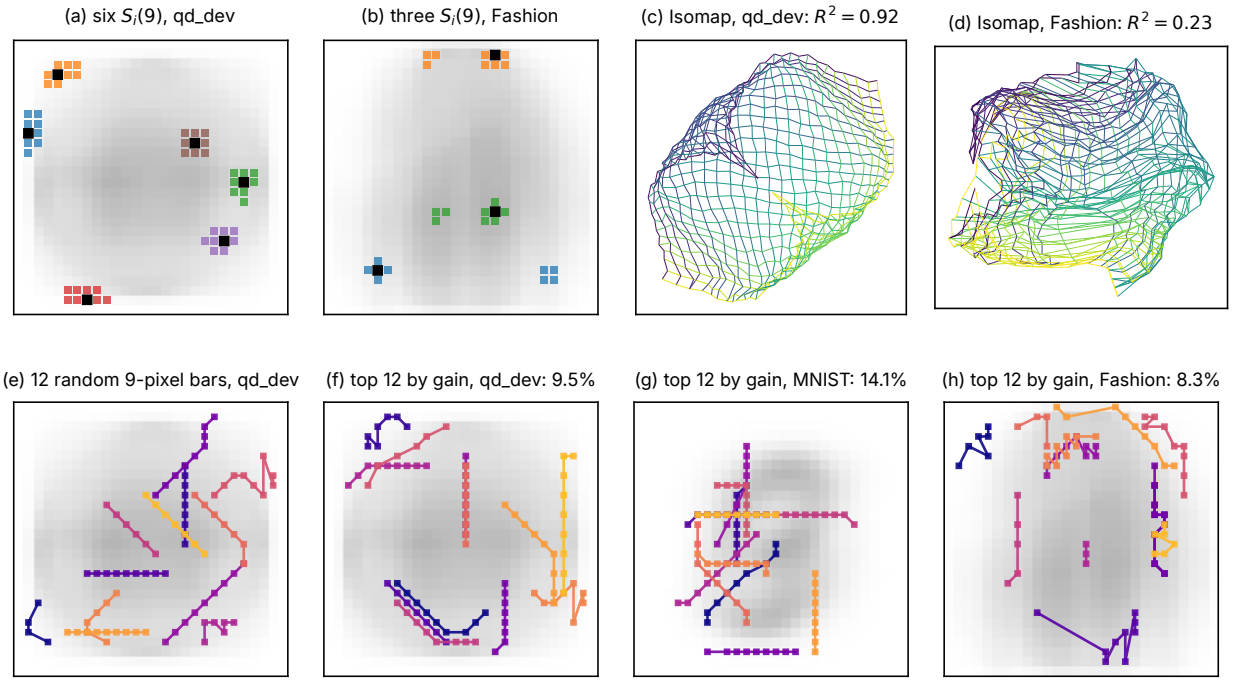


Figure 6: Pooling sets and bars of v3 on the true pixel grid, $n = 10000$, seed 0 [Numerical]. Background: the average of the ten class-mean training images. (a) $S_i(9)$ at six pixels of `qd_dev` (black: i). (b) $S_i(9)$ at three inked Fashion-MNIST pixels whose sets contain mirror pixels. (c, d) The 2-D Isomap embedding of the $k = 9$ mutual pooling-set graph, Procrustes-aligned to the grid and drawn as the true lattice: each pixel at its embedded position, joined to its horizontal and vertical grid neighbours (horizontal edges coloured by row, vertical edges by column). (e) Twelve randomly chosen 9-pixel bars ($L = 4$) of `qd_dev`. (f–h) The twelve 9-pixel bars with the most split gain in `cgbt`, with their combined share of `cgbt`'s total gain. Simplified from `fig_img_topology.pdf` and `fig_img_bars.pdf` in `study/part10/interp/figures/`; figure by `make_interp.py`.

Bars are short, nearly straight runs along recovered lines [Numerical] (Table 13, Figure 6).

- *Shape on `qd_dev`.* At $n = 10000$, 42% of the 5-pixel bars ($L = 2$) are exact lattice lines, and 96% stay within one pixel of their principal axis. For the 9-pixel bars ($L = 4$) the figures are 24% and 63%, since each further step can turn.
- *Growth with n.* The exact-line fraction grows with n (0.33, 0.38, 0.42 for $L = 2$ at $n = 1000$, 3000, 10000; 0.16, 0.21, 0.24 for $L = 4$). The within-one-pixel fraction does not, and it is a weak criterion for 5 pixels.
- *Orientation.* On QuickDraw all four orientations occur about equally often; on Fashion-MNIST horizontal and vertical bars dominate (0.37 and 0.40 at $L = 2$).
- *Other sets.* On MNIST the bars are less straight (0.20 and 0.06 exact lines). On Fashion-MNIST they are rarely lines (0.11 and 0.04), and 27% ($L = 2$) and 30% ($L = 4$) contain a mirror pixel, against 2% and 9% on `qd_dev`. There a bar is often two short pieces on the two sides of the garment (Figure 6h).

Straightness is a property of the construction and the data, not of anything the booster learned.

`cgbt` is the member that carries v3 on QuickDraw and MNIST [Numerical] (Table 14).

		share of the stacking weight						out-of-fold error		test error		
set	n	cgbt	ggbt	gknn	knn	gbt, et	lin	cgbt	ggbt	stack	cgbt alone	ggbt alone
qd_dev	1000	0.32	0.09	0.44	0.16	0.00	0.00	0.272	0.280	0.2118	0.2208	0.2478
qd_dev	3000	0.40	0.08	0.33	0.19	0.00	0.00	0.197	0.212	0.1753	0.1851	0.1968
qd_dev	10000	0.47	0.07	0.17	0.25	0.03	0.00	0.149	0.161	0.1330	0.1368	0.1578
qd_a	10000	0.56	0.00	0.15	0.24	0.06	0.00	0.161	0.172	0.1422	0.1442	0.1654[a]
MNIST	10000	0.40	0.00	0.23	0.24	0.13	0.00	0.031	0.043	0.0255	0.0278	0.0392[a]
Fashion	10000	0.10	0.16	0.00	0.04	0.50	0.20	0.126	0.127	0.1238	0.1216	0.1222
v2, qd_dev	10000	–	0.41	0.24	0.24	0.11	0.00	–	0.161	0.1467	–	–

Table 14: Which members carry v3 on images (seed 0, one fit each) [Numerical]. Left: each member family's share of the total stacking weight. `gknn`: k-NN votes on pooled sums; `knn`: k-NN votes on pixels; `gbt`, `et`, `lin`: Part 8's LightGBM, ExtraTrees and logistic regression on pixels. Middle: out-of-fold error of the two boosters (5-fold at $n \leq 3000$; one holdout of 2000 rows at $n = 10000$). Right: test error of the stacked learner (v3; v2 in the last row) and of each booster alone (its own argmax). [a] `ggbt` had stacking weight 0, so the booster shown is its out-of-fold fit on 8000 (`qd_a`) or 7998 (MNIST) rows; only the out-of-fold comparison is like for like there. Last row: v2's analysis fit. The weights multiply log-probabilities of different calibration and are not shares of the decision. Sources: the analysis files in study/part10/interp/results/image_final/, via `make_interp.py`.

- *The best member.* `cgbt` has the lowest out-of-fold error of all 18 members in every v3 fit. The margin over `ggbt` is within holdout noise on Fashion-MNIST and at `qd_dev`, $n = 1000$. It has the largest single-member weight everywhere except Fashion-MNIST, where Part 8's `gbt` has it.
- *Its weight.* Its share of the stacking weight grows with n on `qd_dev` (0.32, 0.40, 0.47) and is 0.56 on `qd_a`, 0.40 on MNIST and 0.10 on Fashion-MNIST.
- *`ggbt` loses its weight.* At `qd_dev`, $n = 10000$, `ggbt`'s share falls from 0.41 in v2's fit (0.37 in v2's evaluation cell) to 0.07 in v3. On `qd_a` and MNIST it is 0.
- *The rest of the weight.* Most of it goes to the k-NN members, on pooled sums and on pixels. On Fashion-MNIST half goes to Part 8's `gbt` and `et` and a fifth to logistic regression.
- *Alone.* On the test set `cgbt` alone beats `ggbt` alone on QuickDraw and MNIST (on `qd_a` and MNIST against `ggbt`'s out-of-fold fit on fewer rows; the like-for-like out-of-fold errors, 0.161 against 0.172 and 0.031 against 0.043, point the same way): 13.68% against 15.78% at `qd_dev`, $n = 10000$. The two agree on 89–98% of test points. The stack is 0.2–1.0 points better than `cgbt` alone on QuickDraw and MNIST; the 0.2 points on `qd_a` are not resolved by the test set. On Fashion-MNIST the stack (12.38%) is not better than either booster alone (12.16%, 12.22%), a difference the test set does not resolve either.

Three cautions apply to the weights.

- They act on log-probabilities of different calibration. Boosters stopped early on error give over-confident log-probabilities (clipped at 10^{-5}), while k-NN votes, ExtraTrees and logistic regression are smoother. Equal weights are not equal influence.
- The convex stacker splits weight between two nearly collinear boosters on one 2000-row holdout, so "`ggbt` gets weight 0" describes one fit. The 8 held-out fits at $n \geq 10000$ show the same shift (`ggbt`'s weight falls from 0.25–0.50 in v2 to 0–0.12 in v3; Section 5.3).

	cgbt				ggbt	
feature type	qd_dev	qd_a	MNIST	Fashion	qd_dev	Fashion
pixels	0.01 (0.1)	0.01 (0.1)	0.01 (0.1)	0.09 (1.0)	0.04 (0.1)	0.16 (0.7)
pooled sums, $k = 4$	0.02 (0.3)	0.03 (0.3)	0.05 (0.6)	0.10 (1.1)	0.10 (0.4)	0.15 (0.7)
pooled sums, $k = 9$	0.04 (0.5)	0.05 (0.5)	0.07 (0.9)	0.10 (1.2)	0.16 (0.7)	0.16 (0.7)
pooled sums, $k = 16$	0.15 (1.8)	0.12 (1.5)	0.07 (0.8)	0.12 (1.4)	0.32 (1.5)	0.15 (0.6)
bars, $L = 2$	0.24 (0.7)	0.25 (0.8)	0.30 (0.9)	0.25 (0.7)	–	–
bars, $L = 4$	0.53 (1.6)	0.54 (1.6)	0.51 (1.5)	0.35 (1.1)	–	–
prototype pairs	–	–	–	–	0.39 (4.1)	0.38 (4.4)
all bars	0.77 (1.2)	0.79 (1.2)	0.80 (1.2)	0.60 (0.9)	–	–
all pooled sums	0.22 (0.9)	0.20 (0.8)	0.19 (0.8)	0.31 (1.3)	0.57 (0.9)	0.46 (0.7)
columns	8220	8220	8286	9408	3112	3436

Table 15: Split gain by feature type in the final boosters, $n = 10000$, seed 0 [Numerical]. Each entry is the type's share of the booster's total split gain, with its ratio to the type's share of the columns in brackets (1.0: as much gain per column as the booster's average). Bars are two thirds of `cgbt`'s columns; the 300 prototype pairs are about a tenth of `ggbt`'s. LightGBM samples 5% (`cgbt`) or 12% (`ggbt`) of the columns per tree. `ggbt` on `qd_a` and MNIST, an out-of-fold fit, is not shown (pairs 0.30 and 0.41 of its gain). Split gain describes how the trees split, not what the predictions depend on. Sources: the analysis files in `study/part10/interp/results/image_final/v3/`, via `make_interp.py`.

- The out-of-fold errors at $n = 10000$ rest on those 2000 rows. Unpaired, the standard error of an error near 0.15 is about 0.008; for two members on the same rows the paired one is smaller.

Split gain by feature type [Numerical] (Table 15).

- *The bars in `cgbt`.* The bars carry 0.77–0.80 of `cgbt`'s split gain on QuickDraw and MNIST at $n = 10000$ (0.72 at `qd_dev`, $n = 1000$) and 0.60 on Fashion-MNIST. They are two thirds of its columns, so this is 1.1–1.2 times their column share (0.9 on Fashion-MNIST): more than their share, not dominance. They also make 0.75 of the splits at `qd_dev`, $n = 10000$. Per column, the 9-pixel bars (1.5–1.6; 1.1 on Fashion-MNIST) and, on QuickDraw, the 16-pixel pooled sums (1.5–1.8) carry the most. The 5-pixel bars carry 0.7–0.9 of the average, and single pixels 0.1, except on Fashion-MNIST (1.0).
- *The pairs in `ggbt`.* The 300 prototype pairs carry about 0.4 of `ggbt`'s gain (0.30 in `qd_a`'s out-of-fold fit), about four times their column share. On `qd_dev` the 16-pixel pooled sums come next (1.5).
- *Use is wide, gain concentrated.* Almost every bar is split on at least once (at `qd_dev`, $n = 10000$: 2520 and 2596 of 2704 for $L = 2$ and 4). But the top 10% of bars carry 0.65–0.66 of the bar gain there (0.76 at $n = 1000$), 0.78–0.84 on Fashion-MNIST and 0.86 on MNIST. The twelve 9-pixel bars with the most gain carry 9.5%, 14.1% and 8.3% of `cgbt`'s total gain on `qd_dev`, MNIST and Fashion-MNIST (Figure 6f–h).

Which bars are used [Numerical].

- *Straightness.* On QuickDraw the bars weighted by their gain are straighter than the average bar at $L = 4$ (0.32 exact lines against 0.24 at `qd_dev`, $n = 10000$), but not at $L = 2$ (0.43 against 0.42). Straightness correlates with how often the bar's centre is inked (+0.48 for $L = 4$), and the ink-frequency mix of the used bars alone predicts 0.30 of the 0.32. On

MNIST a preference remains after that correction ($L = 2$: 0.36 weighted, 0.20 for all bars, 0.27 predicted from ink frequency).

- *Orientation.* On QuickDraw the gain splits over the four orientations as the bars do. On MNIST horizontal bars carry 0.40 of the bar gain at both lengths, against 0.27 and 0.26 of the bars. But several of MNIST's top bars lie in margins that are rarely inked; the top $L = 2$ bar (4 pixels), centred at row 1, column 18, has pixels inked in 0–7.5% of training images. So they look more like detectors of a digit's extent than of strokes.

These gain-weighted statistics rest on a few bars of one fit.

Class evidence maps: method and limits. For each class, 100 test points of that class are taken. LightGBM's TreeSHAP contributions [21] of each feature to that class's raw score are computed and spread over pixels. A pixel keeps its own contribution. A pooled sum or bar spreads its contribution equally over its pixels. A prototype pair $u = x_a - x_b$ spreads it over pixels j in proportion to $|u_j x_j|$. The maps are averaged over the 100 points. They can show where in the image the features that raise a class's score look. They cannot show more than that, for five reasons.

- *Own score only.* A map is the contribution to the class's own raw score in a 10-class softmax, not to its margin over the other classes.
- *No causal test.* Nothing was occluded or ablated, so no region is shown to be necessary.
- *The spreading rule.* Spreading a subset sum equally over its pixels books part of the evidence of a partly inked bar on blank pixels. On `qd_dev`, of the members of bars with at least one inked member, 43% ($L = 2$) and 52% ($L = 4$) are blank. So evidence on blank pixels marks where a feature looks, not necessarily what drove it.
- *Built-in agreement.* The pairs' maps follow the class's own ink by construction. `cgbt` and `ggbt` share the pixels, the pooled sums and the spreading rule, so part of their agreement is built in.
- *Post-hoc regions.* The regions named below were drawn after looking at the maps. Their shares describe the maps; they do not test them.

What the maps show [Numerical] for the shares (Figure 7).

- *Sign.* The mean maps are almost all positive: positive values make at least 0.96 of each map's absolute mass.
- *Blank pixels.* The share of the net evidence on pixels blank in the point, averaged over classes, is 0.61 for `cgbt` on `qd_dev`, 0.56 on Fashion-MNIST, 0.69 on MNIST and 0.62 on `qd_a`. For `ggbt` it is 0.39–0.44, and 0.53–0.68 once its pairs are removed. Blank pixels are 0.75, 0.73, 0.83 and 0.55 of these images (`qd_dev`, `qd_a`, MNIST, Fashion-MNIST), so per pixel, inked pixels carry more of the evidence on QuickDraw and MNIST, and Fashion-MNIST is at par. It is not so for every class: grapes, soccer ball and campfire (0.32–0.39 on `qd_dev`), pullover, shirt and coat (0.25–0.29), camouflage on `qd_a` (0.15) and the digit 8 (0.42) draw most of their evidence from ink.
- *Agreement.* The `cgbt` and `ggbt` maps of a class correlate at 0.63–0.94 on `qd_dev`, Fashion-MNIST and MNIST, and at 0.40–0.88 on `qd_a` (slightly less over non-constant pixels only: 0.60–0.94 and 0.33–0.87).

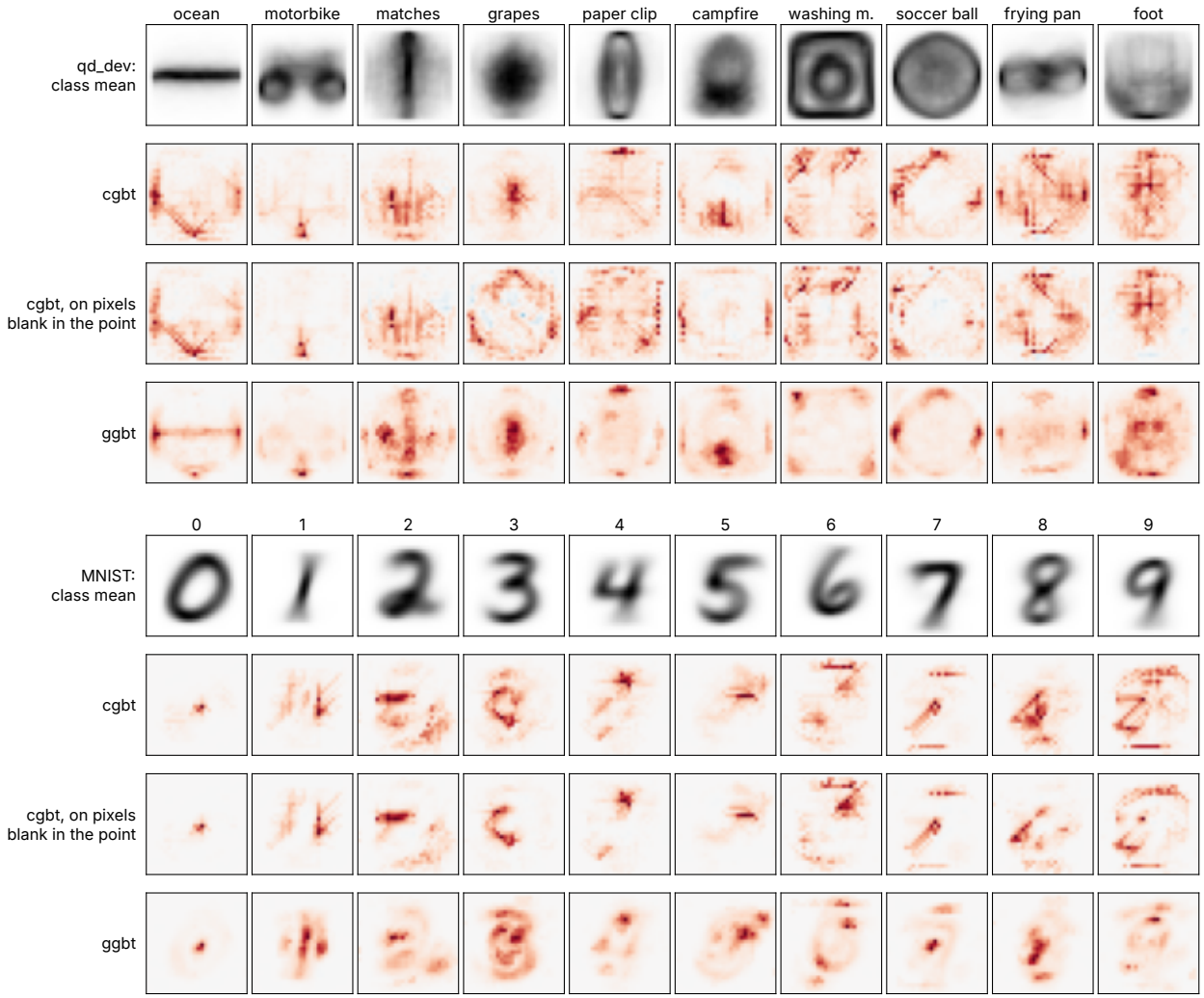


Figure 7: Class evidence maps of v3's two boosters on `qd_dev` and MNIST, $n = 10000$, seed 0 [Numerical]. Each map is the mean, over 100 test points of the class, of the TreeSHAP contributions to that class's own raw score, spread over pixels as described in the text; red is positive, and each map is scaled to its own maximum. Rows: the class's mean training image; `cgbt`; the part of `cgbt`'s map on pixels blank in the point ($x \leq 0.1$); `ggbt`, prototype pairs included. The maps are descriptive (see the text for what they cannot show). Simplified from `fig_img_contrib_qd_dev.pdf` and `fig_img_contrib_mnist.pdf` in `study/part10/interp/figures/`, which also show the pooled-sum and pair parts, and Fashion-MNIST and `qd_a`; figure by `make_interp.py`.

- *By feature type.* By absolute contribution, `cgbt`'s evidence on `qd_dev` is 0.78 bars, 0.20 pooled sums and 0.02 pixels. `ggbt`'s is 0.64 pooled sums, 0.29 pairs and 0.07 pixels.

Examples follow. The shares are `cgbt` / `ggbt`, as parts of the class's net evidence; the descriptions are [Heuristic].

- *MNIST.* 0: the hole carries 0.46 / 0.41 of the evidence, less than half, on 3% of the image and 99% on blank pixels. 1: strips beside the stroke (0.46 / 0.32), which `cgbt` draws as bars parallel to it. 7: a '/' band of blank pixels left of the stem (0.30 / 0.23). 3: the open left side (0.32 / 0.23). 4: the open top (0.28 / 0.25). 8: the central crossing, on ink (0.26 / 0.30).
- *`qd_dev`.* Motorbike: the blank gap between the wheels at the bottom centre (0.20 / 0.16 on 2.6% of the image, where the class's mean ink is 0.04 and the other classes' 0.28). Ocean: for `ggbt` the horizon line (0.40 on 11% of the image). For `cgbt`, the horizon's ends (0.17) and a diagonal band of mostly blank pixels from the left edge to the bottom centre (0.26 on 8% of the image, 84% on blank pixels), where ocean has little ink (0.13, against 0.26 for the other nine classes) and soccer ball, campfire, grapes and washing machine have the most (0.37, 0.33, 0.30, 0.29). Campfire: the base of the fire, on ink (0.36 / 0.32). Soccer ball: `cgbt` follows the upper-left arc as a chain of diagonal bars (0.18 against 0.12). Foot: the blank middle (0.22 / 0.17).
- *Fashion-MNIST* (not in the figure). Trouser: the gap between the legs (0.26 / 0.31). Boot: a blank strip left of the shaft (0.41 / 0.25; the class's mean ink there is 0.001, the others' 0.34). Sandal and sneaker: a blank upper-left region (0.39 / 0.30 and 0.36 / 0.26).

Reading [Heuristic]. In these maps a sizable part of a class's evidence sits where the class has no ink and other classes do; for several digits the largest named region is an open one (0, 1, 3, 4, 7). `cgbt` does not look at different places from `ggbt`. It looks at the same places through thinner, oriented windows: lines and chains of short segments, where `ggbt`'s pooled sums give blobs. That difference in shape follows from the feature shapes and is not learned. What the fit chose is which segments carry weight.

`ggbt`'s prototype pairs [Numerical]. The 300 pairs carry 0.38–0.41 of `ggbt`'s gain (0.30 on `qd_a`), and almost all are split on, but the top 10 carry 0.29 (`qd_dev`), 0.50 (Fashion-MNIST) and 0.36 (MNIST) of the pair gain. In several of the class pairs with the most gain one drawn pair carries most of it (grapes–soccer ball 0.85, motorbike–frying pan 0.85, shirt–sneaker 0.89, MNIST 0–1 0.90), but not in all (trouser–boot 0.59, MNIST 1–2 0.69). Per drawn pair, relative to the average pair, on Fashion-MNIST the most gain goes to contrasts of very different silhouettes (trouser–boot 5.8×, shirt–sneaker 4.9×), and similar classes get almost none (T-shirt–dress 0.01, sandal–boot 0.02, T-shirt–shirt 0.04). Elsewhere there is no such pattern: `qd_dev`'s top class pair is grapes–soccer ball (5.3×), two round shapes and its third most confused pair. MNIST's pairs (1–2 4.0×, 0–1 3.5×) are from `ggbt`'s out-of-fold fit, which has stacker weight 0. Over class pairs, gain per pair correlates with `ggbt`'s own confusion between the two classes at -0.59 on Fashion-MNIST and at -0.34 to $+0.07$ elsewhere. Which class pairs win depends on which drawings happened to be drawn: on `qd_dev` the top one changes between $n = 3000$ and 10000. *Reading* [Heuristic]: a projection on $x_{\text{trouser}} - x_{\text{boot}}$ is a coarse template that separates two groups of silhouettes, which the trees refine with local features.

v3 against v2 at `qd_dev`, $n = 10000$ [Numerical].

- *The same `ggbt`.* The two fits have identical pooling sets and a bit-identical `ggbt`: the same gain vector, test predictions and maps, and 123 rounds in both. So, as far as checked, the fits

differ by `cgbt`, the stacking weights, and which members are refitted.

- *Error.* 0.1467 for v2 (its evaluation cell: 0.1463) and 0.1330 for v3. 2.08% of the test points are right in v3 and wrong in v2, and 0.71% the reverse.
- *Per class, v2 → v3.* Campfire 0.200 → 0.158, grapes 0.280 → 0.243, matches 0.269 → 0.247, motorbike 0.110 → 0.095, soccer ball 0.082 → 0.075, washing machine 0.060 → 0.055, paper clip 0.138 → 0.134, ocean 0.087 → 0.084, frying pan 0.121 → 0.119, foot 0.120 → 0.120.
- *Where `cgbt` beats `ggbt`.* The largest gains are mostly on the classes where `cgbt` alone beats `ggbt` alone most: campfire (0.155 against 0.201), matches (0.254 against 0.288), grapes (0.229 against 0.262). But the match is not one for one. Frying pan has the fourth-largest `cgbt` advantage (0.026) and almost no gain (0.002), and the rank correlation over the ten classes is about 0.79. Foot is the only class where `cgbt` alone is worse (0.123 against 0.121). No measurement ties the per-class gains to particular bars.
- *Other sets.* On Fashion-MNIST, v2's cell has 0.1259 and v3 0.1238, but the two runs' `ggbt` differ (162 against 242 rounds; out-of-fold 0.1321 against 0.1271), and `cgbt`'s weight share there is 0.10. So the difference cannot be attributed to `cgbt`. On `qd_a` v2's and v3's cells give 0.1509 and 0.1422, and `ggbt` stops at 206 rounds with the same out-of-fold error in both. On MNIST v2's image path gives 0.0299 (Part 9's evaluation) against v3's 0.0255 here.
- *Cost.* `cgbt` takes 302 of the 530 CPU seconds of v3's evaluation fit at `qd_dev`, $n = 10000$.

Reading [Heuristic]. On QuickDraw and MNIST, `cgbt` is a better booster than `ggbt` on a bank that contains `ggbt`'s pooled sums plus the bars, and the stacker moves its weight there. On Fashion-MNIST, where the bars are rarely lines and the evidence is mostly the silhouette's outline, the two boosters are about equal, and the stack gains nothing measurable from `cgbt`. That matches the held-out picture (Section 5.3): the gain over v2 is a better fixed representation for the trees, not a different kind of learning.

Summary of what the analysis shows.

- *Toy targets* [Numerical]. On 49 of the 54 sum-structured targets, v3's DFG model comes to hold the target's exact blocks with their signs, and its output follows the target's function of the block level by level. On the 25 pure symmetric targets, DFG's test would admit the true block at $n = 128$ every time. The data needed beyond that to find the block grows as the target's degree-1 and degree-2 signal shrinks ($\rho = -0.86$), and the 10% error point arrives with the block. The residue classes mod 3 fail in the search, not in the test: at $n = 16384$ the true block would pass with 134–155 times the threshold, but nothing at degrees 1–2 points to it.
- *Images* [Numerical]. The pooling sets and bars recover local pieces of the pixel grid from correlations alone: blobs and short, nearly straight runs on QuickDraw and MNIST, and on Fashion-MNIST local sets of which 7% (at $k = 4$) include the centre's mirror pixel. `cgbt`, which splits mostly on bars and 16-pixel pooled sums, has the lowest out-of-fold error of all members in every analysed fit and takes over `ggbt`'s weight on QuickDraw and MNIST. A large share of the evidence in its maps lies on blank pixels where other classes have ink, though less than their share of the image on QuickDraw and MNIST.
- *What it does not show.* Nothing here is an ablation of the learner or a causal test, and every structural measurement is from one seed (seed 0). The mechanism labels for v1.3's changes are read off the fitted models. Stacking weights, split gain and contribution maps describe the fits, not what their predictions depend on.

7 Discussion

7.1 What kind of learner this turned out to be

v3 is not one mechanism. It is three learners under one convex stacker, and the input type decides which of them carries the fit (Section 2).

- *Boolean inputs:* enumerate and test. DFG proposes characters and signed-sum blocks from a structured dictionary, admits each one only when its correlation with the residual passes a union-bound test over its whole class, and reads the admitted features out linearly.
- *Images:* a fixed, data-derived feature hierarchy read out by gradient-boosted trees. Pixels, sums over the 4, 9 and 16 pixels most correlated with each pixel, sums over bars of up to 5 and 9 pixels along lines recovered from the correlations, and (in `ggbt`) projections on 300 differences of two training images are computed once. Trees split on them, and nothing in them is fitted.
- *Other tabular data:* a classical stack of LightGBM, ExtraTrees, nearest-neighbour votes and logistic regression, unchanged since Part 8.

The Parts 6–9 paper already described the portfolio as an AutoML-style dispatch by input type [5], and Part 10 does not change that description. Its two additions sit inside the first two regimes: a wider sum-block search for DFG, and a second booster over a richer bank.

7.2 Why it works where it works

Boolean targets: an Occam-like sample cost, when the search can propose [Heuristic]. A union-bound test over a class of N candidates admits a feature with residual correlation ρ (in units of the residual's scale) once $n \gtrsim 2\ln(4KN/\delta)/\rho^2$ (Section 2.3). For a k-sparse character, $\ln N \approx k \ln d$; for a signed block on k of d inputs, $\ln N \approx \ln(\binom{d}{k}2^{k-1})$. Either way the sample cost is logarithmic in the size of the dictionary: the Occam rate, paid for with FLOPs of enumeration and search that fit inside one network run at $d = 30$. This is the classical low-degree algorithm [20], made greedy, budgeted and test-gated, with a second dictionary for symmetric functions of signed sums. Gradient-trained networks pay in samples instead. For parities the companion paper measured rates close to the leap and correlational-SQ rate d^{k-1} [1–3, 6, 16]. For an even function of one signed sum, a network has to find the block's sign vector in the degree-2 statistic $\mathbb{E}[f\,xx^\top]$ against sampling noise. That is the information-exponent picture of single-index learning [4, 8]. Networks can trade data for width and compute along a frontier [9]; DFG sits at an extreme of that trade.

The interpretability analysis sharpens the picture in one respect (Section 6.1). On the 25 pure symmetric targets, the test is not the bottleneck. The true block would pass at $n = 128$ every time, and on each of the 22 found targets the seed-0 10% error point lies within the doubling below the first grid size at which the model holds the exact block. This is consistent with the data cost being set by when the search first *proposes* the block. Every seed of the block search reads the block's inputs off degree-1 and degree-2 statistics of the residual, so the delay between detectability and discovery tracks the target's weight at those degrees ($\rho = -0.86$). So DFG relies on the same low-order statistics that a network's gradient sees. It needs far fewer samples because it rounds a coarse spectral estimate to an exact signed subset and then tests that subset with a union bound, where a network has to learn the direction to useful precision by noisy local steps [Heuristic].

Where neither has a foothold. The residue classes mod 3 of 10–14-input counts have almost no Fourier weight at degrees 1 and 2 (none for h6_00 and f7_05; coefficients of about 0.002 for fd_10, undetectable at the study's n). DFG's search never proposes their block, although at $n = 16384$ the block would pass the test at 134–155 times the threshold (Sections 4.4 and 6.1). The networks mostly fail too: none learns f7_05, and only the most regularized variant learns h6_00, at $n^* = 12304$ (fd_10, in `dlfamdev`, has no network runs). Their signal sits at high degree on an unknown subset of inputs, which learners that rely on low-order correlations should find hard [16]. Exhaustive search does not help within the budget: there are 1.5×10^{10} to 1.2×10^{12} blocks of the true size, each costing at least n operations to score, against 1.6×10^{11} FLOPs per toy fit. So this is a shared limit, not a loss specific to discrete search [Heuristic]. The only target on which a network is clearly better than v3 is of this kind.

Images: locality recovered from data, read out by trees [Heuristic]. On QuickDraw the most correlated pixels of a pixel are its neighbours. The pooling sets are therefore small blobs of the true grid, and the bars are short runs along it. A 2-D embedding of the pooling-set graph matches the grid with R^2 0.97–0.99 at $n \geq 3000$ (0.84 at $n = 1000$; seed 0; Section 6.2), and the 5-pixel bars lie within a pixel of a straight run in about 96% of cases (the 9-pixel bars in 63%). Recovering image topology from such correlations is an old idea [18], as is grouping inputs into receptive fields by correlation [7]. Boosted trees [15] over these fixed local sums then do what a first convolutional layer with a nonlinearity would do, but at fixed positions and without shared weights. The MLP references are denied this structure by design: they are invariant to the pixel permutation, and nothing in their training recovers it. A control and a context run bound how much of the image result is representation.

- An MLP given v2's pooled sums (`mlp_pool`) tied v2 at both of its held-out cells, as in Part 9. v3 is clearly better than it at both (16.5% and 11.9% lower error; `mlp_pool` seed 0), while v3 and v2 tie there. This is consistent with `cgbt`'s bank adding something an MLP on v2's features does not have, but it is not a test of the bars: `cgbt` differs from `ggbt` in three ways, and no MLP was given the bars.
- A small CNN that sees the layout (seed 0 only) has lower error than v3 at all three cells where both have run: clearly at `qd_a`, $n = 10000$ (12.00% against 14.22%), and within the tie band at `qd_c`, $n = 1000$ and 10000 (24.36% against 26.58%; 14.84% against 16.17%).

So on images v3 shows that a learner which never reads the layout, and whose features are fixed subset sums chosen by rank statistics, can beat MLP ensembles that spend 2.0 and 5.0 times its FLOP budget (`mlp_strong` at all six held-out cells, `mlp_wide` at five of six), and ties the 1024-wide ensemble, which spends 9.2 budgets and a comparable time per fit (0.9–1.6× v3's). It does not show that discrete search beats gradient-based learning on images.

Tabular data [Heuristic]. Here the result is the known strength of tree ensembles and stacking on medium-sized tabular data [10, 12, 25, 27]. Careful regularization and tuning of MLPs narrow that gap [11, 14], and the MLP references here are standard recipes. DFG runs on 2 of the 26 datasets, and there v3's errors equal v1's (mushroom) or differ from them at no cell clearly (kr_vs_kp; Section 4.5).

7.3 Limits

- **Targets without a low-degree foothold.** Residue classes mod q of long counts, and more generally blocks whose weight at degrees 1–2 is small, are found late or never (Section 4.4).

Weaker cases cost data. h5s7, an arbitrary accept set of a 13-input count with 0.017 of its variance at degrees 1–2, is detectable at $n = 128$ and found at 8192. h5s1 reaches 10% error in no learner's 3-seed mean (v3's seed 0 at $n \approx 14000$); in v3's seed-0 fits the bottleneck is detecting its modulating parity first. The Part 10 toy builder measured the search landscape of fd_10 and concluded that a random-restart search would need far more than 10^5 restarts `study/part10/p10_sum/DESIGN.md`.

- **The layout is recovered, so locality is used in spirit.** The image members are exactly permutation-equivariant in their objects and never read a pixel index. But the correlations give back local neighbourhoods and lines, which is the information the permuted-pixel protocol was meant to withhold from every learner. On Fashion-MNIST some sets pool mirror-image pixels instead, and the bars do not help there.
- **Equal FLOPs are not equal CPU time.** On the held-out image cells v3 uses 0.30–0.58 of its FLOP budget, and the MLP ensembles spend 2.0–9.2 budgets per fit. But a v3 fit takes 1361 and 1464 CPU seconds at $n = 60000$, comparable to the CPU-matched `mlp_xl` (2196 and 1335 s of wall time, 0.9–1.6× v3's), and that is the reference v3 only ties. At $n = 3000$ on the development sets v3 uses 0.85 of the budget by its own count, and 0.88–0.98 with Part 9's audited correction of the tree member's charge; no compute audit was made of v3 (Section 5.4).
- **Prediction is expensive and uncharged.** A v2 query costs about 2.8×10^8 FLOPs at $n = 60000$, about 500× an MLP's, because the nearest-neighbour members scan the training set. v3 adds a second bank projection (about 1.2×10^7 FLOPs) and about 4000 more tree traversals per query.
- **One seed at $n = 60000$.** At $n \geq 30000$ every image learner except the baseline `mlp` ran seed 0 only, so the two cells that decide the comparison with the CPU-matched ensemble rest on one fit each, and on the test set's binomial noise for the standard error. v3's gain over v2 is below the 10% threshold at every held-out cell; the best-learner rule used mean $\log_2$ ratios, and v3 is lower in all 14 seed-paired fits, which is a description, not a pre-registered test.
- **Small held-out sets.** `dlheld7` has 12 targets from one family, and on 5 of its 10 wins default gradient boosting or logistic regression already beats the networks. `qd_c` and `qd_d` are two 10-class sets from one data source. The 10 fresh tabular datasets of Part 8 give $p = 0.11$. A second family or source could change the picture.
- **The earlier suites are in-sample.** Each step of the study used the earlier held-out sets as development data (Section 3.4). Pooled numbers over all suites summarize the record; they are not held-out estimates. On the toy side the margin against the one network variant run on every suite is smallest on the two draws of the blind family (Section 4.2), which is what some selection on the other suites would produce, though the suites also differ in kind and the differences are within noise.
- **Concrete references.** One toy network architecture at one width and one learning rate, and image MLPs without augmentation or architecture search. The hindsight minima favour the references, but stronger networks would plausibly narrow the margins.
- **No independent audit in Part 10.** Parts 8 and 9 had independent audits of leakage, compute, discreteness and reference strength [5]. Part 10 inherits their findings and fixes, but its own comparisons were not re-audited in that sense. The interpretability analyses were

independently verified, and the paper's claims were checked against the result files and records by independent checkers (Appendix A).

- **The interpretability is descriptive.** Every structural measurement in Section 6 is seed 0, and nothing in it is an ablation of the learner or a causal test. The attributions of v1.3's gains to weighted blocks or to the small-n search are read off fitted models. Stacking weights, split gain and contribution maps describe the fits, not what their predictions depend on.

7.4 What would test it further

1. *The information-matched image comparison.* Give an MLP v3's whole bank, pooled sums and bars, as `mlp_pool` did for v2's. Train a gradient-based network that learns sparse local connectivity [22], or a locally connected network on the recovered neighbourhoods. Either would say how much of v3's image margin is representation that a network can use as well.
2. *A second blind toy family*, written independently of the first, and the toy suites at larger d. At $d = 30$ the advantage on the blind family is a factor of about $2^{1.5} \approx 2.8$. For parities DFG's advantage grew with d in Part 6; whether it does for signed sums was not measured.
3. *Block search beyond degree 2.* A seed that reads higher-order statistics of the residual could reach the mod-q targets. The Part 10 builder found that the true block of fd_10 is detectable only within about 3–4 errors of the truth, so that random restarts would need far more than 10^5 tries; a cheaper route is not known.
4. *Power and attribution.* More seeds at $n = 60000$; a pre-registered paired comparison with v2; image sets from another source; ablations of v1.3's three additions on held-out targets; and a compute audit of v3 that also charges prediction.

References

[1] Emmanuel Abbe, Enric Boix-Adserà, and Theodor Misiakiewicz. The merged-staircase property: a necessary and nearly sufficient condition for SGD learning of sparse functions on two-layer neural networks. In *Conference on Learning Theory (COLT)*, 2022. arXiv:2202.08658.

[2] Emmanuel Abbe, Enric Boix-Adserà, and Theodor Misiakiewicz. SGD learning on neural networks: leap complexity and saddle-to-saddle dynamics. In *Conference on Learning Theory (COLT)*, 2023. arXiv:2302.11055.

[3] Boaz Barak, Benjamin L. Edelman, Surbhi Goel, Sham Kakade, Eran Malach, and Cyril Zhang. Hidden progress in deep learning: SGD learns parities near the computational limit. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[4] Gérard Ben Arous, Reza Gheissari, and Aukosh Jagannath. Online stochastic gradient descent on non-convex losses from high-dimensional inference. *Journal of Machine Learning Research*, 22(106):1–51, 2021.

[5] Claude (Anthropic). Discrete learners against gradient-descent-trained networks: Boolean targets and standard benchmarks. Prompted by Kaarel Hänni. `https://github.com/kaarelh/AI-works/tree/claude/nifty-mccarthy-j0lw2a/gd-circuit-growth/paper_discrete`, 2026. Parts 6–9 of the study this paper continues.

[6] Claude (Anthropic). Gradient descent on Boolean targets as noise-limited greedy feature growth: experiments, discrete models, and generalization bounds. Prompted by Kaarel Hänni. https://github.com/kaarelh/AI-works/tree/claude/nifty-mccarthy-j0lw2a/gd-circuit-growth/paper, 2026. Parts 1–5 of the study this paper continues.

[7] Adam Coates and Andrew Y. Ng. Selecting receptive fields in deep networks. In *Advances in Neural Information Processing Systems 24*, 2011.

[8] Alex Damian, Loucas Pillaud-Vivien, Jason D. Lee, and Joan Bruna. Computational-statistical gaps in Gaussian single-index models. In *Proceedings of the Thirty-Seventh Conference on Learning Theory (COLT)*, 2024.

[9] Benjamin L. Edelman, Surbhi Goel, Sham Kakade, Eran Malach, and Cyril Zhang. Pareto frontiers in neural feature learning: data, compute, width, and luck. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2309.03800.

[10] Nick Erickson, Jonas Mueller, Alexander Shirkov, Hang Zhang, Pedro Larroy, Mu Li, and Alexander Smola. AutoGluon-Tabular: robust and accurate AutoML for structured data, 2020. arXiv:2003.06505.

[11] Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. Revisiting deep learning models for tabular data. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[12] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022.

[13] David Ha and Douglas Eck. A neural representation of sketch drawings, 2017. arXiv:1704.03477; ICLR 2018.

[14] Arlind Kadra, Marius Lindauer, Frank Hutter, and Josif Grabocka. Well-tuned simple nets excel on tabular datasets. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[15] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: a highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[16] Michael Kearns. Efficient noise-tolerant learning from statistical queries. *Journal of the ACM*, 45(6):983–1006, 1998.

[17] Béatrice Laurent and Pascal Massart. Adaptive estimation of a quadratic functional by model selection. *Annals of Statistics*, 28(5):1302–1338, 2000.

[18] Nicolas Le Roux, Yoshua Bengio, Pascal Lamblin, Marc Joliveau, and Balázs Kégl. Learning the 2-D topology of images. In *Advances in Neural Information Processing Systems 20*, 2007.

[19] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[20] Nathan Linial, Yishay Mansour, and Noam Nisan. Constant depth circuits, Fourier transform, and learnability. *Journal of the ACM*, 40(3):607–620, 1993.

[21] Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bala Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1):56–67, 2020.

[22] Behnam Neyshabur. Towards learning convolutions from scratch. In *Advances in Neural Information Processing Systems 33*, 2020.

[23] Randal S. Olson, William La Cava, Patryk Orzechowski, Ryan J. Urbanowicz, and Jason H. Moore. PMLB: a large benchmark suite for machine learning evaluation and comparison. *BioData Mining*, 10:36, 2017.

[24] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[25] Ravid Shwartz-Ziv and Amitai Armon. Tabular data: deep learning is not all you need. *Information Fusion*, 81:84–90, 2022.

[26] Joshua B. Tenenbaum, Vin de Silva, and John C. Langford. A global geometric framework for nonlinear dimensionality reduction. *Science*, 290(5500):2319–2323, 2000.

[27] David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992.

[28] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms, 2017. arXiv:1708.07747.

[29] Greg Yang and Edward J. Hu. Tensor programs IV: feature learning in infinite-width neural networks. In *International Conference on Machine Learning (ICML)*, 2021.

A How Part 10 was run

Part 10 and this paper were produced by an AI system, Claude (Anthropic), on 27–28 October 2026. Kaarel Hänni posed the request (Section 1). No human has reviewed the code, the results or this text. Times are UTC. The records are:

- the protocol `study/part10/PROTOCOL.md`, written before any design work and amended only by dated decision entries and notes;
- the builders' notes `study/part10/p10_sum/DESIGN.md`, `study/part10/p10_hier/DESIGN.md` and `study/part10/p10_codes/DESIGN.md`;
- the assembly record `study/part10/hyb_portfolio3/ASSEMBLY.md`;
- the verification record of the interpretability analyses, `study/part10/interp/VERIFICATION.json`.

Organisation. One orchestrating session wrote the protocol, fixed the held-out sets, ran every held-out evaluation and reference run, applied the rules and assembled v3. It delegated five kinds of work to separate agents with their own instructions: the three builders; two interpretability analysts; four verifiers of the analyses; the drafting of sections of this paper; and three checkers of its claims (Appendix A.5). The builders worked on development data only. The orchestrator read no held-out result of a Part 10 learner before that learner was frozen (Appendix A.3 lists what it did read).

A.1 Protocol and builders

The protocol was committed at 18:50 on 27 October (`dc70266c`), before any design work. It fixed the held-out sets `dlheld7`, `qd_c` and `qd_d` and the development draw `dlfamdev`; the budgets; the three builders' tasks and their fixed development evaluations; the integration rule; the held-out evaluation with its references; the best-learner rule; and the plan for the interpretability analysis and this paper (Section 3.4).

builder	component	development	frozen (md5)	fixed development evaluation
T, `p10_sum`	DFG v1.3: weighted sum blocks, forced blocks at $n \geq 48$, beam at $n \leq 128$	18:52–19:44 (52 min)	19:44 (`5ee995ad`; `dfg13.py` `69cabe5c`)	20:00–21:35 on `designed`, `dlsymdev`, `dlfamdev`, 3 seeds; kr_vs_kp at $n = 1000$
I1, `p10_hier`	a max-pooling second level in `ggbt`'s bank; pairs dropped	18:52–19:45 (53 min)	19:45 (`1da21bf7`)	19:45–21:35 on the 8 image cells and `qd_a` at $n = 60000$, seed 0
I2, `p10_codes`	a second booster `cgbt` on pooled sums and bars	21:37–22:10 (33 min)	22:10 (`fd688acd`)	22:10–23:43, the same 9 cells; interrupted at 22:40 and resumed (below)

Table 16: The three Part 10 builders, 27 October. Times from each builder's `DESIGN.md`; the evaluation's end for `p10_codes` is the commit that recorded it (`16ef5fde`). Every evaluation cell records the md5 of the file it ran.

The builders are summarized in Table 16. Each changed one component of v2, delivered a full learner file, a `DESIGN.md` and an md5 at its freeze, and then ran the fixed development evaluation once. The image builders used validation rows of the development sets only. The toy builder scored its DFG-alone checks on fresh validation draws, and its portfolio checks on the toy harness's test points of the development suites (`designed`, `dlsymdev`, `dlfamdev`, `dlheld6`). No builder used a held-out set. Three events during the evaluations are recorded in the notes:

- *`p10_sum`.* Its evaluation was started at 19:44 as 8 shards in one shell session. The machine's autogroup scheduler gave the shards together about 0.55 cores, so at 20:00 they were stopped, before any result was written, and restarted unchanged in four sessions. Only the restarted run produced results.
- *`p10_hier`.* Its `qd_dev` process was killed by the shared memory limit at about 20:29 and restarted; the finished $n = 1000$ cell was kept and $n = 10000$ refitted with the same random streams. Its $n = 60000$ cell waited 1 h 28 min for the shared large-fit lock and ran 21:13–21:35.
- *`p10_codes`.* It started after the other two builders had finished. The container restarted at 22:40, with 2 of its 9 cells done, and the workflow that ran the builders was lost. The orchestrator resumed the evaluation with the builder's own script. The cells are resumable and

record the learner's md5, which was unchanged. The orchestrator took the builder's summary from its `DESIGN.md`, whose evaluation and cost placeholders (§3, §5) the builder never filled.

A.2 The integration and best-learner decisions

The rules (Section 3.4) were applied by `study/part10/compare_part10.py` from the result files, and each decision was entered in the protocol when it was made. The numbers are in `study/part10/RESULTS.md`.

- *Toy component: adopted* (21:35; commit `33df158e`, 21:50). Against v2 on `designed`, `dlsymdev` and `dlfamdev` (41 targets, inputs permuted, 3 seeds), `p10_sum`'s mean $\log_2 n^*$ ratio is -0.066 over the 40 targets both learn: -0.028 on `designed`, -0.178 on `dlsymdev` and -0.055 on `dlfamdev`. The largest increase is $+0.26$ (fd_06). The rule asked for a mean below -0.05 and no increase above 0.5. v2's development fits are `p9_sum`'s on `designed` and `dlsymdev`, the same code on toy inputs (110 of 110 seed-0 cells on `designed` checked equal), and v2's own on `dlfamdev`.
- *Image component: `p10_codes` adopted* (23:40). Both candidates qualified, and the rule takes the one with the lower mean $\log_2$ error ratio against v2 over the 8 development cells. `p10_codes` had -0.078; at `qd_a`, $n = 60000$, 10.82% error (limit 12.57%), 1453 CPU s (limit 2636 s) and 0.58 of the FLOP budget. `p10_hier` had -0.071, 11.08%, 924 s and 0.44. The builders' own numbers used Part 9's `p9_gates` development errors for v2 on `qd_dev` and Fashion-MNIST (so `p10_hier`'s note reports -0.068); the rule used v2's own runs on those cells, made for Part 10.
- *Assembly and freeze* (23:45; commit `20003f85`, 23:50). The script `study/part10/assemble_v3.py` built v3 from the learner file of `p10_codes`, with DFG v1.3. `learner.py` has md5 `b045fafe1e24418620bf8c503aa601c6` and `dfg13.py` `69cabe5c50345cb24079198ed52b562f`. All 15 identity checks on development data gave identical predictions: v3 against `p10_sum` on 12 toy fits and on kr_vs_kp, against `p10_codes` on a `qd_dev` fit, and against v2 on optdigits. The files were made read-only.
- *Scheduling.* Neither image candidate changes any non-image code path, so v3's behaviour on toy inputs was fixed by the toy decision alone. v3's held-out toy evaluation therefore started after the toy decision, as `p10_sum` on `dlheld7`. Its results were not read before v3 was frozen, and v3 $\equiv$ `p10_sum` on toy inputs was checked at assembly.
- *Best-learner rule: v3* (commit `adcbfd0c`, 04:20 on 28 October; the protocol entry says 04:25). It was applied after every held-out cell of v3 and v2 was in. The mean $\log_2$ error ratio of v3 to v2 over the four held-out image cells with $n \geq 10000$ is -0.125 (rule: below 0), and the mean $\log_2 n^*$ ratio on `dlheld7` is -0.004 (rule: at most $+0.1$).

A.3 Disclosures and notes

What was seen of held-out data. The protocol and the builders' notes record three items.

- *The `p10_hier` builder*, about 15 minutes after its freeze, printed the last 5 lines of the orchestrator's log of the baseline `mlp` on `qd_c` while finding out why its $n = 60000$ cell was waiting for the shared lock. Those lines are the baseline MLP's `qd_c` test errors at $n = 100$–3000. Nothing about a discrete learner on `qd_c` was shown, and the learner file was already frozen and read-only. `p10_hier` was not adopted.

- *The `p10_sum` builder* recorded incidental exposures: a process listing that showed the command lines of reference jobs on `qd_c`, and a one-line function signature `make_dl_heldout7_suite` printed at the end of a source range, with no body. Nothing about `dlheld7`'s targets was seen.
- *The orchestrator*, before v3's freeze, read reference results on held-out data: the `nets_wd` training logs of f7_00–f7_04 on `dlheld7` and the MLP references' `qd_c` progress lines. It read no result of v2 or of a Part 10 learner on `dlheld7`, `qd_c` or `qd_d`. v2's `dlheld7` results and v3's toy results there were committed unread.

BLAS threading. The image analyst found at about 02:00 on 28 October that v2's own development cells on `qd_dev` and Fashion-MNIST had been run without forcing single-threaded BLAS, while the candidates' cells and every reference lane force it. Thread count changes summation order, which can move nearest-neighbour ties and booster early stopping: v2 at `qd_dev`, $n = 10000$, seed 0 gives 0.1463 multi-threaded and 0.1467 in a single-threaded analysis refit (`ggbt` stopped at 166 and 123 rounds). An earlier analysis refit, with unrecorded thread settings, reproduces the cell's 0.1463 exactly, so the BLAS explanation fits the evidence but was not verified. The integration rule compared both image candidates with the same v2 numbers, so the choice between them (-0.078 against -0.071) is unaffected, and both are far from the -0.05 threshold. Every Part 10 held-out run, of v3, v2 and the references, is single-threaded. The note was added to the protocol at 02:03 (`a2b8470e`).

Memory and job control. Up to about ten jobs shared 4 cores and 16 GB. The builders ran at nice 0. Every reference and analysis process ran at nice ≥ 5, was resumable (per cell for the image evaluations, per target for the networks), and had a raised `oom_score_adj`, so that under memory pressure the kernel would kill it before a builder's process or v3's evaluation. Large fits ($n \geq 30000$) took a shared lock. The out-of-memory kills were:

- the baseline `mlp` on `qd_c`, at about 20:00–20:15 during the builders' evaluations, after its $n \leq 10000$ cells had finished;
- `p10_hier`'s `qd_dev` evaluation process (about 20:29, above);
- v2's held-out image evaluation, more than once between about 22:00 and 00:30. One kill, at about 00:24, happened while two image analysis fits were running; the analyst then held its own processes at the highest kill priority, and v2's held-out cells were moved to nice 0 and out of the killer's preference, since they feed the best-learner rule;
- reference processes while v3's $n = 60000$ cells ran: v1 on `qd_c`, `mlp_xl` on `qd_d`, `mlp_pool` and `cnn` on `qd_c`;
- `nets_cewd0.3` and `nets_cewd1.0` on `dlheld7` at about 02:40, with 6 of 12 targets saved each (both have since completed all 12).

Every killed job was rerun with the same code and seeds, keeping its finished cells or targets. The container restart at 22:40 also stopped every job lane; the lanes were resumed the same way.

A.4 The interpretability workflow

Analysts. Two analysts, one for the toy side and one for images, ran instrumented reruns of v3's (and, for comparison, v2's) seed-0 evaluation fits, with the learner's internals recorded (Section 6). The image analysis used `qd_dev`, `qd_a`, MNIST and Fashion-MNIST and opened no result on `qd_c` or `qd_d`. The toy analysis covers every toy suite, and the six sum-structured ones in detail, `dlheld7`

included; its `dlheld7` runs were made from 00:16 on 28 October, after v3's freeze. The work ran in parallel with the held-out evaluation and was committed at 03:54 on 28 October (`9214cfde`), before the best-learner rule was applied at 04:20. The rule is mechanical, and the analysis did not touch its inputs. The protocol fixed the analysis as descriptive, not as evidence that the learner is better than anything.

Verifications. Each side had two independent verifiers. One recomputed every number from the files with its own scripts; the other reviewed the readings against the files and the code. Table 17 counts the verdicts. "Corrected" means that the verifier changed a number, narrowed a statement or rejected a reading. Each verifier also listed further problems that were not tied to one claim.

side	verifier	claims	confirmed	corrected	further problems listed
toy	numbers	13	8	5	12
toy	reading	13	2	11	13
images	numbers	11	10	1	12
images	reading	11	2	9	11

Table 17: The verifications of the interpretability analyses. Source: `study/part10/interp/VERIFICATION.json`.

The corrections that change numbers in Section 6 are these:

- *The detection test for modulated blocks.* The analysis script tested a modulated block on a residual that was neither centred nor fitted on the modulator, and so counted the modulator character's own signal as block signal. A verifier recomputed it as DFG tests it. Seven gate and multiplexer targets became detectable later, which changes the count of targets detectable at $n = 128$ from 43 to 39, the median delay over the targets found from 1 doubling to 0, and Spearman's ρ over all 54 from -0.44 to -0.54.
- *Counts and scopes.* The beam and forced blocks move 5 pure symmetric targets plus one gate mixture, not 6 pure targets. The pooling-set locality fractions exclude the centre pixel. The share of evidence on blank pixels is not above one half for every class; several classes are listed as exceptions.

The corrections of readings are these:

- the "learned function of the block" is the whole DFG model's conditional mean, not the block's own coefficients;
- for fd_10 the pair statistics are small (about 0.002) but not zero;
- stacking-weight shares are not contributions to the prediction;
- split-gain shares must be read against column shares;
- the bars' straightness is a property of the construction and the data, not of the booster;
- the regions named in the contribution maps were chosen after looking at them.

Section 6 uses the corrected numbers and wording; where the claims check below found a correction that had not been carried over, it was applied. `make_interp.py` hard-codes the verifier's corrected detection points and regenerates every interpretability figure and table from the analysis files.

A.5 This paper and reproducibility

This paper. Its sections were drafted by the orchestrating session and by separate writing agents, each from the records it names, and then compiled together. The tables and figures are generated from the result files: `make_tables.py` and `make_figures.py` from `study/part10/LEARNER_RESULTS.json` and the result files it reads, and `make_interp.py` from the analysis files. `study/part10/LEARNER_RESULTS.json` is written by `study/part10/compare_learner.py`, whose docstring says which result file stands for v3 on each input type.

Claims check. After drafting, three independent AI checkers, each with its own instructions, checked the paper's claims on 28 October. One covered the two results sections, one the interpretability section, and one the abstract, the introduction, Sections 2, 3 and 7 and this appendix. Each compared the text with the result files, the records named in it and the learner code, recomputed what it could, and listed each problem with a severity (error, misleading, minor) and a proposed correction. They found stale counts from runs that had landed after drafting, a few arithmetic and transcription errors (for example the `cnn`'s relative error on `qd_a`, and the order of HELDOUT2 and v1's freeze), several verifier corrections of the interpretability analyses that had not been carried over, statements broader than the evidence (for example "fail because" for a descriptive seed-0 analysis), and details of the learner description that did not match the code. Their corrections were applied before the final regeneration of every table and figure, with counts updated for the runs that landed in the meantime (`random` for v3, the remaining `dlheld7` network targets, and the image references on `qd_c` and `qd_d`). The check is not an audit of leakage, compute or reference strength of the kind made in Parts 8 and 9.

Regenerating the numbers. From `gd-circuit-growth/`:

- `cd study/part10 && python3 compare_part10.py` (the rules; writes `RESULTS.md`);
- `python3 compare_learner.py` (every comparison; writes `LEARNER_RESULTS.{md,json}`);
- `cd ../../paper_learner`, then `python3 make_tables.py`, `python3 make_figures.py`, `python3 make_interp.py` and `python3 check_toy_identity.py`;
- `latexmk -pdf main.tex`.

The result files are:

- `<learner>__<dataset>.json`, with `.info.json` records, in `study/standard_tasks/results/`;
- `toy__<learner>__<suite>.json` in `study/hybrid/results/`.

What is not in the repository. The job drivers and queue scripts of Part 10, the builders' development scripts and logs, and the analyses' raw outputs stayed in the session scratch directory. The drivers only wrap the committed scripts, the builders' notes give the commands of their evaluations, and copies of the analysis outputs that the paper uses are in `study/part10/interp/results/`.